



Institute of Information Technology

Jahangirnagar University

Savar, Dhaka-1342

LAB REPORT

Course Code: ICT-3102

Course Title: Operating System Lab

Lab Report: 1

Lab Report Name: Installing Ubuntu on VirtualBox

Submitted to:

Mehrin Anannya

Assistant Professor

Institute of Information Technology

Jahangirnagar University

Submitted by:

Md. Shaon Khan

ID: 1984

Session: 2022-2023

Batch: 52

Lab Report

Installing Ubuntu on VirtualBox

Course: Operating System Lab

Introduction

In this lab, I installed Ubuntu Linux on a virtual machine using Oracle VirtualBox 7.2.6. VirtualBox is a free software that lets us run another operating system inside our current operating system. I used Windows as my host OS and installed Ubuntu 22.04.1 (64-bit) as the guest OS. The main goal of this lab was to learn how to set up VirtualBox, create a new virtual machine, configure its hardware, and install Ubuntu from an ISO image. I followed all the steps one by one and took screenshots at each important step. This report shows those screenshots with a short description for each one.

Steps with Screenshots



Figure 1: VirtualBox Platform Packages Download Page

This page shows the download options for VirtualBox 7.2.6. It has different packages for Windows, macOS, Linux, and Solaris. I selected the Windows hosts option to download VirtualBox for my Windows computer.

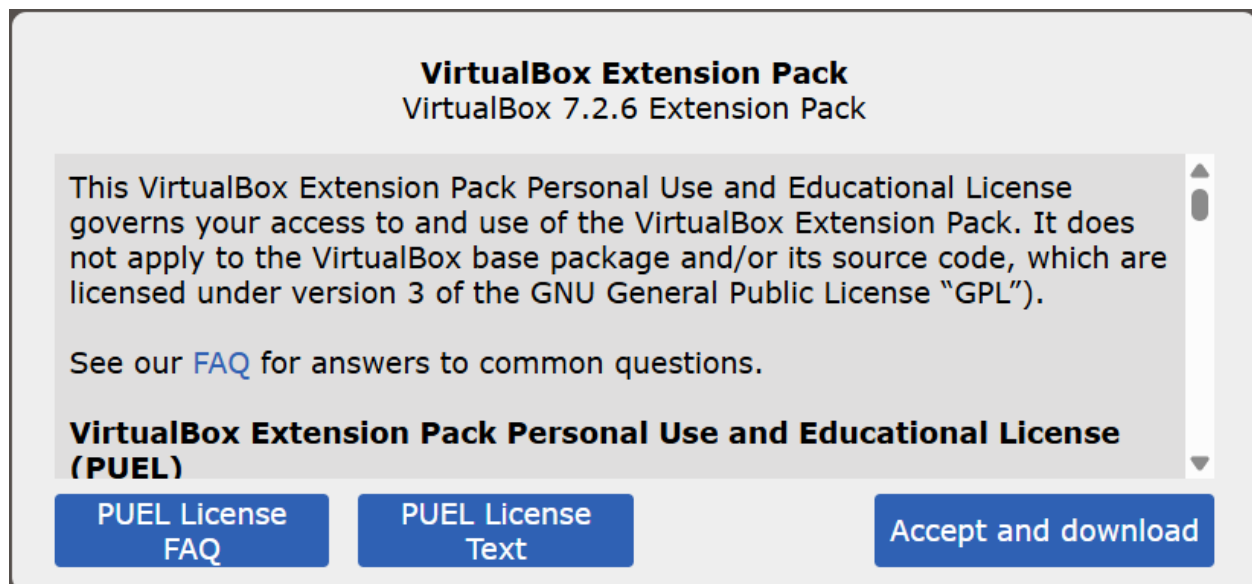


Figure 2: VirtualBox Extension Pack License Page

This is the VirtualBox 7.2.6 Extension Pack download page. It shows the license agreement for personal and educational use. I clicked "Accept and download" to download the extension pack.

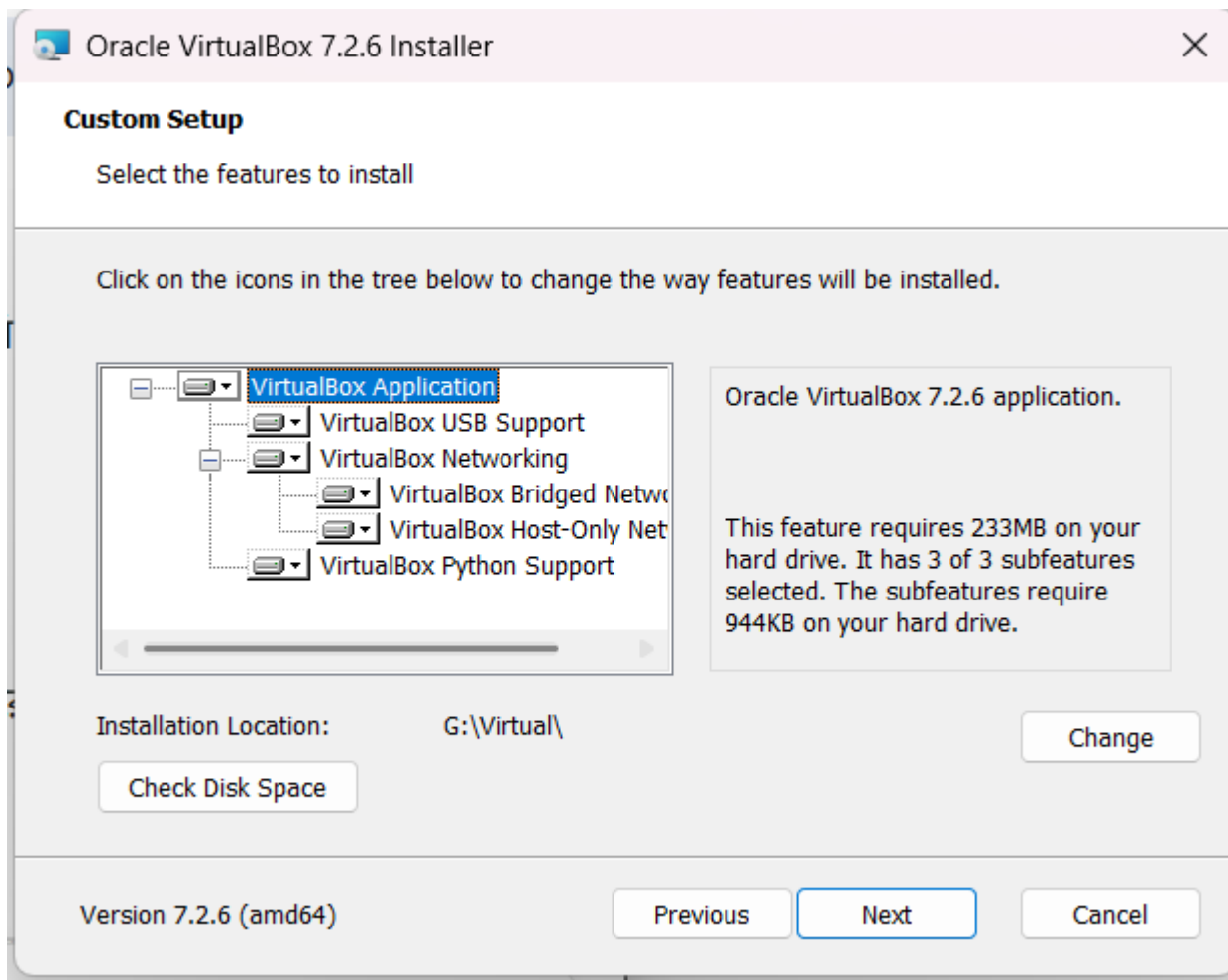


Figure 3: Custom Setup – First Directory Selection

This is the Custom Setup screen. It shows all the features available to install such as VirtualBox USB Support, Networking, Bridged Network, Host-Only Network, and Python Support. Here I selected G:\Virtual\ as the installation location.

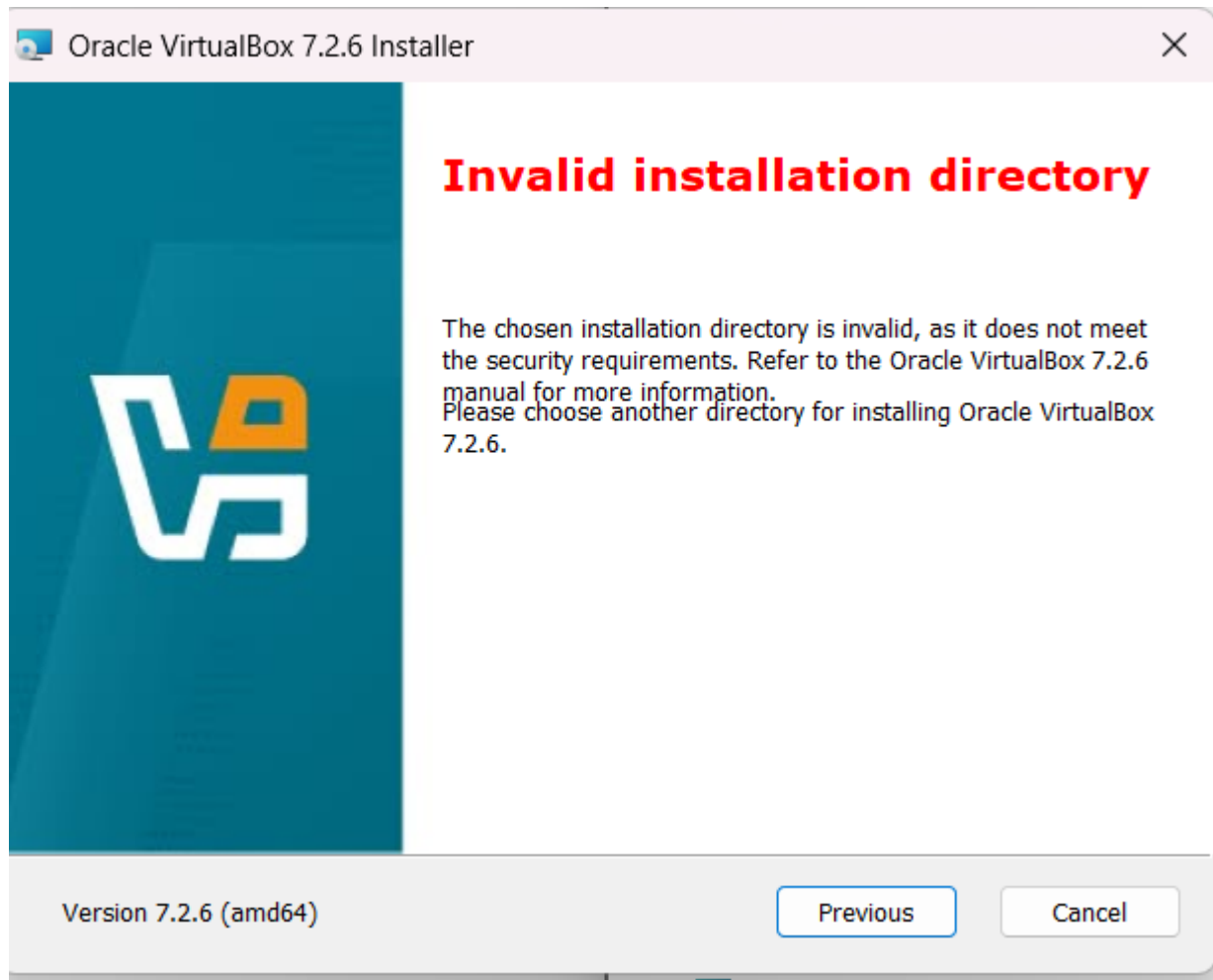


Figure 4: Invalid Installation Directory Error

When I first tried to install VirtualBox, this error appeared. It said the chosen installation directory is invalid because it does not meet security requirements. I had to go back and choose a different directory.

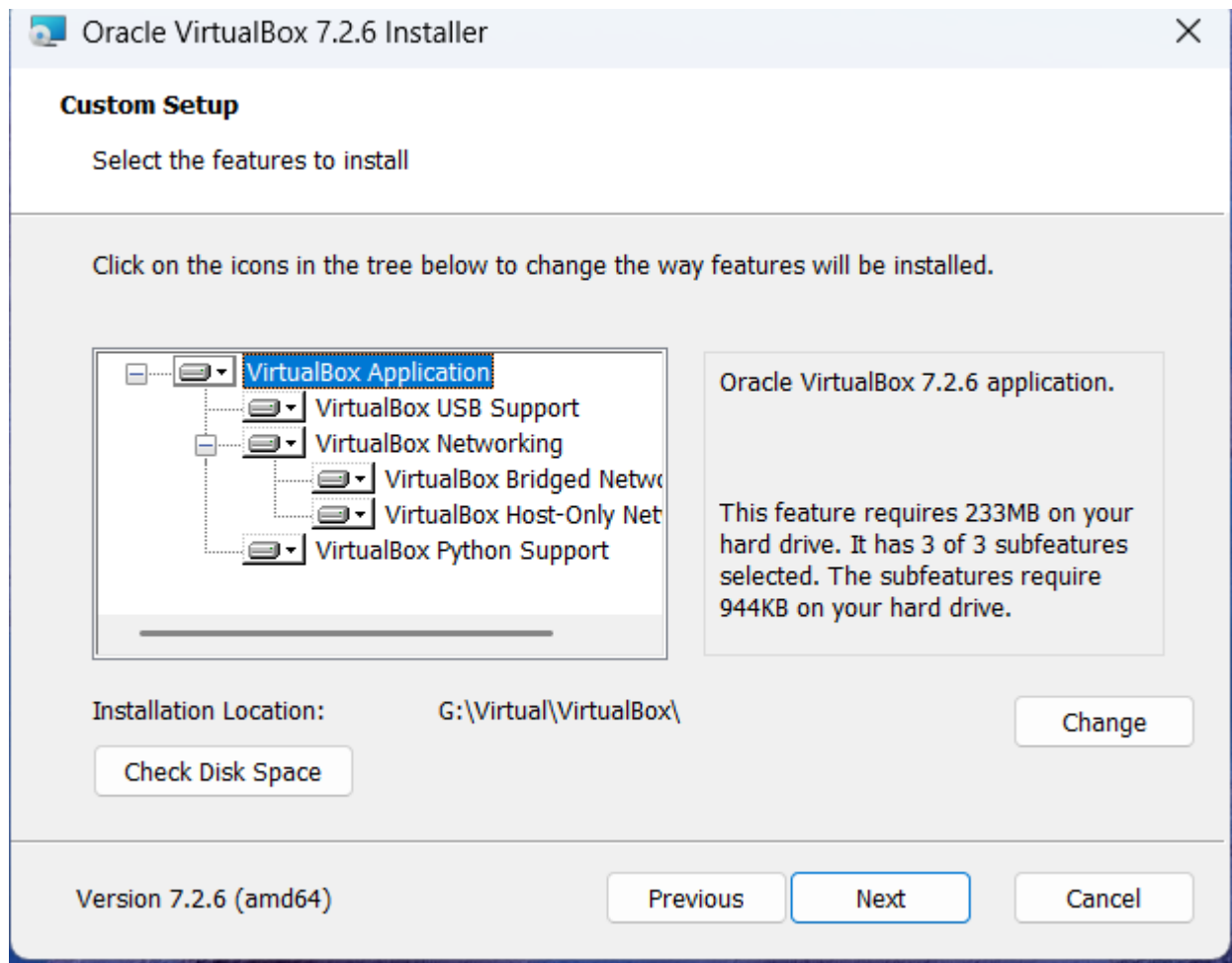


Figure 5: Custom Setup – Corrected Installation Directory

After fixing the directory error, I changed the installation path to G:\Virtual\VirtualBox\. This time the installer accepted the location. I clicked Next to continue with the installation.

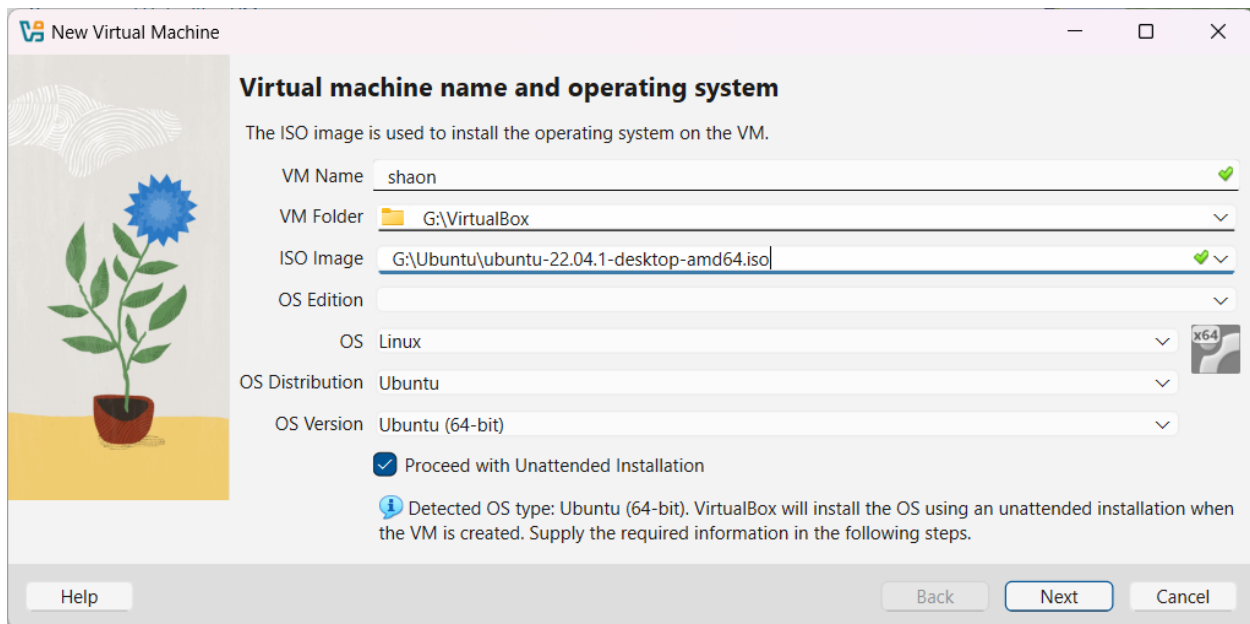


Figure 6: New Virtual Machine – Name and OS Selection

This screen is for creating a new virtual machine. I entered the VM name as "shaon" and set the folder to G:\VirtualBox. I also selected the Ubuntu 22.04.1 desktop ISO image. VirtualBox automatically detected the OS type as Ubuntu 64-bit.

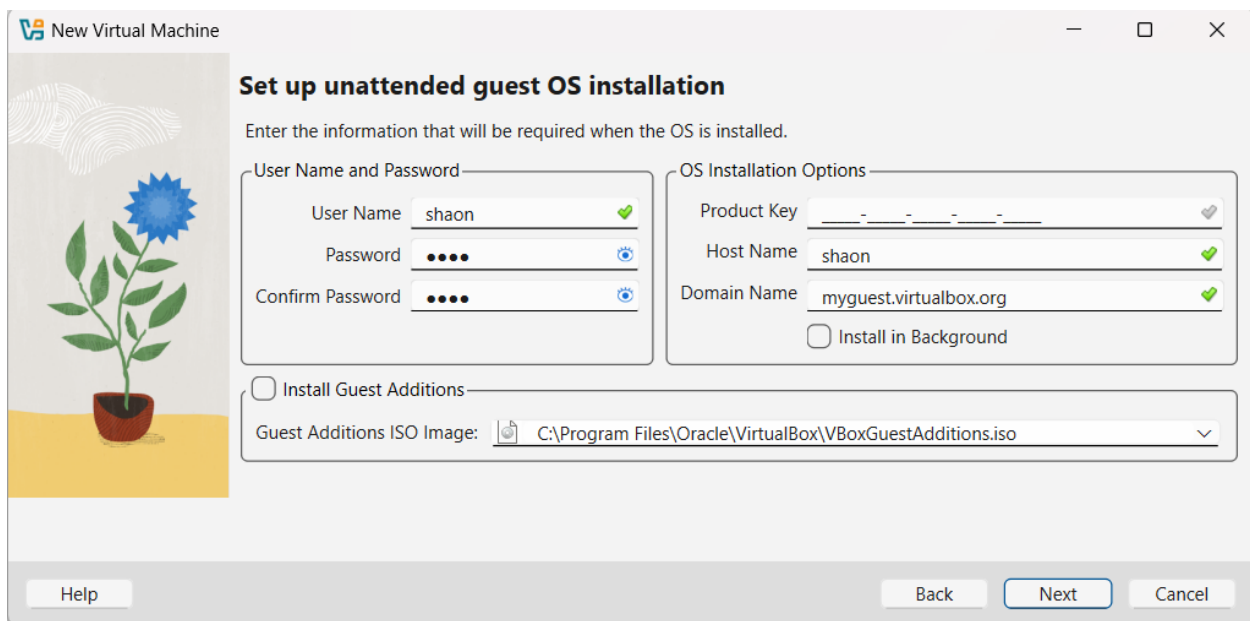


Figure 7: Unattended Guest OS Installation Setup

Here I set up the username and password for the Ubuntu installation. I entered "shaon" as the username and set a password. The host name was also set to "shaon". This step lets VirtualBox install Ubuntu automatically without needing manual input during setup.

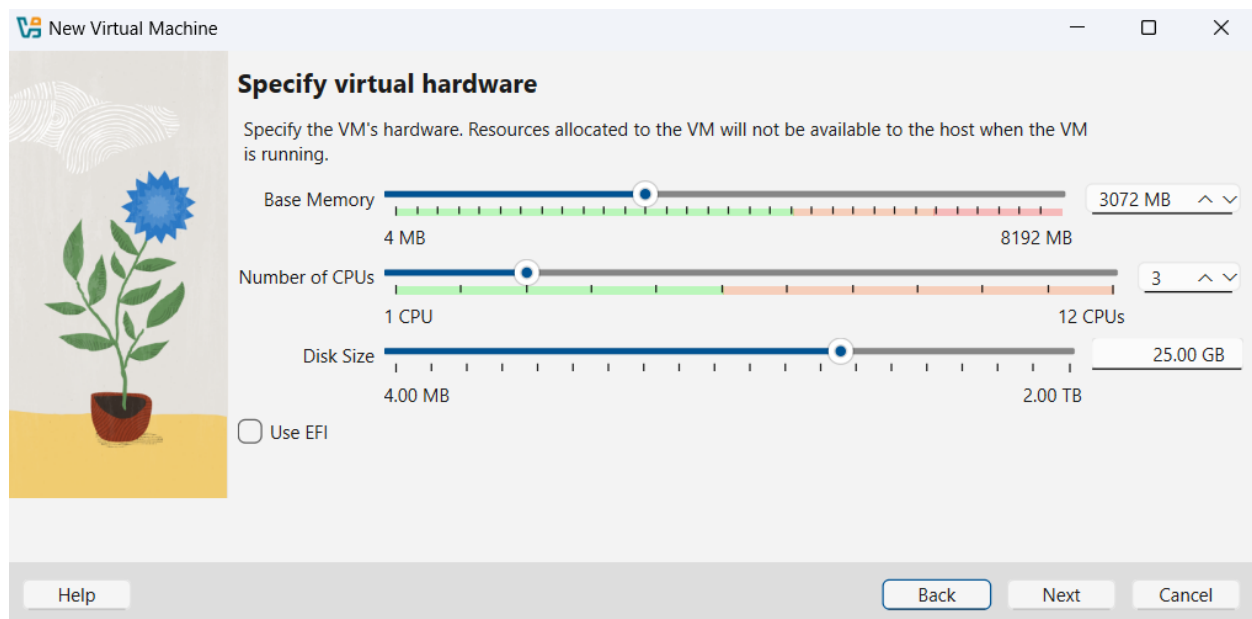


Figure 8: Virtual Hardware Configuration

This screen is for setting the virtual hardware. I assigned 3072 MB of RAM, 3 CPUs, and 25 GB of disk space for the virtual machine. These settings decide how much of the host computer resources the virtual machine can use.

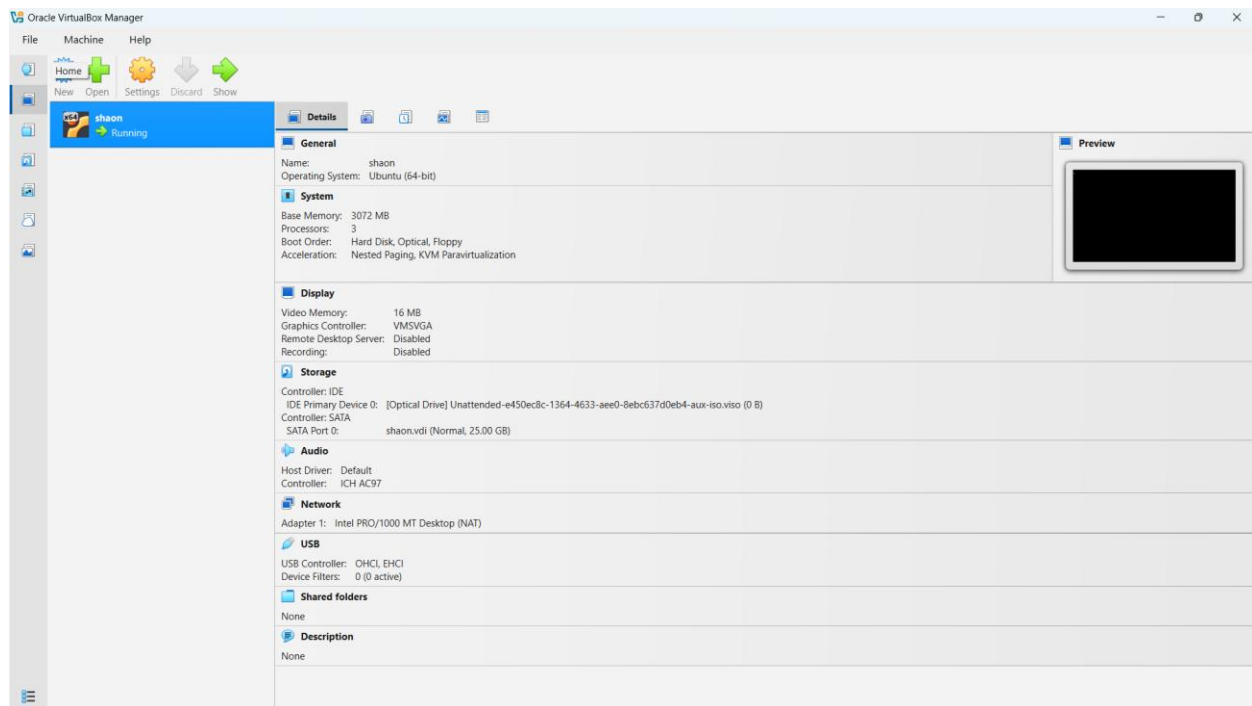


Figure 9: VirtualBox Manager – VM Details Overview

This is the Oracle VirtualBox Manager window showing the full details of the created virtual machine named "shaon". It shows the system info, storage, network adapter, audio, and USB settings. The VM status shows it is running.

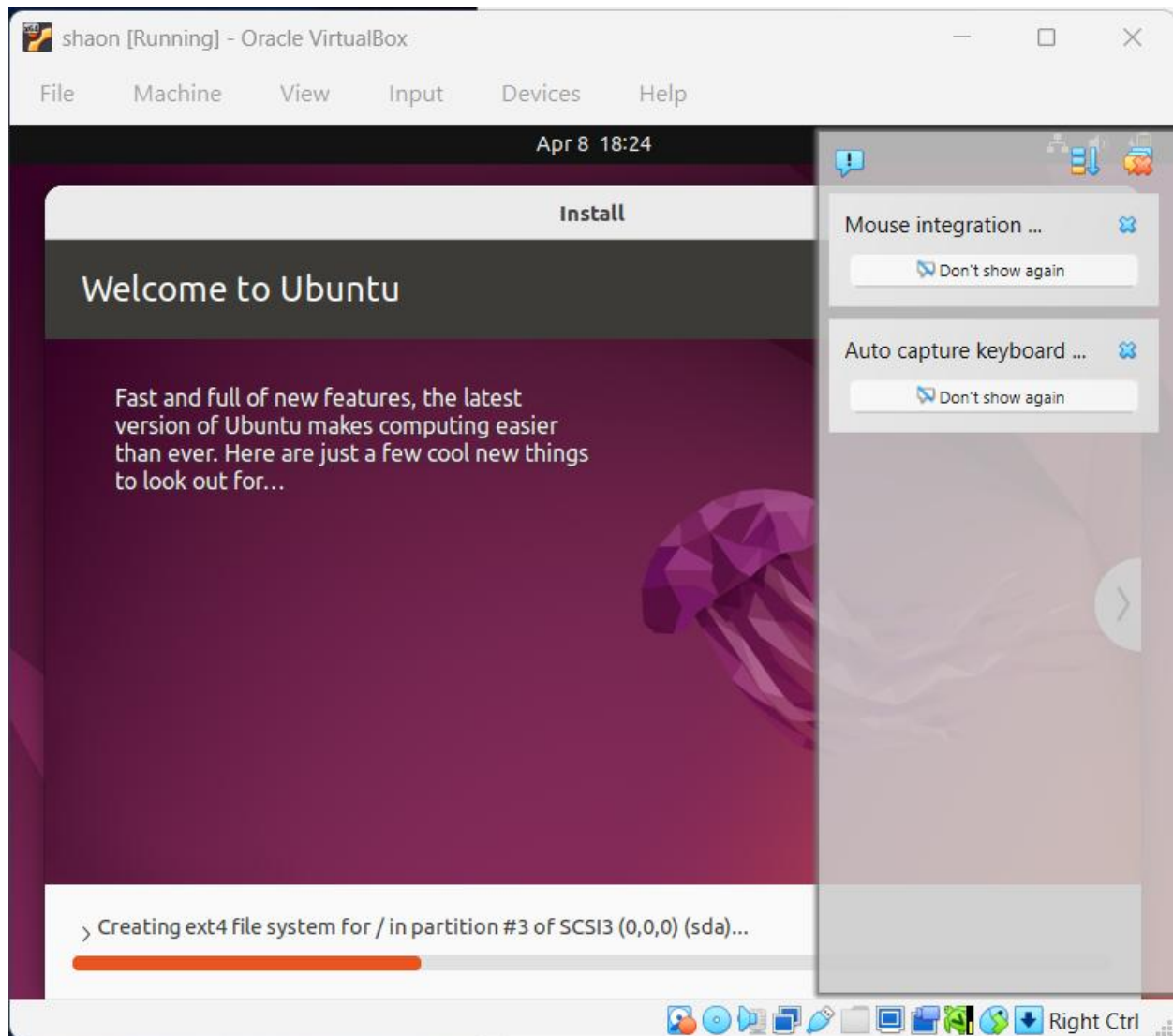


Figure 10: Ubuntu Installation in Progress

The virtual machine is now running and Ubuntu is being installed. The screen shows the "Welcome to Ubuntu" message and at the bottom it shows the progress – it is creating the ext4 file system on the virtual disk. The installation was running automatically.

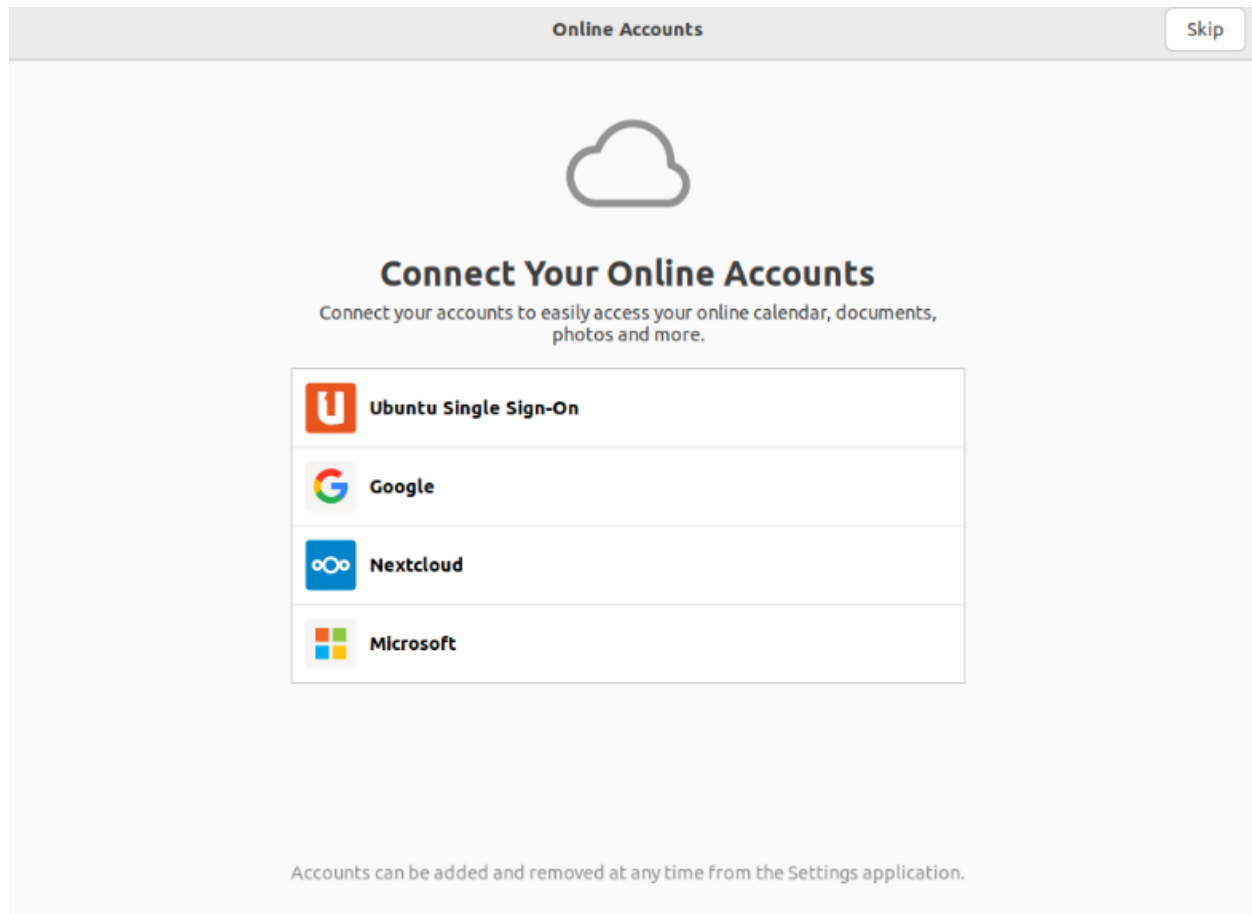


Figure 11: Ubuntu Online Accounts Setup Screen

After Ubuntu finished installing, this screen appeared asking to connect online accounts. It gives options to connect Ubuntu Single Sign-On, Google, Nextcloud, or Microsoft accounts.

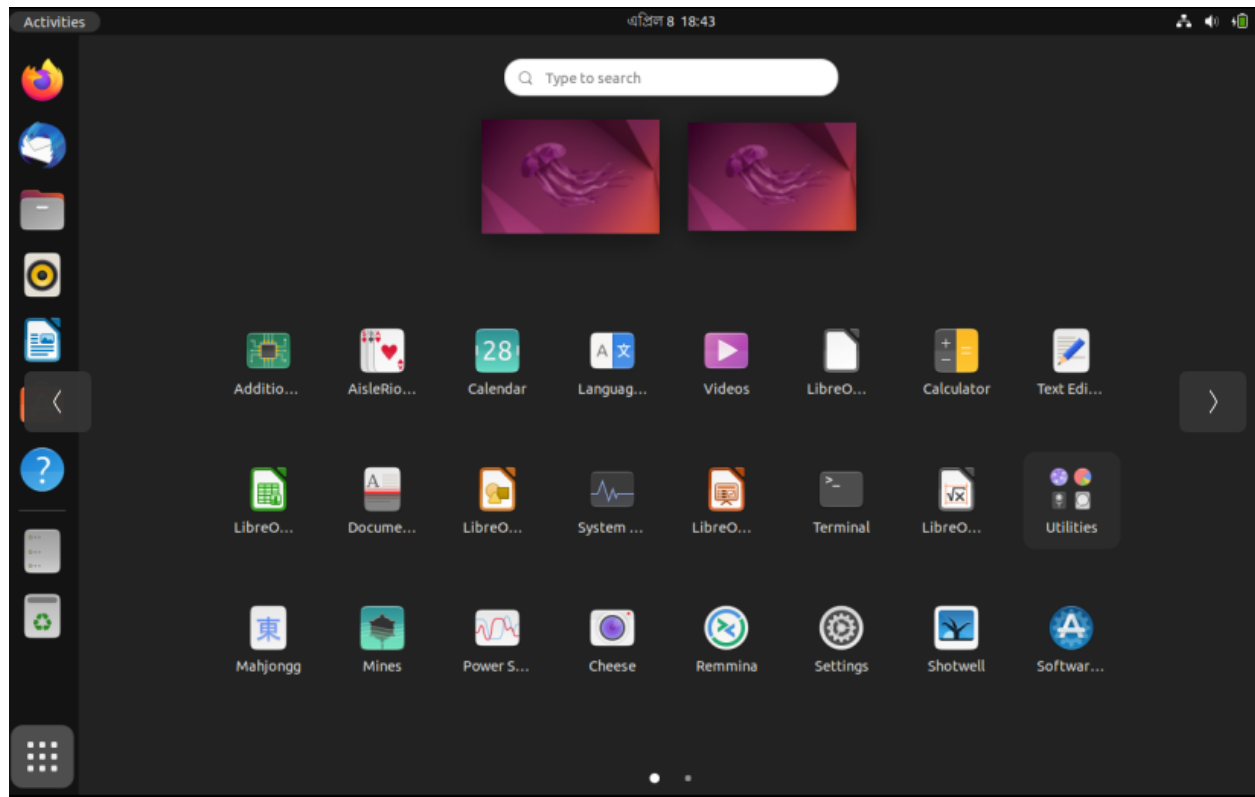


Figure 12: Ubuntu App Drawer – Installed Applications

This is the Ubuntu application drawer showing all the installed apps. I can see apps like LibreOffice, Calendar, Terminal, Calculator, Settings, and many more. This confirms that Ubuntu is fully installed and working inside VirtualBox.

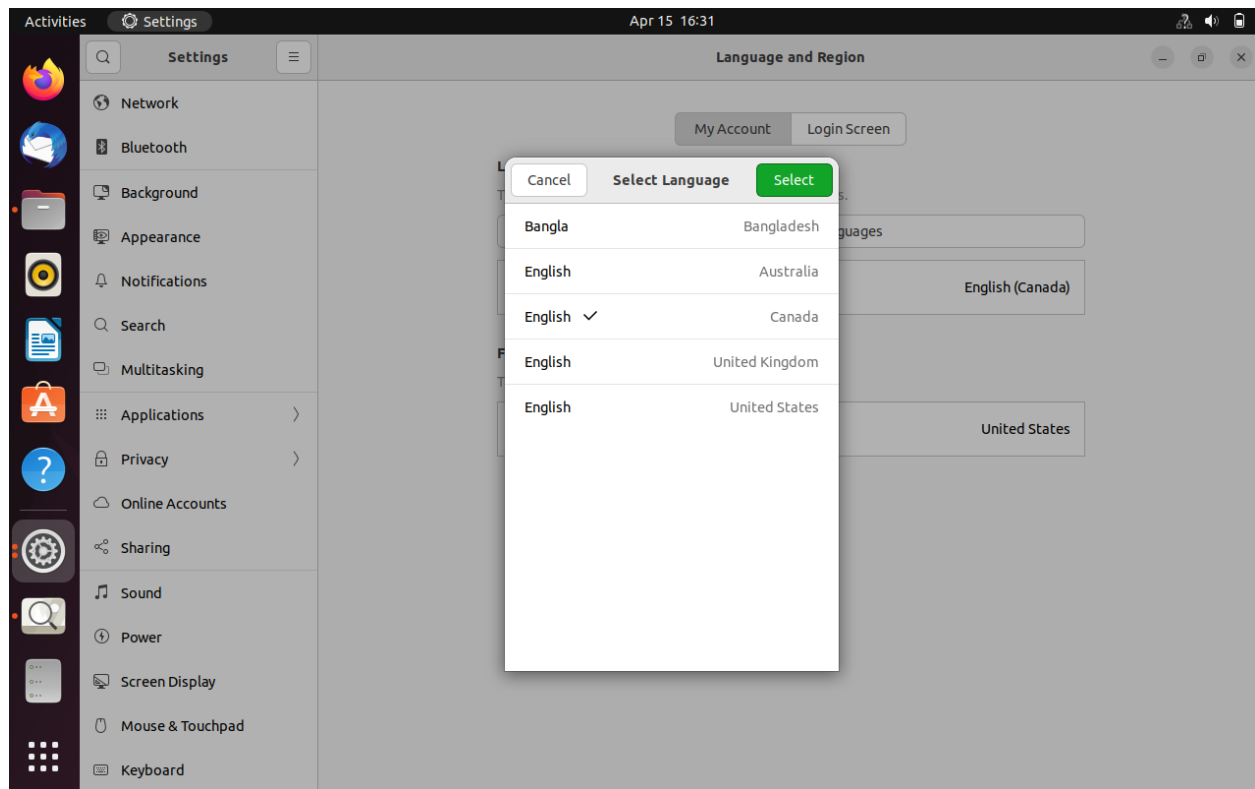


Figure 13: Language and Region Settings – Language Selection

This screen shows the Language and Region settings inside Ubuntu. I opened the language selection dialog and can see options like Bangla (Bangladesh), English (Australia), English (Canada), English (United Kingdom), and English (United States). I was checking the language settings after installation.

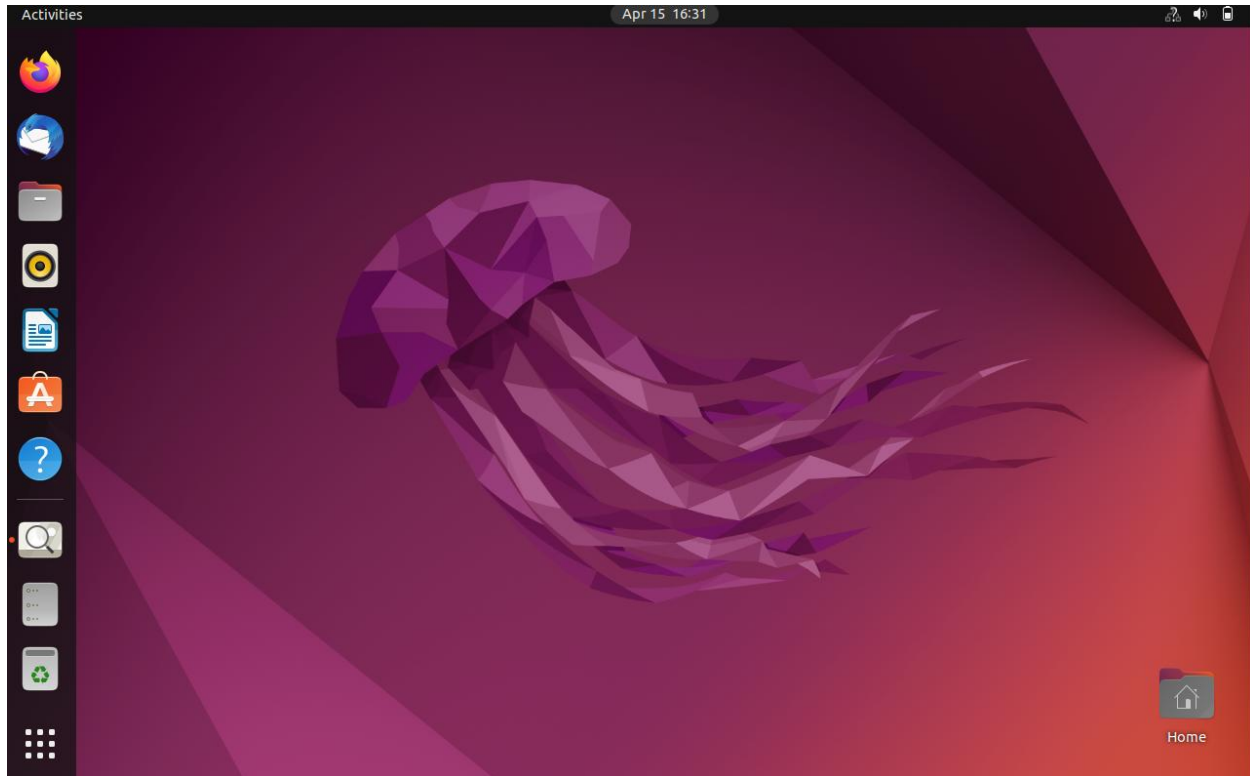


Figure 14: Ubuntu Desktop – Final View

This is the final Ubuntu desktop screen. The GNOME desktop is fully loaded with the default Ubuntu 22.04 jellyfish wallpaper. The taskbar on the left shows icons for Firefox, file manager, and other apps. This confirms that Ubuntu is successfully installed and running inside

Conclusion

I was able to successfully install Ubuntu 22.04.1 inside VirtualBox on my Windows computer. The process was a bit long but not very hard. First I downloaded VirtualBox and the Extension Pack. Then I created a new virtual machine and set the RAM, CPU, and disk size. After that I attached the Ubuntu ISO file and started the installation. VirtualBox installed Ubuntu automatically without needing me to click anything during the install. After the installation finished, I could see the Ubuntu desktop working properly inside VirtualBox. I also checked the app drawer and the language settings to make sure everything was fine. From this lab I learned how to use VirtualBox and how virtual machines work in general. It is a very useful tool for testing operating systems without changing anything on the main computer.



Institute of Information Technology

Jahangirnagar University, Savar, Dhaka-1342

Lab Report - 02

Lab Title: Linux File Commands (File Management & Directory Management)

Course Code: ICT-3102

Course Title: Operating System Lab

Submitted by:

Md. Shaon Khan

Id: 1984

Batch: 52 Batch

Session: 2022-2023

Supervised by:

Mehrin Anannya

Assistant Professor, Institute of Information Technology

Jahangirnagar University

Date: April 27, 2026

1. Objective

In this lab, we practiced frequently used Linux commands for file and directory management. The goal was to get comfortable with the Linux terminal by running real commands and observing what each one does. We learned how to list files, navigate directories, create and delete files, and view file contents.

2. Theory

Linux is a command-line-based operating system where most operations are done through a terminal. Unlike graphical interfaces, the terminal gives more control over the system. In this lab, we used the Ubuntu terminal to run various file management commands.

Linux treats everything as a file including directories. Commands like `ls`, `cd`, `mkdir`, `cat`, `touch`, and `rm` are fundamental tools every Linux user needs to know. These commands are simple but very powerful when combined together.

We also learned that Linux uses a tree-like file structure starting from the root (`/`). Each user has a home directory (e.g., `/home/shaon`) and this is where we worked throughout the lab.

3. Commands Used

<code>ls</code>	Lists all files and directories in the current folder.
<code>ls -al</code>	Shows a detailed list including hidden files (files starting with a dot).
<code>ls -lt</code>	Lists files sorted by modification time, newest first.
<code>cd <dir></code>	Changes the current directory to the specified one.
<code>cd -</code>	Goes back to the previously visited directory.
<code>cd ..</code>	Moves up one level to the parent directory.
<code>cd</code>	Goes directly to the home directory.
<code>pwd</code>	Prints the full path of the current working directory.
<code>mkdir <dir></code>	Creates a new directory with the given name.

<code>cat > <file></code>	Creates a new file and lets the user type content into it.
<code>echo "text" >> <file></code>	Appends a line of text to an existing file.
<code>more <file></code>	Shows the file content page by page.
<code>head <file></code>	Displays the first 10 lines of a file.
<code>tail <file></code>	Displays the last 10 lines of a file.
<code>touch <file></code>	Creates an empty file or updates the timestamp of an existing file.
<code>clear</code>	Clears the terminal screen.
<code>rm <file></code>	Deletes the specified file.
<code>rm -r <dir></code>	Deletes a directory along with everything inside it.
<code>rm -f <file></code>	Force deletes a file without asking for confirmation.
<code>rm -rf <dir></code>	Force deletes a directory and all its contents recursively.

4. Procedure

We opened the terminal on Ubuntu and ran each command one by one. For every command, we first ran it, then observed the output and noted what happened. Below is a step-by-step description of what we did.

4.1 ls Directory Listing

Lists all non-hidden files and folders in the current directory.

Command:

```
$ ls
```

Output:

```
Desktop  Documents  Downloads  Music  Pictures  Projects  Public  snap
Templates  Videos
```

We saw all the directories inside the home folder. Hidden files were not visible with plain ls.

4.2 ls -al Detailed Listing with Hidden Files

Shows a detailed list including file permissions, owner, size, date, and hidden files.

Command:

```
$ ls -al
```

Output:

```
total 88
drwxr-x--- 17 shaon shaon 4096 Apr 22 20:31 .
-rw----- 1 shaon shaon 1510 Apr 22 20:29 .bash_history
... (and more files including hidden ones starting with .)
```


We noticed that hidden files like `.bashrc`, `.bash_history`, and `.profile` appeared here but not in the plain `ls` output.

4.3 `ls -lt` Sort by Modification Time

Lists files and directories sorted by last modified time, newest first.

Command:

```
$ ls -lt
```

Output:

```
total 40
drwxr-xr-x 2 shaon shaon 4096 Apr 22 20:31 Downloads
drwxrwxr-x 2 shaon shaon 4096 Apr 15 14:45 Projects
...
```

The Downloads folder appeared at the top because it was modified most recently. This is useful when you need to quickly find recently changed files.

4.4 `cd` Change Directory

Changes the working directory to the specified folder.

```
$ cd Documents
$ cd -          (back to previous)
$ cd ..        (go up one level)
$ cd           (go to home)
```

Output:

```
/home/shaon
```

We moved into Documents, then used `cd -` to go back to where we were. The `cd ..` moved us up one level. Just typing `cd` brought us back home instantly.

4.5 `pwd` Print Working Directory

Displays the full path of the current directory.

Command:

```
$ pwd
```

Output:

```
/home/shaon
```

This was helpful when we were not sure which directory we were currently in. It always shows the absolute path.

4.6 mkdir Make Directory

Creates a new directory with the given name.

Command:

```
$ mkdir Projects
```

Output:

```
mkdir: cannot create directory 'Projects': File exists (if already exists)
```

Otherwise, no output – directory is created silently.

When we tried to create Projects again, it gave an error saying it already exists. Linux does not overwrite existing directories with mkdir.

4.7 cat > file Create File with Input

Creates a new file and allows typing content directly from keyboard. Press Ctrl+D to save.

Command:

```
$ cat > notes.txt
Hello, This is test file,
created by Shaon Khan
Who are dreamer
want success and try to work hard.
[Ctrl+D]
```

Output:

```
(File notes.txt is created with the typed content)
```

We typed multiple lines and pressed Ctrl+D to finish. The file was saved immediately. We then used cat notes.txt to verify the content.

4.8 echo >> Append to File

Appends a new line of text to an existing file without overwriting it.

Command:

```
$ echo "This is new line which appends at the last of previous text"
>> notes.txt
```

Output:

```
(No output – text is appended silently to notes.txt)
```

After running this, we used `more notes.txt` to see that the new text was added at the end of the existing content.

4.9 more View File Page by Page

Displays file content one screen at a time. Press Space to continue, q to quit.

Command:

```
$ more notes.txt
```

Output:

```
Hello, This is test file,  
created by Shaon Khan  
Who are dreamer  
want success and try to work hard.This is new line...
```

The `more` command was useful when the file was long. It showed content page by page, which was easier to read than `cat`.

4.10 head First 10 Lines

Prints the first 10 lines of the file.

Command:

```
$ head notes.txt
```

Output:

```
Hello, This is test file,  
created by Shaon Khan  
Who are dreamer  
want success and try to work hard.This is new line...  
(first 10 lines shown)
```

The `head` command is handy when a file is very large and we only need to check what is at the beginning.

4.11 tail Last 10 Lines

Prints the last 10 lines of the file.

Command:

```
$ tail notes.txt
```

Output:

```
10
```

```
11
12
13
14
14
15
16
17
18
```

The tail command showed the final 10 lines. We used it after adding many numbered lines to the file using echo -e.

4.12 touch Create Empty File

Creates an empty file. If the file already exists, it updates its timestamp.

Command:

```
$ touch hello.txt
```

Output:

```
(No output - hello.txt is created empty)
```

We used ls after touch to confirm the file was created. The file was empty, which we verified with cat hello.txt.

4.13 rm Remove File

Deletes the specified file permanently.

Command:

```
$ rm hello.txt
```

Output:

```
(No output - file is deleted)
```

After running rm, we did ls again and confirmed hello.txt was gone. There is no trash or undo in Linux, so we need to be careful with rm.

4.14 rm -r Remove Directory

Deletes a directory and all its contents recursively.

Command:

```
$ rm -r Projects
```

Output:

```
rm: cannot remove 'Projects': No such file or directory (if already removed)
Otherwise removes silently.
```

We first tried to remove Projects while we were inside it, which gave an error. After going up (cd), it worked correctly.

4.15 rm -f Force Delete File

Force deletes a file without any confirmation prompt.

Command:

```
$ rm -f notes.txt
```

Output:

```
(No output — file deleted without asking)
```

Unlike rm, this does not prompt at all. It is useful when you want to remove write-protected files quickly.

4.16 rm -rf Force Delete Directory

Force deletes a directory and all its contents without any prompts.

Command:

```
$ rm -rf Pro
```

Output:

```
(No output — directory and all its contents deleted)
```

This command is very powerful and dangerous. We created a test directory Pro, then removed it completely in one command.

4.17 clear Clear the Terminal

Clears all previous output from the terminal screen.

Command:

```
$ clear
```

Output:

```
(Terminal screen is cleared)
```

After running many commands, the screen was cluttered. The clear command made it clean again. The command history was still accessible with the up arrow key.

5. Discussion

In this lab, we ran 18 different Linux commands and observed how each one works. A few things stood out to us during the session.

First, we noticed that `ls` and `ls -al` show very different results. The plain `ls` command only shows visible files, while `ls -al` reveals hidden configuration files like `.bashrc` and `.bash_history`. These hidden files are important for system settings.

Second, we learned that the `>>` operator appends text to a file without overwriting it, while `>` overwrites. This is an important distinction — using the wrong one can delete existing content.

Third, the `rm` command is permanent in Linux. There is no recycle bin. So we had to be careful, especially with `rm -rf`, which deletes everything in a directory silently.

We also noticed that `echo -e` allows us to use escape characters like `\n` for newlines. When we added multiple numbered lines to `notes.txt`, we could then verify them using `head` and `tail` commands, which are very efficient for checking large files.

Overall, this lab helped us understand how the Linux terminal handles files and directories. The commands are simple but require careful usage.

6. Conclusion

We successfully completed Lab 2 of the Operating System Lab course. In this session, we practiced all the fundamental Linux file and directory management commands. We created files, wrote content, appended data, viewed files in different ways, and removed files and directories.

Through this lab, we gained practical experience with the Linux terminal. We now feel more confident using the command line for everyday file operations. The knowledge gained here will be helpful for more advanced Linux and OS topics in future labs.

7. Lab Screenshots

The following screenshots were captured during the lab session and show the actual terminal output for the commands we ran.

```
shaon@shaon:~$ ls
Desktop Documents Downloads Music Pictures Projects Public snap Templates Videos
shaon@shaon:~$ ls -al
total 88
drwxr-x--- 17 shaon shaon 4096 Apr 22 20:31 .
drwxr-xr-x  3 root  root  4096 Apr  8 18:31 ..
-rw-----  1 shaon shaon 1510 Apr 22 20:29 .bash_history
-rw-r--r--  1 shaon shaon  220 Apr  8 18:31 .bash_logout
-rw-r--r--  1 shaon shaon 3771 Apr  8 18:31 .bashrc
drwx----- 16 shaon shaon 4096 Apr  8 18:55 .cache
drwx----- 16 shaon shaon 4096 Apr 15 14:50 .config
drwxr-xr-x  2 shaon shaon 4096 Apr  8 18:40 Desktop
drwxr-xr-x  2 shaon shaon 4096 Apr  8 18:40 Documents
drwxr-xr-x  2 shaon shaon 4096 Apr 22 20:31 Downloads
drwx-----  2 shaon shaon 4096 Apr  8 18:44 .gnupg
drwx-----  3 shaon shaon 4096 Apr  8 18:40 .local
drwxr-xr-x  2 shaon shaon 4096 Apr  8 18:40 Music
-rw-r--r--  1 shaon shaon  361 Apr 15 14:42 .pam_environment
drwxr-xr-x  3 shaon shaon 4096 Apr  8 18:40 Pictures
-rw-r--r--  1 shaon shaon  807 Apr  8 18:31 .profile
drwxrwxr-x  2 shaon shaon 4096 Apr 15 14:45 Projects
drwxr-xr-x  2 shaon shaon 4096 Apr  8 18:40 Public
drwx-----  4 shaon shaon 4096 Apr  8 18:50 snap
drwx-----  2 shaon shaon 4096 Apr  8 18:44 .ssh
drwxr-xr-x  2 shaon shaon 4096 Apr  8 18:40 Templates
drwxr-xr-x  2 shaon shaon 4096 Apr  8 18:40 Videos
shaon@shaon:~$ ls -lt
total 40
drwxr-xr-x 2 shaon shaon 4096 Apr 22 20:31 Downloads
drwxrwxr-x 2 shaon shaon 4096 Apr 15 14:45 Projects
drwx----- 4 shaon shaon 4096 Apr  8 18:50 snap
drwxr-xr-x 3 shaon shaon 4096 Apr  8 18:40 Pictures
drwxr-xr-x 2 shaon shaon 4096 Apr  8 18:40 Desktop
drwxr-xr-x 2 shaon shaon 4096 Apr  8 18:40 Documents
drwxr-xr-x 2 shaon shaon 4096 Apr  8 18:40 Music
```

Screenshot 1: ls and ls -al commands showing directory listing and hidden files


```

shaon@shaon:~$ ls -lt
total 40
drwxr-xr-x 2 shaon shaon 4096 Apr 22 20:31 Downloads
drwxrwxr-x 2 shaon shaon 4096 Apr 15 14:45 Projects
drwx----- 4 shaon shaon 4096 Apr  8 18:50 snap
drwxr-xr-x 3 shaon shaon 4096 Apr  8 18:40 Pictures
drwxr-xr-x 2 shaon shaon 4096 Apr  8 18:40 Desktop
drwxr-xr-x 2 shaon shaon 4096 Apr  8 18:40 Documents
drwxr-xr-x 2 shaon shaon 4096 Apr  8 18:40 Music
drwxr-xr-x 2 shaon shaon 4096 Apr  8 18:40 Public
drwxr-xr-x 2 shaon shaon 4096 Apr  8 18:40 Templates
drwxr-xr-x 2 shaon shaon 4096 Apr  8 18:40 Videos
shaon@shaon:~$ cd Documents
shaon@shaon:~/Documents$ ls
shaon@shaon:~/Documents$ cd -
/home/shaon
shaon@shaon:~$ cd ..
shaon@shaon:/home$ cd
shaon@shaon:~$

```

Screenshot 2: `ls -lt` and `cd` commands sorted listing and directory navigation

```

shaon@shaon:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Projects  Public  snap  Templates  Videos
shaon@shaon:~$ ls -al
total 88
drwxr-x--- 17 shaon shaon 4096 Apr 22 20:31 .
drwxr-xr-x  3 root  root  4096 Apr  8 18:31 ..
-rw-----  1 shaon shaon 1510 Apr 22 20:29 .bash_history
-rw-r--r--  1 shaon shaon  220 Apr  8 18:31 .bash_logout
-rw-r--r--  1 shaon shaon 3771 Apr  8 18:31 .bashrc
drwx----- 16 shaon shaon 4096 Apr  8 18:55 .cache
drwx----- 16 shaon shaon 4096 Apr 15 14:50 .config
drwxr-xr-x  2 shaon shaon 4096 Apr  8 18:40 Desktop
drwxr-xr-x  2 shaon shaon 4096 Apr  8 18:40 Documents
drwxr-xr-x  2 shaon shaon 4096 Apr 22 20:31 Downloads
drwx-----  2 shaon shaon 4096 Apr  8 18:44 .gnupg
drwx-----  3 shaon shaon 4096 Apr  8 18:40 .local
drwxr-xr-x  2 shaon shaon 4096 Apr  8 18:40 Music
-rw-r--r--  1 shaon shaon  361 Apr 15 14:42 .pam_environment
drwxr-xr-x  3 shaon shaon 4096 Apr  8 18:40 Pictures
-rw-r--r--  1 shaon shaon  807 Apr  8 18:31 .profile
drwxrwxr-x  2 shaon shaon 4096 Apr 15 14:45 Projects
drwxr-xr-x  2 shaon shaon 4096 Apr  8 18:40 Public
drwx-----  4 shaon shaon 4096 Apr  8 18:50 snap
drwx-----  2 shaon shaon 4096 Apr  8 18:44 .ssh
drwxr-xr-x  2 shaon shaon 4096 Apr  8 18:40 Templates
drwxr-xr-x  2 shaon shaon 4096 Apr  8 18:40 Videos
shaon@shaon:~$ ls -lt
total 40
drwxr-xr-x 2 shaon shaon 4096 Apr 22 20:31 Downloads
drwxrwxr-x 2 shaon shaon 4096 Apr 15 14:45 Projects
drwx----- 4 shaon shaon 4096 Apr  8 18:50 snap
drwxr-xr-x 3 shaon shaon 4096 Apr  8 18:40 Pictures
drwxr-xr-x 2 shaon shaon 4096 Apr  8 18:40 Desktop
drwxr-xr-x 2 shaon shaon 4096 Apr  8 18:40 Documents
drwxr-xr-x 2 shaon shaon 4096 Apr  8 18:40 Music

```

Screenshot 3: `ls -al` and `ls -lt` command outputs


```

drwxr-xr-x 2 shaon shaon 4096 Apr  8 18:40 Public
drwxr-xr-x 2 shaon shaon 4096 Apr  8 18:40 Templates
drwxr-xr-x 2 shaon shaon 4096 Apr  8 18:40 Videos
shaon@shaon:~$ cd Documents
shaon@shaon:~/Documents$ ls
shaon@shaon:~/Documents$ cd -
/home/shaon
shaon@shaon:~$ cd ..
shaon@shaon:/home$ cd
shaon@shaon:~$ pwd
/home/shaon
shaon@shaon:~$ mkdir Projects
mkdir: cannot create directory 'Projects': File exists
shaon@shaon:~$ mkdir Projects
shaon@shaon:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Projects  Public  snap  Templates  Videos
shaon@shaon:~$ cd Projects
shaon@shaon:~/Projects$ ls
shaon@shaon:~/Projects$ cat > notes.txt
Hello, This is test file,
created by Shaon Khan
Who are dreamer
want success and try to work hard.shaon@shaon:~/Projects$
shaon@shaon:~/Projects$ cat notes.txt
Hello, This is test file,
created by Shaon Khan
Who are dreamer
want success and try to work hard.shaon@shaon:~/Projects$
shaon@shaon:~/Projects$ echo "This is new line which appends at the last of previous text" >> notes.
txt

```

Screenshot 4: mkdir, cd, cat > notes.txt creating directory and writing file

```

shaon@shaon:~/Projects$ more notes.txt
Hello, This is test file,
created by Shaon Khan
Who are dreamer
want success and try to work hard.This is new line which appends at the last of previous text
shaon@shaon:~/Projects$ echo

shaon@shaon:~/Projects$ echo "This is
> new
> line
> which
> add
> at
> the
> last
> of
> the
> previous
> line
> seriously
> trust
> me
> .
> Our
> mam
> teaches
> us
> ok?" >> notes.txt

```

Screenshot 5: more notes.txt and echo append using multi-line input

```

shaon@shaon:~/Projects$ more notes.txt
Hello, This is test file,
created by Shaon Khan
Who are dreamer
want success and try to work hard.This is new line which appends at the last of previous text
This is
new
line
which
add
at
the
last
of
the
previous
line
seriously
trust
me
.
Our
mam
teaches
us
ok?
shaon@shaon:~/Projects$ echo -e"Line 1\n 2 \n 3 \n 3\n 4 \n 5 \n 6 \n 7 \n 8 \n 9 \n 10 \n 11 \n 12
\n 13 \n 14 \n 14 \n 15 \n 16 \n 17 \n 18" >> notes.txt

```

Screenshot 6: more notes.txt showing appended content, and echo -e command

```

shaon@shaon:~/Projects$ echo -e"Line 1\n 2 \n 3 \n 3\n 4 \n 5 \n 6 \n 7 \n 8 \n 9 \n 10 \n 11 \n 12
\n 13 \n 14 \n 14 \n 15 \n 16 \n 17 \n 18" >> notes.txt
shaon@shaon:~/Projects$ more notes.txt
Hello, This is test file,
created by Shaon Khan
Who are dreamer
want success and try to work hard.This is new line which appends at the last of previous text
This is
new
line
which
add
at
the
last
of
the
previous
line
seriously
trust
me
.
Our
mam
teaches
us
ok?
-eLine 1\n 2 \n 3 \n 3\n 4 \n 5 \n 6 \n 7 \n 8 \n 9 \n 10 \n 11 \n 12 \n 13 \n 14 \n 14 \n 15 \n 16
\n 17 \n 18
shaon@shaon:~/Projects$ echo -e "Line 1\n 2 \n 3 \n 3\n 4 \n 5 \n 6 \n 7 \n 8 \n 9 \n 10 \n 11 \n 12
\n 13 \n 14 \n 14 \n 15 \n 16 \n 17 \n 18" >> notes.txt

```

Screenshot 7: echo -e output with multiple numbered lines added to notes.txt

```
shaon@shaon:~/Projects$ more notes.txt
Hello, This is test file,
created by Shaon Khan
Who are dreamer
want success and try to work hard.This is new line which appends at the last of previous text
This is
new
line
which
add
at
the
last
of
the
previous
line
seriously
trust
me
.
Our
mam
teaches
us
ok?
-eLine 1\n 2 \n 3 \n 3\n 4 \n 5 \n 6 \n 7 \n 8 \n 9 \n 10 \n 11 \n 12 \n 13 \n 14 \n 14 \n 15 \n 16
\n 17 \n 18
Line 1
2
3
3
4
5
6
```

Screenshot 8: more notes.txt showing all content including numbered lines

```
7
8
9
10
11
12
13
14
14
15
16
17
18
shaon@shaon:~/Projects$ head notes.txt
Hello, This is test file,
created by Shaon Khan
Who are dreamer
want success and try to work hard.This is new line which appends at the last of previous text
This is
new
line
which
add
at
shaon@shaon:~/Projects$ tail notes.txt
10
11
12
13
14
14
15
16
17
18
```

Screenshot 9: Continuation of notes.txt, head and tail commands

```
10
shaon@shaon:~/Projects$ touch hello.txt
shaon@shaon:~/Projects$ ls
hello.txt  notes.txt
shaon@shaon:~/Projects$
```

Screenshot 10: touch hello.txt creating an empty file

```
shaon@shaon:~/Projects$ touch hello.txt
shaon@shaon:~/Projects$ ls
hello.txt  notes.txt
shaon@shaon:~/Projects$ rm hello.txt
shaon@shaon:~/Projects$ ls
notes.txt
shaon@shaon:~/Projects$ rm -r Projects
rm: cannot remove 'Projects': No such file or directory
shaon@shaon:~/Projects$ cd
shaon@shaon:~$ rm -r Projects
shaon@shaon:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
shaon@shaon:~$ touch notes.txt
shaon@shaon:~$ ls
Desktop  Documents  Downloads  Music  notes.txt  Pictures  Public  snap  Templates  Videos
shaon@shaon:~$ rm -f notes.txt
shaon@shaon:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
shaon@shaon:~$ mkdir Pro
shaon@shaon:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Pro  Public  snap  Templates  Videos
shaon@shaon:~$ rm -rf Pro
shaon@shaon:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
shaon@shaon:~$ clear
```

Screenshot 11: rm, rm -r, rm -f, rm -rf, and clear commands

```
shaon@shaon:~$
```

Screenshot 12: Terminal after clear command — clean screen



Institute of Information Technology

Jahangirnagar University, Savar, Dhaka-1342

Lab Report - 03

Lab Title: Linux File Commands (File Management & Directory Management)

Course Code: ICT-3102

Course Title: Operating System Lab

Submitted by:

Md. Shaon Khan

Id: 1984

Batch: 52 Batch

Session: 2022-2023

Supervised by:

Mehrin Anannya

Assistant Professor, Institute of Information Technology
Jahangirnagar University

Date: April 27, 2026

1. Objective

In this lab, we practiced frequently used Linux commands for file and directory management, specifically focusing on copying, moving, renaming files and directories, searching within files using grep, and text editing using the Vim editor. The goal was to become comfortable with these powerful Linux terminal operations through hands-on practice.

2. Theory

Linux provides a rich set of command-line tools for managing files and directories. Unlike graphical file managers, the terminal gives precise control over file operations. In this lab, we used the Ubuntu terminal to execute commands for copying, moving, searching, and editing files.

The `cp` command copies files or directories from one location to another. The `mv` command moves or renames files and directories. The `grep` command searches for patterns or text within files and supports options like case-insensitive search, line-number display, and recursive directory search.

Vim is a powerful terminal-based text editor available on most Linux systems. It operates in different modes: normal mode for navigation and commands, and insert mode for typing text. Files can be saved and exited using specific commands in normal mode.

3. Commands Used

Command	Description
<code>cp file1 file2</code>	Copy file1 to file2
<code>cat file1.txt >> file2.txt</code>	Append content of file1 to file2
<code>cp -r dir1 dir2</code>	Recursively copy a directory
<code>mv file1 file2</code>	Rename or move a file
<code>mv file /path/dir/</code>	Move a file to another directory
<code>cp file /path/dir/</code>	Copy a file to another directory
<code>cp file .</code>	Copy a file to the current directory
<code>grep "pattern" file</code>	Search for a pattern in a file
<code>grep -i "pattern" file</code>	Case-insensitive pattern search
<code>grep -n "pattern" file</code>	Show matching lines with line numbers
<code>grep -r "pattern" dir/</code>	Recursively search in all files
<code>grep -v "pattern" file</code>	Show lines NOT containing the pattern
<code>vim filename.txt</code>	Open a file in the Vim text editor

4. Procedure

We opened the terminal on Ubuntu and ran each command one by one. For every command, we first ran it, then observed the output and noted what happened. Below is a step-by-step description of what we did.

4.1 cp file1 file2 - Copy File

Copies the content of file1.txt into file2.txt.

Command:

```
$ touch file1.txt
$ echo "Hello" >> file1.txt
$ cp file1.txt file2.txt
$ cat file2.txt
```

Output:

```
Hello
```

We verified that file2.txt now contains the same content as file1.txt after copying.

4.2 Append File Content

Appends the content of file1.txt to file2.txt without overwriting.

Command:

```
$ echo "World" > file2.txt
$ cat file1.txt >> file2.txt
$ cat file2.txt
```

Output:

```
World
Hello
```

We confirmed that >> appends instead of overwriting, so both 'World' and 'Hello' appeared.

4.3 cp -r dir1 dir2 - Copy Directory

Copies an entire directory recursively from one location to another.

Command:

```
$ cp -r Projects folder1
$ cd folder1
$ ls
```

Output:

```
file1.txt  file2.txt
```

The entire Projects directory and its files were copied into folder1 successfully.

4.4 mv file1 file2 - Rename File

Renames old_name.txt to new_name.txt in the same directory.

Command:

```
$ mv old_name.txt new_name.txt
$ ls
```

Output:

```
file1.txt  file2.txt  new_name.txt
```

The file was renamed and no copy was left behind.

4.5 Move File to Another Directory

Moves new_name.txt to another directory using its absolute path.

Command:

```
$ mv new_name.txt /home/shaon/folder1/
$ cd folder1
$ ls
```

Output:

```
file1.txt  file2.txt  new_name.txt  test.txt
```

The file was successfully moved to folder1 without creating a copy in the original location.

4.6 grep - Search in Files

The grep command searches for text patterns inside files. We created a file called search_file.txt and ran various grep variants on it.

Command:

```
$ echo "This is shaon khan, I am trying my best" > search_file.txt
$ grep "shaon" search_file.txt
```

Output:

```
This is shaon khan, I am trying my best
```

The matching word was highlighted in the terminal output.

Case-insensitive search:

```
$ echo "SHAON Shaon ShAoN Khan" >> search_file.txt
$ grep -i "shaon" search_file.txt
```

Output:

```
This is shaon khan, I am trying my best
SHAON Shaon ShAoN Khan
```

Using -i made the search case-insensitive, so all variations of 'shaon' were matched.

Search with line numbers:

```
$ grep -n "shaon" search_file.txt
```

Output:

```
1:This is shaon khan, I am trying my best
```

The -n flag shows line numbers along with the matching lines.

Recursive search:

```
$ grep -r "main()" codes/
```

Output:

```
codes/test2.txt:main()
```

The -r flag allowed grep to search recursively through all files in the codes/ directory.

Inverse search (lines NOT containing pattern):

```
$ grep -v "debug" file3.txt
```

Output:

```
Line 1: Starting....
```

```
Line 3: Process ....
```

Using -v excluded lines containing 'debug', showing only the non-matching lines.

4.7 Text Editing with Vim

Vim is a terminal-based text editor. We first installed it using sudo, then opened and edited files.

Installation:

```
$ sudo apt install vim -y
```

After getting sudo privileges using the usermod command, vim was installed successfully.

Basic Vim workflow:

- vim file4.txt - Open (or create) a file in Vim
- Press i to enter insert mode and type content
- Press Esc to return to normal (command) mode
- Type :w to save the file
- Type :q to quit Vim
- Type :wq to save and quit at once
- Type :q! to quit without saving

Command:

```
$ touch file4.txt
```

```
$ vim file4.txt
```

Inside Vim, we pressed i and typed the following content:

```
This is the  
first  
line  
everything  
looks  
good
```

```
so
far
The
debug log
```

We then pressed Esc and typed :wq to save and exit.

5. Discussion

This lab covered more advanced file operations compared to the previous lab. A few key observations stood out:

First, the difference between cp and mv is critical. The cp command always leaves the original file intact, while mv removes it from the source location. The -r flag is required when copying entire directories.

Second, the grep command proved very versatile. The -i, -n, -r, and -v flags greatly extend its usefulness. The recursive search with -r is especially powerful when looking for a pattern across an entire project directory.

Third, we encountered permission issues with sudo. The user account was initially not in the sudoers file, so we had to switch to root using su - and run usermod -aG sudo shaon to grant the necessary privileges. After logging back in, sudo commands worked correctly.

Fourth, learning Vim was a significant part of this lab. Vim has a steep learning curve due to its modal editing. We had to remember that typing normally only works in insert mode (press i), and all save/quit commands must be typed in normal mode (press Esc first). Once we understood the modes, the editor became usable.

6. Conclusion

We successfully completed Lab 3 of the Operating System Lab course. In this session, we practiced file copying, directory copying, file renaming, moving files between directories, pattern searching with grep, and text editing using Vim.

Through this lab, we gained practical knowledge of powerful Linux file management commands. We also learned how to handle permission issues by switching to root and granting sudo access. The knowledge of grep and Vim will be especially valuable in future labs and real-world Linux system work.

7. Lab Screenshots

The following screenshots were captured during the lab session and show the actual terminal output for the commands we ran.

```
shaon@shaon:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
shaon@shaon:~$ mkdir Projects
shaon@shaon:~$ cd Projects
shaon@shaon:~/Projects$ ls
shaon@shaon:~/Projects$ touch file1.txt
shaon@shaon:~/Projects$ echo "Hello" >> file1.txt
shaon@shaon:~/Projects$ cat file1.txt
Hello
shaon@shaon:~/Projects$ cp file1.txt file2.txt
shaon@shaon:~/Projects$ ls
file1.txt  file2.txt
shaon@shaon:~/Projects$ cat file2.txt
Hello
shaon@shaon:~/Projects$ echo "World" > file2.txt
shaon@shaon:~/Projects$ cat file2.txt
World
shaon@shaon:~/Projects$ cat file1.txt >> file2.txt
shaon@shaon:~/Projects$ cat file2.txt
World
Hello
shaon@shaon:~/Projects$ cd
shaon@shaon:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Projects  Public  snap  Templates  Videos
shaon@shaon:~$ cp -r Projects folder1
shaon@shaon:~$ ls
Desktop  Documents  Downloads  folder1  Music  Pictures  Projects  Public  snap  Templates  Videos
shaon@shaon:~$ cd folder1
shaon@shaon:~/folder1$ ls
file1.txt  file2.txt
shaon@shaon:~/folder1$ cat file2.txt
World
Hello
```

```

shaon@shaon:~/Projects$ mv old_name.txt new_name.txt
shaon@shaon:~/Projects$ ls
file1.txt  file2.txt  new_name.txt
shaon@shaon:~/Projects$ cd
shaon@shaon:~$ touch test.txt
shaon@shaon:~$ ls
Desktop  Downloads  Music      Projects  snap      test.txt
Documents  folder1  Pictures  Public    Templates  Videos
shaon@shaon:~$ cp test.txt folder1/
shaon@shaon:~$ cd folder1
shaon@shaon:~/folder1$ ls
file1.txt  file2.txt  test.txt
shaon@shaon:~/folder1$ cd
shaon@shaon:~$ cd Projects
shaon@shaon:~/Projects$ ls
file1.txt  file2.txt  new_name.txt
shaon@shaon:~/Projects$ pwd
/home/shaon/Projects
shaon@shaon:~/Projects$ mv new_name.txt /home/shaon/folder1/
shaon@shaon:~/Projects$ cd folder1
bash: cd: folder1: No such file or directory
shaon@shaon:~/Projects$ cd
shaon@shaon:~$ cd folder1
shaon@shaon:~/folder1$ ls
file1.txt  file2.txt  new_name.txt  test.txt
shaon@shaon:~/folder1$ cd
shaon@shaon:~$ cd Projects
shaon@shaon:~/Projects$ echo "This is shaon khan, I am trying my best" > search_file.txt
shaon@shaon:~/Projects$ ls
file1.txt  file2.txt  search_file.txt
shaon@shaon:~/Projects$ cat search_file.txt
This is shaon khan, I am trying my best
shaon@shaon:~/Projects$ grep "shaon" search_file.txt
This is shaon khan, I am trying my best

```

```

shaon@shaon:~/Projects/codes$ sudo apt install vim -y
[sudo] password for shaon:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  systemd-hwe-hwdb
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
  vim-common vim-runtime vim-tiny
Suggested packages:
  ctags vim-doc vim-scripts indent
The following NEW packages will be installed:
  vim vim-runtime
The following packages will be upgraded:

```



This is the first line everything looks

Files

far
The
debug log

W

```
This is the
first
line
everything
looks
good
so
far
The
debug log
```

○ : 〇



Institute of Information Technology

Jahangirnagar University, Savar, Dhaka-1342

Lab Report - 04

Lab Title: Unix — File Permission / Access Mode

Course Code: ICT-3102

Course Title: Operating System Lab

Submitted by:

Md. Shaon Khan

Id: 1984

Batch: 52 Batch

Session: 2022-2023

Supervised by:

Mehrin Anannya

Assistant Professor, Institute of Information Technology
Jahangirnagar University

Date: 12 May, 2026

1. Objective

The objective of this lab session is to develop a hands-on understanding of Linux file system operations and Unix-based file permission management. Specifically, the lab aims to:

- Create and navigate directories and files using terminal commands.
- Understand and apply wildcard metacharacters for pattern-based file listing.
- Create and edit text files using the vi editor and cat redirection.
- Analyze file contents using cat, cat -b, more, and wc commands.
- Interpret file permission strings from ls -l output.
- Modify file access permissions using both symbolic and numeric (octal) modes of the chmod command.

2. Theory

Linux is a multi-user operating system that enforces strict file permission controls to ensure security and data integrity. Every file and directory in a Linux system has associated access permissions that govern who can read, write, or execute it.

File Permission Structure

When the ls -l command is used, each file is prefixed with a 10-character permission string, for example: -rw-rw-r--. The first character indicates the file type (- for regular file, d for directory). The next nine characters are divided into three groups of three: Owner (u), Group (g), and Others (o), each containing read (r), write (w), and execute (x) bits.

Wildcard Metacharacters

Wildcards are special characters used in shell commands to match filenames based on patterns. The asterisk (*) matches zero or more characters, allowing commands like ls *.txt to list all files with the .txt extension. The question mark (?) matches any single character.

The vi Text Editor

The vi editor is a standard Unix text editor that operates in two primary modes: Command Mode (default, for navigation and issuing commands) and Insert Mode (for typing text, activated by pressing i). To save and exit, the user presses Esc to return to command mode, then types :wq and presses Enter.

The chmod Command

The chmod (change mode) command is used to modify file permissions. It supports two modes:

- **Symbolic Mode:** Uses letters (u, g, o, a) with operators (+ to add, - to remove, = to set exactly).
Example: chmod g+x file adds execute permission to the group.
- **Numeric (Octal) Mode:** Uses a three-digit octal number where Read = 4, Write = 2, Execute = 1. The digits correspond to Owner, Group, and Others respectively. Example: chmod 755 file sets Owner to rwx (7), Group to r-x (5), and Others to r-x (5).

3. Commands Used

The following table summarizes all commands used in this lab session:

Command	Description
ls	Lists files and directories in the current location
ls -l	Displays detailed listing with permissions, owner, size, and timestamps
ls file1*.txt	Lists files matching the wildcard pattern (starting with file1, ending in .txt)
ls fi*.txt	Lists files starting with "fi" and ending in ".txt"
ls *.txt	Lists all files with the .txt extension
mkdir	Creates a new directory
cd	Changes the current directory; cd alone returns to home
touch	Creates empty files or updates file timestamps
cat > file	Creates a new file and writes standard input into it (end with Ctrl+D)
cat file	Displays the contents of a file on screen
cat -b file	Displays file contents with line numbers for non-blank lines
more file	Displays file content one screen at a time
wc file	Counts and displays lines, words, and characters in a file
vi file	Opens the vi text editor to create or modify a file
chmod (symbolic)	Changes file permissions using letters (u/g/o) and operators (+/-/=)
chmod (numeric)	Changes file permissions using octal numbers (r=4, w=2, x=1)

4. Procedure

4.1 Directory and File Creation

Created the OS and Lab directories, navigated into OS, then created multiple files using the touch command.

Command:

```
$ mkdir OS Lab
$ cd OS
$ touch file1.txt file12.txt file2.txt notes.txt file1.pdf
```

Output:

```
(No output, files created silently)
```

Observation: The touch command created all specified files in a single call. The mkdir command successfully created two separate directories, OS and Lab, visible in the subsequent ls listing.

4.2 Using Wildcard Metacharacters

Demonstrated wildcard patterns to filter file listings based on name patterns.

Command:

```
$ ls file1*.txt
$ ls fi*.txt
$ ls *.txt
```

Output:

```
file12.txt file1.txt
file12.txt file1.txt file2.txt
file12.txt file1.txt file2.txt notes.txt
```

Observation: Each pattern produced a different subset of files. The * metacharacter effectively matched zero or more characters, narrowing or widening the result based on the prefix pattern provided.

4.3 Creating Files with cat Redirection

Used the cat command with output redirection to create text files with content typed directly from the terminal.

Command:

```
$ cat > notes.txt
This is note text created by Shaon Khan.
(Ctrl+D to save)

$ cat > routine.txt
Sunday: OS Lab
Monday: Networking
(Ctrl+D to save)
```

Output:

```
$ ls *.txt
file12.txt file1.txt file2.txt notes.txt routine.txt
```

Observation: The cat > filename syntax redirected keyboard input into the specified file. Pressing Ctrl+D sent the EOF signal, finalizing the file. Both files appeared in the subsequent directory listing.

4.4 Creating and Editing Files with vi

Used the vi editor to create hello.txt and enter content in Insert Mode, then saved and exited using :wq.

Command:

```
$ vi hello.txt
(Press i to enter INSERT mode)
This is file created by Shaon.
I am from IIT, JU.
(Press Esc)
:wq
```

Observation: The vi editor opened in Command Mode. Pressing i switched to INSERT mode (shown at the bottom of the screen). Text was typed successfully. After pressing Esc, the :wq command saved the file and returned to the terminal prompt.

4.5 Viewing and Analyzing File Contents

Used cat, cat -b, wc, and more to display and analyze the content of hello.txt and Hello1.txt.

Command:

```
$ cat hello.txt
$ cat -b hello.txt
$ wc hello.txt
$ more Hello1.txt
```

Output:

```
This is file created by Shaon.
I am from IIT, JU.

1 This is file created by Shaon.
2 I am from IIT, JU.

3 11 51 hello.txt
```

Observation: The cat command printed the raw file contents. cat -b added sequential line numbers to each non-blank line. The wc command revealed the file has 3 lines, 11 words, and 51 characters. The more command displayed Hello1.txt content one screen at a time.

4.6 Viewing Detailed File Listing

Used ls -l to inspect file permissions, ownership, and metadata for all files in the OS directory.

Command:

```
$ ls -l
$ ls -l hello.txt
```

Output:

```
-rw-rw-r-- 1 shaonkhan shaonkhan  0 May  6 15:58 file12.txt
-rw-rw-r-- 1 shaonkhan shaonkhan  0 May  6 15:58 file1.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
...
-rw-rw-r-- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
```

Observation: The ls -l output clearly displayed the 9-character permission string, the number of hard links, the owner name, the group name, the file size in bytes, and the last modification timestamp for every file. The hello.txt file showed 51 bytes, consistent with the wc output.

4.7 Modifying Permissions Using Symbolic Mode

Applied a series of symbolic chmod operations to hello.txt, verifying each change with ls -l.

Command:

```
$ chmod g+x hello.txt → -rw-rwxr--
$ chmod o+wx hello.txt → -rw-rwxrwx
$ chmod u+x hello.txt → -rwxrwxrwx
$ chmod u-rwx hello.txt → ----rwxrwx
$ chmod g-x hello.txt → ----rw-rwx
$ chmod o-r hello.txt → ----rw--wx
$ chmod o=x hello.txt → ----rw---x
$ chmod g=x hello.txt → -----x--x
```

```
$ chmod u=r hello.txt → -r---x--x
```

Observation: Each symbolic operation precisely modified the intended permission bits without affecting the others. The = operator was particularly notable in that it completely replaced the target group's permissions rather than adding or removing individual bits.

4.8 Modifying Permissions Using Numeric Mode

Applied numeric (octal) chmod values to hello.txt and verified the resulting permission strings.

Command:

```
$ chmod 444 hello.txt → -r--r--r--
$ chmod 000 hello.txt → -----
$ chmod 777 hello.txt → -rwxrwxrwx
$ chmod 562 hello.txt → -r-xrw--w-
```

Observation: The numeric mode provided a concise and efficient way to set all nine permission bits in a single command. chmod 000 removed all permissions entirely, while chmod 777 granted full access to all three groups. The value 562 decomposed to r-x (5) for Owner, rw- (6) for Group, and -w- (2) for Others, which was verified in the ls -l output.

5. Discussion

This lab session provided comprehensive, practical exposure to Linux file management and permission control. Several important observations were made during the session:

- Wildcard metacharacters proved to be highly efficient for filtering large directory listings. The asterisk (*) is especially powerful as it can match any string, making pattern-based file operations fast and convenient.
- The vi editor, while initially non-intuitive due to its modal design, offers powerful editing capabilities once the distinction between Command Mode and Insert Mode is understood. The :wq convention is a universal save-and-quit mechanism.
- The ls -l command is indispensable for diagnosing permission-related issues. The 10-character permission string encodes rich information about file type, ownership model, and access rights in a compact format.
- Symbolic chmod is more readable and human-friendly, making it ideal for targeted permission changes such as adding execute permission for a specific group. Numeric chmod is faster for setting complete permission profiles from scratch.
- The chmod 000 operation demonstrated that even the file owner can be locked out of their own file, highlighting the importance of careful permission management in multi-user environments.
- The wc command output (3 11 51 hello.txt) confirmed that the file had 3 lines (including a trailing newline), 11 words, and 51 bytes, providing a precise characterization of file content.

6. Conclusion

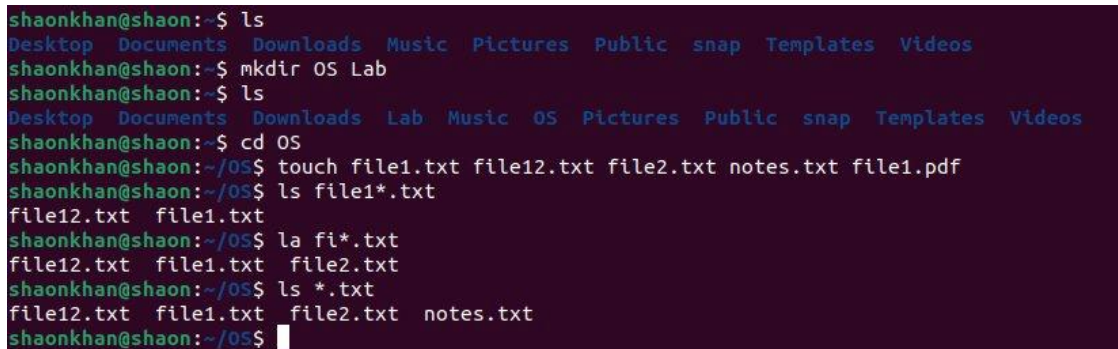
This laboratory exercise successfully achieved its intended objectives. Through a structured series of experiments, the fundamental aspects of Linux file operations were mastered, ranging from directory navigation and file creation to content analysis and permission management.

The ability to control file access through `chmod` is a cornerstone of Linux system administration and software development. Understanding both symbolic and numeric modes provides the flexibility to handle any permission-related task efficiently. The concepts practiced in this lab — particularly around the `chmod` command, `ls -l` interpretation, and `vi` editing — form the foundation for more advanced topics such as shell scripting, process management, and system security hardening.

Future lab sessions will build upon these skills, extending into areas such as process control, piping, I/O redirection, and network administration.

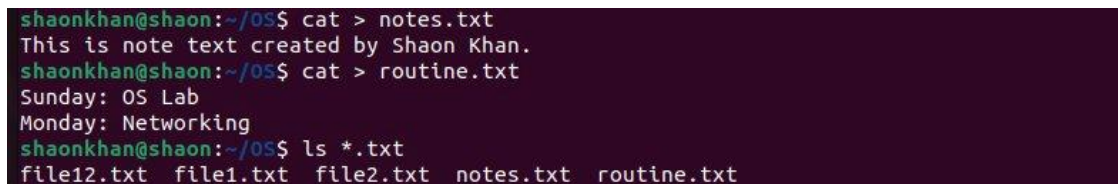
7. Lab Screenshots

The following terminal screenshots document all commands executed during this lab session in chronological order.



```
shaonkhan@shaon:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
shaonkhan@shaon:~$ mkdir OS Lab
shaonkhan@shaon:~$ ls
Desktop  Documents  Downloads  Lab  Music  OS  Pictures  Public  snap  Templates  Videos
shaonkhan@shaon:~$ cd OS
shaonkhan@shaon:~/OS$ touch file1.txt file12.txt file2.txt notes.txt file1.pdf
shaonkhan@shaon:~/OS$ ls file1*.txt
file12.txt  file1.txt
shaonkhan@shaon:~/OS$ la fi*.txt
file12.txt  file1.txt  file2.txt
shaonkhan@shaon:~/OS$ ls *.txt
file12.txt  file1.txt  file2.txt  notes.txt
shaonkhan@shaon:~/OS$
```

Figure 1: Creating directories, navigating filesystem, and using wildcard metacharacters (`ls file1*.txt`, `ls fi*.txt`, `ls *.txt`)



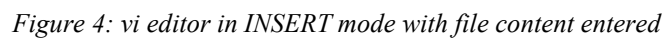
```
shaonkhan@shaon:~/OS$ cat > notes.txt
This is note text created by Shaon Khan.
shaonkhan@shaon:~/OS$ cat > routine.txt
Sunday: OS Lab
Monday: Networking
shaonkhan@shaon:~/OS$ ls *.txt
file12.txt  file1.txt  file2.txt  notes.txt  routine.txt
```

Figure 2: Creating files using `cat` redirection (`cat > notes.txt`, `cat > routine.txt`) and listing all `.txt` files



```
shaonkhan@shaon:~/OS$ vi hello.txt
```

Figure 3: Opening the `vi` text editor to create `hello.txt`



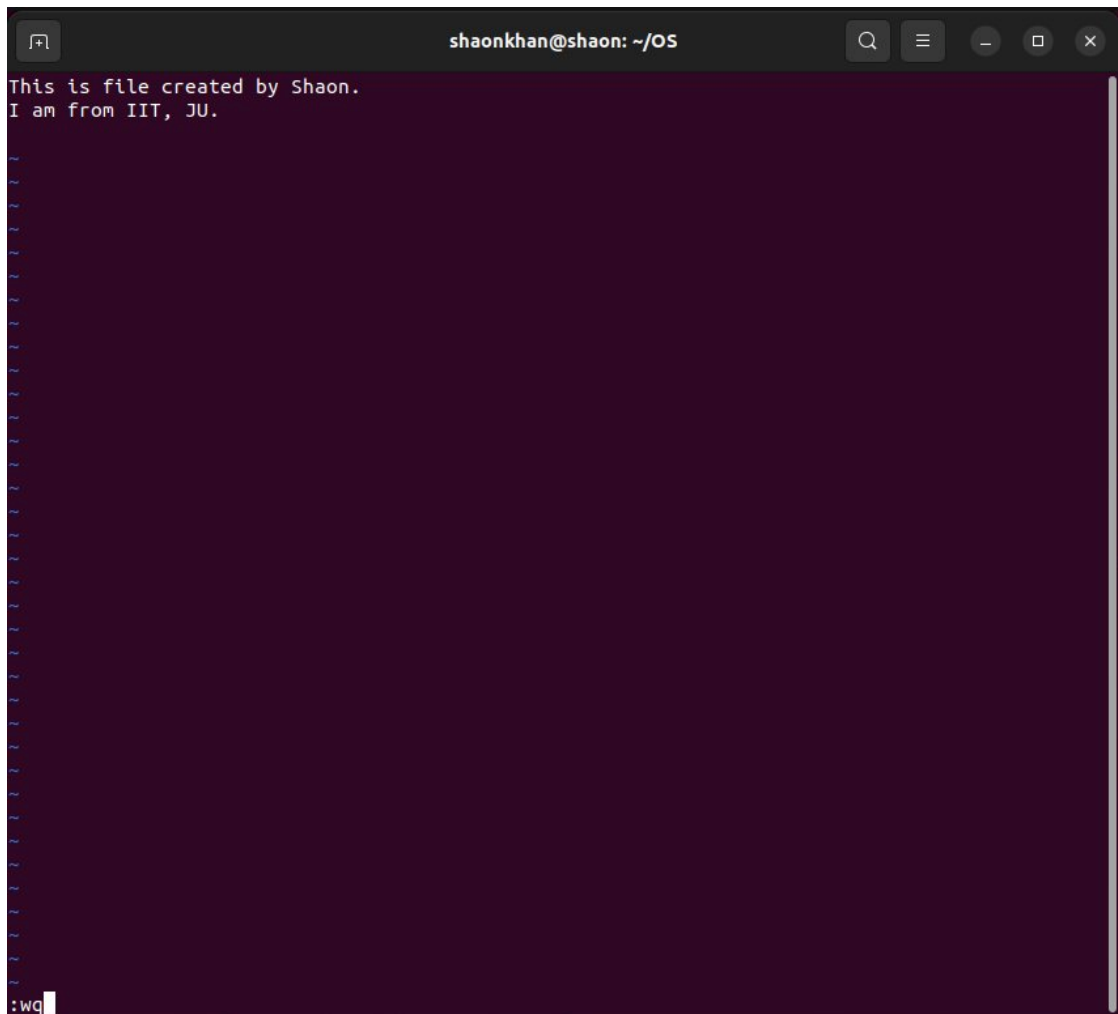


Figure 5: Saving and quitting vi editor using :wq command

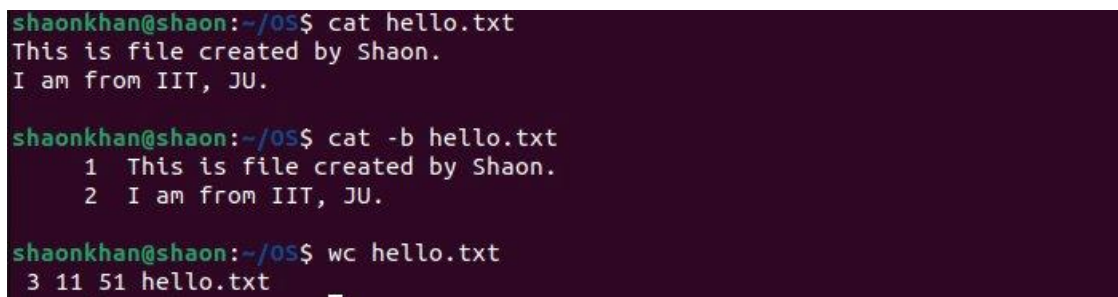


Figure 6: Displaying file content using `cat`, `cat -b` (with line numbers), and `wc` (word count)

```

shaonkhan@shaon:~/OS$ ls -l
total 16
-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 6 15:58 file12.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 6 15:58 file1.pdf
-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 6 15:58 file1.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 6 15:58 file2.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 41 May 6 16:00 notes.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 37 May 12 18:26 practice_2.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 34 May 6 16:01 routine.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt

```

Figure 7: Detailed directory listing (`ls -l`) showing file permissions, ownership, size, and timestamps

```

shaonkhan@shaon:~/OS$ ls -l hello.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod g+x hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
-rw-rwxr-- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod o+wx hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
-rw-rwxrwx 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod u+x hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
-rwxrwxrwx 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod u-rwx hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
---rwxrwx 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod g-x hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
---rwxrwx 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod o-r hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
---rw--wx 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod o=x hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
---rw---x 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod g=x hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
-----x--x 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$ chmod u=r hello.txt
shaonkhan@shaon:~/OS$ ls -l hello.txt
-r-----x--x 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/OS$

```

Figure 8: Changing file permissions using symbolic mode `chmod` (`g+x`, `o+wx`, `u+x`, `u-rwx`, `g-x`, `o-r`, `o=x`, `g=x`, `u=r`)

```
shaonkhan@shaon:~/05$ ls -l hello.txt
-r---x--x 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/05$ chmod 444 hello.txt
shaonkhan@shaon:~/05$ ls -l hello.txt
-r--r--r-- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/05$ chmod 000 hello.txt
shaonkhan@shaon:~/05$ ls -l hello.txt
----- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/05$ chmod 777 hello.txt
shaonkhan@shaon:~/05$ ls -l hello.txt
-rwxrwxrwx 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
shaonkhan@shaon:~/05$ chmod 562 hello.txt
shaonkhan@shaon:~/05$ ls -l hello.txt
-r-xrw--w- 1 shaonkhan shaonkhan 51 May 12 18:25 hello.txt
```

Figure 9: Changing file permissions using numeric (octal) mode chmod (444, 000, 777, 562)

```
shaonkhan@shaon:~/05$ cd
shaonkhan@shaon:~$
```

Figure 10: Returning to home directory using cd command

```
shaonkhan@shaon:~$ cat > Hello1.txt
Hello World!!
shaonkhan@shaon:~$ more Hello1.txt
Hello World!!
```

Figure 11: Creating Hello1.txt using cat redirection and viewing with more command



**INSTITUTE OF INFORMATION TECHNOLOGY
JAHANGIRNAGAR UNIVERSITY
SAVAR, DHAKA-1342**

OPERATING SYSTEM LAB

Course Code: ICT-3102

LAB REPORT NO: 05

Lab Title:

**Working with Users, Groups, and File Permissions
(User and Group Management)**

Submitted by:

Md. Shaon Khan

Student ID: 1984

52nd Batch | Session: 2022-2023

Supervised by:

Mehrin Anannya

Assistant Professor

Institute of Information Technology

Jahangirnagar University

Submission Date: May 26, 2026

1. Objective

The primary objective of this laboratory session is to develop a working understanding of Linux user and group administration from the perspective of a system administrator. The lab covers the complete lifecycle of a user account, from creation to deletion, and explores the mechanisms by which Linux enforces access control through file ownership and permission bits.

Specifically, the following goals guided the experiment:

- (i) To understand the structure and role of the Linux multi-user architecture, including how user and group information is stored in system configuration files.
- (ii) To learn the commands required to create, configure, switch between, and remove user accounts on a Linux system.
- (iii) To distinguish between the behavior of `su` and `su -` when switching user sessions, and to understand the consequence of each on the shell environment.
- (iv) To practice group creation, group membership assignment, and the use of the `usermod` command for assigning supplementary groups to an existing user.
- (v) To modify file ownership and group ownership using the `chown` and `chgrp` commands, and to understand the access constraints governing these operations.
- (vi) To apply special permission bits, specifically the Set Group ID (SGID) flag, to directories using the `chmod` command, and to observe its effect on the permission string displayed by `ls -l`.
- (vii) To understand the security and administrative significance of each concept practiced, including privilege separation, permission enforcement, and collaborative workspace management.

2. Introduction and Theory

2.1 Linux as a Multi-User Operating System

Linux is a multi-user and multi-tasking operating system derived from the UNIX tradition. Its design inherently supports simultaneous access by multiple users, each of whom is given an isolated environment within which their files, processes, and credentials are protected from unauthorized access by other users. This isolation is enforced at the kernel level through a system of user identifiers (UIDs) and group identifiers (GIDs), which are associated with every process and every file on the filesystem.

Unlike single-user operating systems, Linux maintains strict boundaries between accounts. A standard user can read, write, and execute files only to the extent permitted by the file permission bits and ownership attributes. This arrangement ensures that a compromised or misbehaving user account cannot affect system-wide resources without elevated privileges. The superuser account, traditionally named root, holds UID 0 and is exempt from these permission checks, allowing it to perform unrestricted administrative operations.

Administrative tasks such as creating users, modifying groups, and changing file ownership are generally restricted to the root account or to users who have been granted the ability to execute specific commands with elevated privilege via the sudo mechanism. This separation of ordinary and privileged operations is a fundamental security design principle in Linux.

2.2 User Account Databases

Linux stores user account information in two primary configuration files: `/etc/passwd` and `/etc/shadow`. The `/etc/passwd` file contains one entry per account and is world-readable. Each entry records the username, a placeholder for the password field, the UID, the primary GID, the GECOS field (which holds optional personal information such as full name, room number, and phone numbers), the home directory path, and the default login shell. The world-readability of this file allows programs to map UIDs to usernames without requiring elevated privileges.

Since storing actual password hashes in a world-readable file would be a security vulnerability, Linux separates authentication credentials into `/etc/shadow`. This file is readable only by the root user and the shadow group. It stores the hashed password for each account, along with metadata such as the last password change date, minimum and maximum password age, and account expiration settings. This division between public account metadata and private authentication data is known as shadow password support and is enabled by default on all modern Linux distributions.

Group information is stored in `/etc/group`, which lists each group by name, its GID, and the usernames of its supplementary members. A user always has a primary group, whose GID is recorded in `/etc/passwd`,

and may belong to any number of supplementary groups recorded in `/etc/group`. Supplementary group membership determines what group-owned resources the user can access in addition to those accessible through the primary group.

2.3 User Switching: su versus su -

The `su` command, which stands for switch user, allows a logged-in user to assume the identity of another account by providing that account's password. However, the behavior of `su` differs meaningfully depending on whether it is invoked with the hyphen flag or without it.

When executed as `su username` without the hyphen, Linux spawns a new shell under the target user's identity but preserves the calling user's environment. This means that environment variables such as `HOME`, `PATH`, and `SHELL` retain their values from the original session, and the working directory remains wherever the user was before invoking the command. In practical terms, this can cause unexpected behavior because the new session does not properly initialize the target user's environment.

When executed as `su - username`, Linux invokes a login shell for the target user. This means the system reads the target user's shell initialization files, such as `/etc/profile` and `~/.bashrc`, sets `HOME` to the target user's home directory, updates `PATH` and other environment variables appropriately, and changes the working directory to the target user's home directory. This behavior is equivalent to a fresh login as that user. In administrative practice, `su -` is the preferred form because it guarantees a properly initialized environment, avoids inheriting environment variables from the calling session, and accurately reflects what the target user would see upon logging in directly.

2.4 Special Permission Bits: SUID and SGID

Linux supports three special permission bits beyond the standard read, write, and execute triplets: the Set User ID (SUID) bit, the Set Group ID (SGID) bit, and the sticky bit. The SUID and SGID bits are particularly relevant in the context of privilege management and collaborative file access.

The Set User ID (SUID) bit, when applied to an executable file, causes the process launched by that executable to run with the effective UID of the file's owner rather than the UID of the user who invoked it. A well-known example is the `/usr/bin/passwd` program, which must write to `/etc/shadow` despite being executed by an ordinary user. Since `/etc/shadow` is readable only by root, the `passwd` binary carries the SUID bit and runs as root regardless of who invokes it. This mechanism allows controlled temporary elevation of privilege without granting the user unrestricted root access.

The Set Group ID (SGID) bit has a related but distinct function. When applied to an executable, it causes the process to run with the effective GID of the file's group rather than the calling user's primary group. When SGID is applied to a directory, it has an even more useful effect: all files and subdirectories created

within that directory automatically inherit the group ownership of the directory itself, rather than the primary group of the user who created them. This behavior is particularly valuable in shared or collaborative directories where multiple users from the same group are expected to contribute files that must remain accessible to all members of that group. Without SGID, each file would inherit the creator's primary group, making it potentially inaccessible to other group members who do not share that primary group.

In the permission string output of `ls -l`, the presence of SUID or SGID is indicated by the letter `s` in the position normally occupied by the execute (`x`) bit. If the execute bit was already set, the permission displays as `s`. If it was not set, it displays as `S`, indicating that the special bit is present but the ordinary execute permission is absent.

3. Commands and Utilities Used

Command	Purpose	Description
getent passwd	Display user accounts	Retrieves entries from the Name Service Switch databases, including /etc/passwd, and prints complete account information for all users on the system.
compgen -u	List usernames only	A shell built-in command that generates a plain list of all usernames known to the system, without additional fields, making the output easier to scan than getent passwd.
sudo adduser <user>	Create a new user account	Creates a new user account interactively, assigning a UID, creating a primary group, initializing the home directory from /etc/skel, and prompting for the password and GECOS information.
sudo passwd <user>	Reset a user's password	Allows a privileged user to set or change the password of any account directly, bypassing the interactive adduser password prompt. Useful for setting a known password after account creation.
su - <user>	Switch to user with full login environment	Opens a login shell as the specified user, fully initializing that user's environment variables, home directory, and shell configuration files, as if the user had logged in directly.
sudo userdel -r <user>	Delete a user and their home directory	Removes the specified user account from the system. The -r flag additionally removes the user's home directory and mail spool, ensuring that no orphaned files remain after deletion.
sudo groupadd <group>	Create a new group	Creates a new group with the specified name. The kernel assigns the next available GID automatically, and the group is registered in /etc/group.
sudo tail /etc/group	View recently added groups	Displays the last ten lines of the /etc/group file, making it convenient to verify newly created groups and confirm their assigned GID values.
groups [user]	List group memberships	Prints the names of all groups to which the specified user belongs, including the primary group and any supplementary groups. When called without an argument, it reports the current user's groups.
sudo usermod -aG <groups> <user>	Add user to supplementary groups	Modifies an existing user account to add them to the specified supplementary groups. The -a flag appends without removing existing memberships; the -G flag specifies the comma-separated list of target groups.

Command	Purpose	Description
sudo chown <user> <file>	Change file owner	Transfers ownership of the specified file to the specified user. Only root or a user with sudo privileges can transfer file ownership to another account.
sudo chgrp <group> <file>	Change file group ownership	Changes the group associated with the specified file to the given group name. An ordinary user may only change a file's group if they own the file and belong to the target group; sudo is required otherwise.
chmod ug+s <dir>	Set SUID and SGID bits	Applies the Set User ID and Set Group ID special permission bits to the specified directory or file. For a directory, SGID causes all newly created files within it to inherit the directory's group ownership automatically.

4. Step-by-Step Experimental Procedure

4.1 Exploring Existing System Accounts

The experiment began by examining the set of user accounts already present on the system. The first command used was `getent passwd`, which queries the Name Service Switch database and prints the full contents of `/etc/passwd` in a structured, colon-delimited format. Each line of the output corresponds to one account and contains the username, a placeholder `x` in the password field indicating that the actual hash is stored in `/etc/shadow`, the numeric UID, the numeric GID, the GECOS field, the home directory, and the default shell.

The output showed a large number of system accounts, including well-known service accounts such as `root`, `daemon`, `bin`, `sys`, `sync`, `games`, `man`, `lp`, `mail`, `news`, `uucp`, `proxy`, `www-data`, `backup`, and many others maintained by the Ubuntu package management system. System accounts generally have UIDs below 1000 and are assigned `/usr/sbin/nologin` or `/bin/false` as their shell, which prevents interactive login. The only human account visible in the output was `shaonkhan`, assigned UID 1002 and the shell `/bin/bash`, confirming it as the primary interactive account on the machine.

```
shaonkhan@shaon:~$ getent passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:100:102:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:101:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
messagebus:x:102:105:/:nonexistent:/usr/sbin/nologin
systemd-timesync:x:103:106:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
syslog:x:104:111:/:home/syslog:/usr/sbin/nologin
_apt:x:105:65534:/:nonexistent:/usr/sbin/nologin
tss:x:106:112:TPM software stack,,,:/var/lib/tpm:/bin/false
uuidd:x:107:115:/:run/uuidd:/usr/sbin/nologin
systemd-oom:x:108:116:systemd Userspace OOM Killer,,,:/run/systemd:/usr/sbin/nologin
tcpdump:x:109:117:/:nonexistent:/usr/sbin/nologin
avahi-autoipd:x:110:119:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/usr/sbin/nologin
usbmux:x:111:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
dnsmasq:x:112:65534:dnsmasq,,,:/var/lib/misc:/usr/sbin/nologin
kernoops:x:113:65534:Kernel Oops Tracking Daemon,,,:/usr/sbin/nologin
avahi:x:114:121:Avahi mDNS daemon,,,:/run/avahi-daemon:/usr/sbin/nologin
cups-pk-helper:x:115:122:user for cups-pk-helper service,,,:/home/cups-pk-helper:/usr/sbin/nologin
rtkit:x:116:123:RealtimeKit,,,:/proc:/usr/sbin/nologin
whoopie:x:117:124:/:nonexistent:/bin/false
sssd:x:118:125:SSSD system user,,,:/var/lib/sss:/usr/sbin/nologin
speech-dispatcher:x:119:29:Speech Dispatcher,,,:/run/speech-dispatcher:/bin/false
nm-openvpn:x:120:126:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin
saned:x:121:128:/:var/lib/saned:/usr/sbin/nologin
colord:x:122:129:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nologin
geoclue:x:123:130:/:var/lib/geoclue:/usr/sbin/nologin
pulse:x:124:131:PulseAudio daemon,,,:/run/pulse:/usr/sbin/nologin
```

Figure 1(a): Output of `getent passwd` showing system and service accounts

```
gnome-initial-setup:x:125:65534:/:run/gnome-initial-setup:/bin/false
hplip:x:126:7:HPLIP system user,,,:/run/hplip:/bin/false
gdm:x:127:133:Gnome Display Manager:/var/lib/gdm3:/bin/false
shaonkhan:x:1002:1002:Shaon Khan,1984,01633040670,01633040670:/home/shaonkhan:/bin/bash
```

Figure 1(b): End of getent passwd output including the shaonkhan account entry

To obtain a cleaner view of usernames only, the command `compugen -u` was then executed. This shell built-in command reads the same account database but prints only the account names, one per line, without any additional fields. The output provided an uncluttered list of all accounts, making it straightforward to count and inspect them. The last entry in the `compugen -u` output was `shaonkhan`, confirming it as the most recently created human account at the time.

```
colord
geoclue
pulse
gnome-initial-setup
hplip
gdm
shaonkhan
```

Figure 1(c): `compugen -u` output showing usernames including `shaonkhan`

4.2 Creating a New User Account

The next step was to create a new user account named `sehrish`. The command `sudo adduser sehrish` was executed from the `shaonkhan` session. Since `adduser` requires root privileges, the `sudo` prefix prompted for the `shaonkhan` account's password before proceeding.

The `adduser` command then performed several operations automatically. It created a new group named `sehrish` with GID 1000 and assigned that group as the user's primary group. It created the user account `sehrish` with UID 1000. It created the home directory `/home/sehrish` and populated it by copying the skeleton files from `/etc/skel`, which typically includes default configuration files such as `.bashrc`, `.profile`, and `.bash_logout`.

The command then prompted for a password. The first password attempted was flagged by the system with the warning "BAD PASSWORD: The password fails the dictionary check - it does not contain enough DIFFERENT characters." Despite this warning, the system allowed the weak password to be accepted when the confirmation was entered, since the `adduser` tool does not enforce rejection in interactive mode. Following the password prompt, the system displayed a form asking for GECOS information. The following details were entered: Full Name as `Sehrish Khan`, Room Number as `1`, Work Phone as `01511111111`, Home Phone as `01522222222`, and the Other field was left blank. When asked "Is the information correct? [Y/n]", `Y` was entered to confirm.

```

shaonkhan@shaon:~$ sudo adduser sehrish
[sudo] password for shaonkhan:
Adding user `sehrish' ...
Adding new group `sehrish' (1000) ...
Adding new user `sehrish' (1000) with group `sehrish' ...
Creating home directory `/home/sehrish' ...
Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: The password fails the dictionary check - it does not contain enough DIFFERENT characters
Retype new password:
passwd: password updated successfully
Changing the user information for sehrish
Enter the new value, or press ENTER for the default
  Full Name []: Sehrish Khan
   Room Number []: 1
  Work Phone []: 01511111111
  Home Phone []: 01522222222
   Other []:
Is the information correct? [Y/n] Y

```

Figure 2: Creating user sehrish using sudo adduser with GECOS information

After account creation, a stronger password was set using `sudo passwd sehrish`. This command, when run with elevated privileges, allows the administrator to change any user's password without knowing the current one. A shorter test password was entered to demonstrate that the system still issues a BAD PASSWORD warning for passwords shorter than eight characters, but again proceeds to update the credential upon confirmation. The terminal displayed "passwd: password updated successfully," confirming that the credential change was applied to `/etc/shadow`.

```

shaonkhan@shaon:~$ sudo -i
root@shaon:~#
logout
shaonkhan@shaon:~$ sudo u
sudo: u: command not found
shaonkhan@shaon:~$ sudo su
root@shaon:/home/shaonkhan# cd
root@shaon:~#
exit
shaonkhan@shaon:~$ sudo passwd sehrish
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully

```

Figure 3: Resetting the password for sehrish using sudo passwd

4.3 Switching Between User Accounts

With the sehrish account created, the difference in behavior between `su username` and `su - username` was demonstrated. First, `su sehrish` was executed from within the shaonkhan home directory. The system prompted for the sehrish account's password. After authentication succeeded, a new shell was opened under the sehrish identity. However, the working directory did not change; the shell remained inside `/home/shaonkhan`, as evidenced by the prompt and subsequent behavior.

When `ls` was executed in this state, the terminal returned "ls: cannot open directory '.': Permission denied." This error occurred because the directory `/home/shaonkhan` is owned by shaonkhan and its permissions do not allow other users to read or traverse it. The sehrish account, being a different user with no special privileges over shaonkhan's home directory, was denied access. This demonstrates a

practical consequence of the non-login su behavior: the shell session is left in a directory that the new identity may not have permission to access.

When cd was executed, the shell moved to the sehrish account's home directory at /home/sehrish, and ls now returned an empty listing, confirming that the home directory existed but contained no user-created files. The working directory was confirmed to be /home/sehrish using pwd.

To return to the shaonkhan account, su shaonkhan was entered. After re-authentication, the shell returned to the shaonkhan context, now with the working directory at /home/sehrish due to where the session had been.

```
shaonkhan@shaon:~$ su sehrish
Password:
sehrish@shaon:/home/shaonkhan$ ls
ls: cannot open directory '.': Permission denied
sehrish@shaon:/home/shaonkhan$ cd
sehrish@shaon:~$ ls
sehrish@shaon:~$ pwd
/home/sehrish
sehrish@shaon:~$ su shaonkhan
Password:
shaonkhan@shaon:/home/sehrish$ cd
shaonkhan@shaon:~$
shaonkhan@shaon:~$ su - sehrish
Password:
sehrish@shaon:~$
logout
```

Figure 4: Demonstrating su sehrish behavior and Permission denied error

The su - sehrish command was then used to demonstrate the proper login shell behavior. This time, the shell immediately changed to /home/sehrish, and the terminal prompt reflected the correct home directory. The environment was fully initialized as if sehrish had logged in directly. From within this session, the mkdir Downloads command was executed to create a directory inside sehrish's home directory. Running ls confirmed that Downloads had been created successfully.

```
sehrish@shaon:~$ ls
sehrish@shaon:~$ pwd
/home/sehrish
sehrish@shaon:~$ mkdir Downloads
sehrish@shaon:~$ ls
Downloads
```

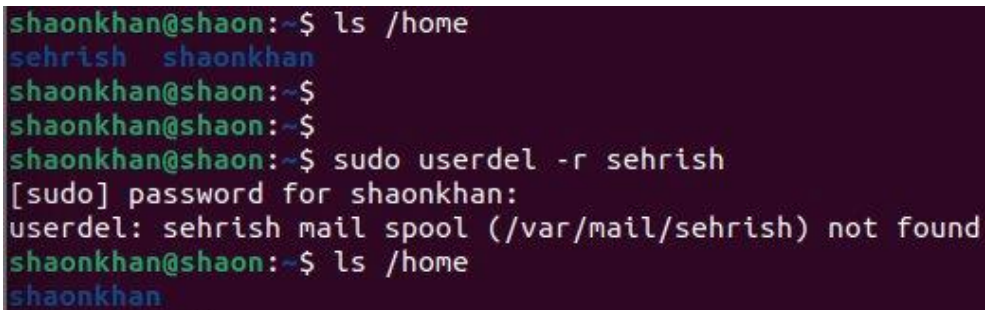
Figure 5: Creating the Downloads directory inside sehrish home directory

4.4 Deleting a User Account

After completing the session as sehrish, the logout command was used to return to the shaonkhan shell. Before deletion, `ls /home` was run to verify that both `/home/sehrish` and `/home/shaonkhan` directories were present on the filesystem.

The account was then removed using `sudo userdel -r sehrish`. The `-r` flag instructs `userdel` to recursively delete the user's home directory and mail spool in addition to removing the account entry from `/etc/passwd` and `/etc/shadow`. The system displayed a minor warning: "userdel: sehrish mail spool (`/var/mail/sehrish`) not found." This warning simply indicates that no mail spool had been created for this account during its short lifetime, which is normal for newly created accounts that have not yet received system mail. The warning is not an error and does not prevent successful account removal.

After the command completed, `ls /home` was run again. The output showed only `shaonkhan`, confirming that the `/home/sehrish` directory and all its contents, including the Downloads directory created earlier, had been removed along with the account.

A terminal window with a dark purple background. The text shows a user named shaonkhan at a host named shaon. The user runs 'ls /home' and sees 'sehrish' and 'shaonkhan'. Then they run 'sudo userdel -r sehrish'. A password prompt appears, and the user provides one. A warning message is shown: 'userdel: sehrish mail spool (/var/mail/sehrish) not found'. Finally, the user runs 'ls /home' again and only 'shaonkhan' is listed.

```
shaonkhan@shaon:~$ ls /home
sehrish  shaonkhan
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$ sudo userdel -r sehrish
[sudo] password for shaonkhan:
userdel: sehrish mail spool (/var/mail/sehrish) not found
shaonkhan@shaon:~$ ls /home
shaonkhan
```

Figure 6: Verifying deletion of sehrish using `ls /home` before and after `userdel`

4.5 Creating a Group

The command `sudo groupadd phyra` was executed to create a new supplementary group named `phyra`. The kernel automatically assigned the next available GID to this group. To confirm that the group was created and to inspect its assigned GID, `sudo tail /etc/group` was run. This command printed the last ten lines of the group database, which included recently added entries.

The relevant portion of the output showed that the system already had several existing groups at high GID values, including `sambashare` at GID 135, `shaonkhan` at GID 1002, `Triangle_2` at GID 1004, `Triangle_1` at GID 1005, and `shayankhan` at GID 1000. The newly created `phyra` group appeared as the last entry with GID 1006, confirming successful creation. The `groups` command was also run to confirm the current user's own group memberships, which showed `shaonkhan` belonging to the `sudo` and `Triangle_2` groups in addition to the primary `shaonkhan` group.

```

shaonkhan@shaon:~$ sudo groupadd phyra
shaonkhan@shaon:~$ sudo tail /etc/group
pulse:x:131:
pulse-access:x:132:
gdm:x:133:
lxd:x:134:
sambashare:x:135:
shaonkhan:x:1002:
Triangle_2:x:1004:shaonkhan
Triangle_1:x:1005:
shayankhan:x:1000:
phyra:x:1006:
shaonkhan@shaon:~$ groups
shaonkhan sudo Triangle_2
shaonkhan@shaon:~$ groups shayankhan
shayankhan : shayankhan
shaonkhan@shaon:~$ sudo shayankhan -aG shaonkhan,phyra,Triangle_1
sudo: shayankhan: command not found
shaonkhan@shaon:~$ groups shayankhan
shayankhan : shayankhan

```

Figure 7: Group creation with groupadd phyra and verification using tail /etc/group

4.6 Creating a User and Assigning Group Memberships

Before working with group assignments, a second user account named shayankhan was created using `sudo adduser shayankhan`. The creation process followed the same interactive sequence as for sehrish: a primary group shayankhan was created with GID 1000, the home directory `/home/shayankhan` was initialized, a password was set, and GECOS information was entered with the Full Name field set to Shayan Khan and Room Number set to 1.

```

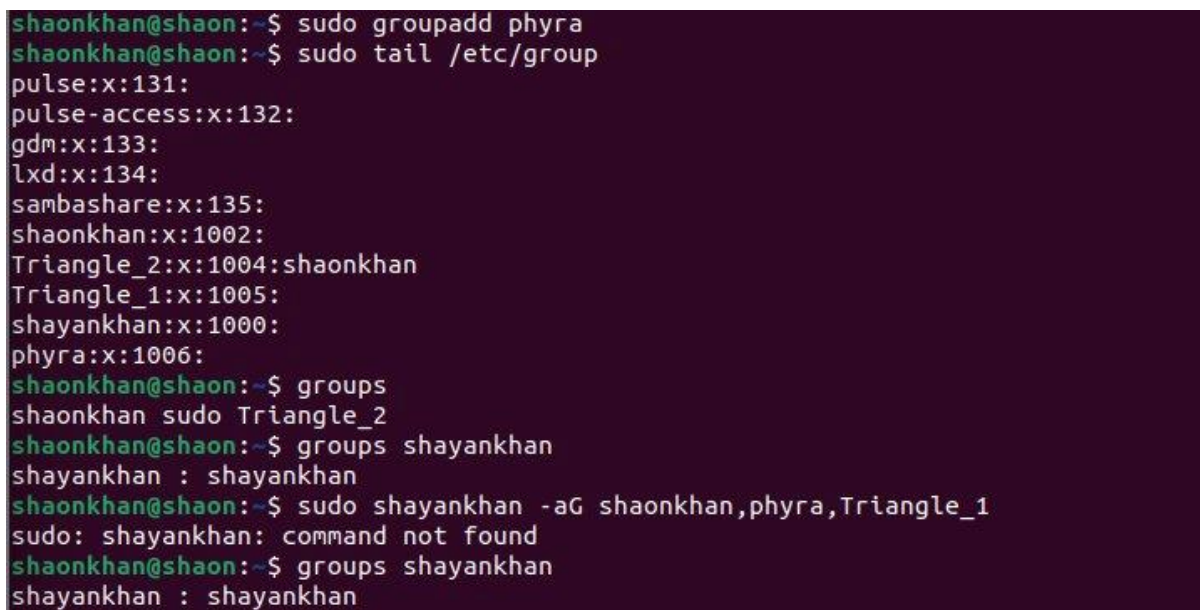
shaonkhan@shaon:~$ touch testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 26 09:57 testfile.txt
shaonkhan@shaon:~$ 
shaonkhan@shaon:~$ 
shaonkhan@shaon:~$ sudo adduser shayankhan
Adding user `shayankhan' ...
Adding new group `shayankhan' (1000) ...
Adding new user `shayankhan' (1000) with group `shayankhan' ...
Creating home directory `/home/shayankhan' ...
Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: The password fails the dictionary check - it does not contain enough DIFFERENT characters
Retype new password:
passwd: password updated successfully
Changing the user information for shayankhan
Enter the new value, or press ENTER for the default
    Full Name []: Shayan Khan
    Room Number []: 1
    Work Phone []: 1111111111
    Home Phone []: 1111111111
    Other []:
Is the information correct? [Y/n] Y
shaonkhan@shaon:~$ ls /homee
ls: cannot access '/homee': No such file or directory
shaonkhan@shaon:~$ ls /home
shaonkhan shayankhan
shaonkhan@shaon:~$ touch testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 26 09:59 testfile.txt
shaonkhan@shaon:~$ sudo chown shayankhan testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shayankhan shaonkhan 0 May 26 09:59 testfile.txt

```

Figure 8: Creating user shayankhan using sudo adduser

With the shayankhan account created, the next objective was to assign it membership in multiple supplementary groups: shaonkhan, phyra, Triangle_1, and Triangle_2. An error was made during the first attempt: the command entered was `sudo shayankhan -aG shaonkhan,phyra,Triangle_1`, which is syntactically incorrect. Instead of using `usermod` to invoke the user modification utility, the username shayankhan was accidentally written as the command itself. The shell interpreted shayankhan as an executable program name and returned "sudo: shayankhan: command not found." This is a common typographical error when working quickly in the terminal.

The groups shayankhan command was run after this failed attempt to confirm that no change had been made; the output showed shayankhan belonging only to its primary group, as expected.



```
shaonkhan@shaon:~$ sudo groupadd phyra
shaonkhan@shaon:~$ sudo tail /etc/group
pulse:x:131:
pulse-access:x:132:
gdm:x:133:
lxd:x:134:
sambashare:x:135:
shaonkhan:x:1002:
Triangle_2:x:1004:shaonkhan
Triangle_1:x:1005:
shayankhan:x:1000:
phyra:x:1006:
shaonkhan@shaon:~$ groups
shaonkhan sudo Triangle_2
shaonkhan@shaon:~$ groups shayankhan
shayankhan : shayankhan
shaonkhan@shaon:~$ sudo shayankhan -aG shaonkhan,phyra,Triangle_1
sudo: shayankhan: command not found
shaonkhan@shaon:~$ groups shayankhan
shayankhan : shayankhan
```

Figure 9: Incorrect command attempt and unchanged group membership of shayankhan

The corrected command `sudo usermod -aG shaonkhan,phyra,shayankhan shaonkhan` was first used to add shaonkhan itself to additional groups, and then `sudo usermod -aG shaonkhan, phyra, Triangle_1, Triangle_2 shayankhan` was executed to assign shayankhan to all four intended supplementary groups. The `id shayankhan` command was run immediately afterward to verify the result. The output confirmed:

```
uid=1000(shayankhan) gid=1000(shayankhan)
groups=1000(shayankhan),1002(shaonkhan),1004(Triangle_2),1005(Triangle_1),1006(phyra)
```

This confirmed that shayankhan was now a member of all four supplementary groups in addition to its own primary group.

```

shaonkhan@shaon:~$ sudo usermod -aG shaonkhan,phyra,shayankhan
Usage: usermod [options] LOGIN

Options:
  -b, --badnames          allow bad names
  -c, --comment COMMENT   new value of the GECOS field
  -d, --home HOME_DIR     new home directory for the user account
  -e, --expiredate EXPIRE_DATE set account expiration date to EXPIRE_DATE
  -f, --inactive INACTIVE set password inactive after expiration
                           to INACTIVE
  -g, --gid GROUP         force use GROUP as new primary group
  -G, --groups GROUPS     new list of supplementary GROUPS
  -a, --append            append the user to the supplemental GROUPS
                           mentioned by the -G option without removing
                           the user from other groups
  -h, --help             display this help message and exit
  -l, --login NEW_LOGIN   new value of the login name
  -L, --lock             lock the user account
  -m, --move-home        move contents of the home directory to the
                           new location (use only with -d)
  -o, --non-unique        allow using duplicate (non-unique) UID
  -p, --password PASSWORD use encrypted password for the new password
  -R, --root CHROOT_DIR   directory to chroot into
  -P, --prefix PREFIX_DIR prefix directory where are located the /etc/* files
  -s, --shell SHELL       new login shell for the user account
  -u, --uid UID           new UID for the user account
  -U, --unlock           unlock the user account
  -v, --add-subuids FIRST-LAST add range of subordinate uids
  -V, --del-subuids FIRST-LAST remove range of subordinate uids
  -w, --add-subgids FIRST-LAST add range of subordinate gids
  -W, --del-subgids FIRST-LAST remove range of subordinate gids
  -Z, --selinux-user SEUSER new SELinux user mapping for the user account

shaonkhan@shaon:~$ id shaonkhan
Show Applications an) gid=1002(shaonkhan) groups=1002(shaonkhan),27(sudo),1004(Triangle_2)
shaonkhan@shaon:~$

```

Figure 10: usermod help output displayed due to missing LOGIN argument in earlier attempt

```

shaonkhan@shaon:~$ sudo usermod -aG shaonkhan,phyra,shayankhan shaonkhan
shaonkhan@shaon:~$ id shaonkhan
uid=1002(shaonkhan) gid=1002(shaonkhan) groups=1002(shaonkhan),27(sudo),1004(Triangle_2),1000(shayankhan),1006(phyra)
shaonkhan@shaon:~$ id shayankhan
uid=1000(shayankhan) gid=1000(shayankhan) groups=1000(shayankhan)
shaonkhan@shaon:~$ sudo usermod -aG shaonkhan,phyra,Triangle_1,Triangle_2 shayankhan
shaonkhan@shaon:~$ id shayankhan
uid=1000(shayankhan) gid=1000(shayankhan) groups=1000(shayankhan),1002(shaonkhan),1004(Triangle_2),1005(Triangle_1),1006(phyra)
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shayankhan shaonkhan 0 May 26 09:59 testfile.txt
shaonkhan@shaon:~$ chgrp phyra testfile.txt
chgrp: changing group of 'testfile.txt': Operation not permitted
shaonkhan@shaon:~$ sudo chgrp phyra testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shayankhan phyra 0 May 26 09:59 testfile.txt
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$ chmod ug+s Downloads
shaonkhan@shaon:~$ ls -l
total 36
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Desktop
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:35 Documents
drwxr-sr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Downloads
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Music
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Pictures
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Public
drwx----- 3 shaonkhan shaonkhan 4096 May 6 15:25 snap
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Templates
-rw-rw-r-- 1 shayankhan phyra 0 May 26 09:59 testfile.txt
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Videos

```

Figure 11: Successful usermod command and id verification showing all group memberships

4.7 Modifying File Ownership

A test file was created using `touch testfile.txt`. Running `ls -l testfile.txt` immediately afterward showed the file with the following attributes: `-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 26 09:59 testfile.txt`. Both the owner and the group owner were shaonkhan, reflecting that the file was created in the shaonkhan session and inherited that account's primary group.

The ownership was then transferred to shayankhan using `sudo chown shayankhan testfile.txt`. Running `ls -l` again confirmed the change: `-rw-rw-r-- 1 shayankhan shaonkhan 0 May 26 09:59 testfile.txt`. The user owner field now showed shayankhan while the group owner remained shaonkhan, demonstrating that `chown` in this form affects only the user owner and leaves the group field unchanged.

```
shaonkhan@shaon:~$ touch testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 26 09:57 testfile.txt
shaonkhan@shaon:~$ 
shaonkhan@shaon:~$ 
shaonkhan@shaon:~$ sudo adduser shayankhan
Adding user `shayankhan' ...
Adding new group `shayankhan' (1000) ...
Adding new user `shayankhan' (1000) with group `shayankhan' ...
Creating home directory `/home/shayankhan' ...
Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: The password fails the dictionary check - it does not contain enough DIFFERENT characters
Retype new password:
passwd: password updated successfully
Changing the user information for shayankhan
Enter the new value, or press ENTER for the default
  Full Name []: Shayan Khan
    Room Number []: 1
    Work Phone []: 1111111111
    Home Phone []: 1111111111
      Other []:
Is the information correct? [Y/n] Y
shaonkhan@shaon:~$ ls /homee
ls: cannot access '/homee': No such file or directory
shaonkhan@shaon:~$ ls /home
shaonkhan  shayankhan
shaonkhan@shaon:~$ touch testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shaonkhan shaonkhan 0 May 26 09:59 testfile.txt
shaonkhan@shaon:~$ sudo chown shayankhan testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shayankhan shaonkhan 0 May 26 09:59 testfile.txt
```

Figure 12: File creation, ownership inspection, and `chown` applied to `testfile.txt`

4.8 Modifying File Group Ownership

The next step was to change the group ownership of `testfile.txt` from shaonkhan to phyra. The first attempt used the command `chgrp phyra testfile.txt` without `sudo`. The terminal returned the error "chgrp: changing group of 'testfile.txt': Operation not permitted." This error occurred because at this point in the session, the file's user owner was shayankhan, not shaonkhan. Since shaonkhan does not own the file, it cannot modify its group, even if it belongs to the target group phyra. Linux enforces the rule that only the file's owner can change its group, and only to a group the owner belongs to. A non-owner cannot perform this operation without root privileges.

The operation was then retried using `sudo chgrp phyra testfile.txt`. With elevated privileges, the command succeeded. Running `ls -l` again confirmed the change: `-rw-rw-r-- 1 shayankhan phyra 0 May 26 09:59 testfile.txt`. The group owner field now correctly showed `phyra`.

```
shaonkhan@shaon:~$ sudo usermod -aG shaonkhan,phyra,shayankhan shaonkhan
shaonkhan@shaon:~$ id shaonkhan
uid=1002(shaonkhan) gid=1002(shaonkhan) groups=1002(shaonkhan),27(sudo),1004(Triangle_2),1000(shayankhan),1006(phyra)
shaonkhan@shaon:~$ id shayankhan
uid=1000(shayankhan) gid=1000(shayankhan) groups=1000(shayankhan)
shaonkhan@shaon:~$ sudo usermod -aG shaonkhan,phyra,Triangle_1,Triangle_2 shayankhan
shaonkhan@shaon:~$ id shayankhan
uid=1000(shayankhan) gid=1000(shayankhan) groups=1000(shayankhan),1002(shaonkhan),1004(Triangle_2),1005(Triangle_1),1006(phyra)
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shayankhan shaonkhan 0 May 26 09:59 testfile.txt
shaonkhan@shaon:~$ chgrp phyra testfile.txt
chgrp: changing group of 'testfile.txt': Operation not permitted
shaonkhan@shaon:~$ sudo chgrp phyra testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shayankhan phyra 0 May 26 09:59 testfile.txt
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$ chmod ug+s Downloads
shaonkhan@shaon:~$ ls -l
total 36
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Desktop
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:35 Documents
drwsr-sr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Downloads
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Music
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Pictures
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Public
drwxr-xr-x 3 shaonkhan shaonkhan 4096 May 6 15:25 snap
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Templates
-rw-rw-r-- 1 shayankhan phyra 0 May 26 09:59 testfile.txt
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Videos
```

Figure 13: `chgrp` failure without `sudo` and successful execution with `sudo`, followed by `ls -l` verification

4.9 Applying Special Permission Bits (SGID)

The final step of the experiment involved applying the SGID permission bit to the Downloads directory, which was located in the shaonkhan home directory. The command `chmod ug+s Downloads` was executed. This applies both the SUID (`u+s`) and SGID (`g+s`) bits to the Downloads directory.

Following this, `ls -l` was executed to display the permission strings for all entries in the home directory. The line corresponding to Downloads read: `drwsr-sr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Downloads`. In this permission string, the third character of the owner permission triplet (normally `x` for execute) had changed to `s`, and similarly the third character of the group permission triplet had changed to `s`. These two `s` characters indicate that both the SUID and SGID bits are active on the directory. For the Downloads directory, the SGID effect is particularly meaningful: any file created inside Downloads by any user will inherit the group ownership of the directory itself rather than the creating user's primary group. This ensures consistent group ownership across all files placed in the directory, which is essential for collaborative access patterns.

```

shaonkhan@shaon:~$ sudo usermod -aG shaonkhan,phyra,shayankhan shaonkhan
shaonkhan@shaon:~$ id shaonkhan
uid=1002(shaonkhan) gid=1002(shaonkhan) groups=1002(shaonkhan),27(sudo),1004(Triangle_2),1000(shayankhan),1006(phyra)
shaonkhan@shaon:~$ id shayankhan
uid=1000(shayankhan) gid=1000(shayankhan) groups=1000(shayankhan)
shaonkhan@shaon:~$ sudo usermod -aG shaonkhan,phyra,Triangle_1,Triangle_2 shayankhan
shaonkhan@shaon:~$ id shayankhan
uid=1000(shayankhan) gid=1000(shayankhan) groups=1000(shayankhan),1002(shaonkhan),1004(Triangle_2),1005(Triangle_1),1006(phyra)
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shayankhan shaonkhan 0 May 26 09:59 testfile.txt
shaonkhan@shaon:~$ chgrp phyra testfile.txt
chgrp: changing group of 'testfile.txt': Operation not permitted
shaonkhan@shaon:~$ sudo chgrp phyra testfile.txt
shaonkhan@shaon:~$ ls -l testfile.txt
-rw-rw-r-- 1 shayankhan phyra 0 May 26 09:59 testfile.txt
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$
shaonkhan@shaon:~$ chmod ug+s Downloads
shaonkhan@shaon:~$ ls -l
total 36
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Desktop
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:35 Documents
drwsr-sr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Downloads
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Music
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Pictures
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Public
drwx----- 3 shaonkhan shaonkhan 4096 May 6 15:25 snap
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Templates
-rw-rw-r-- 1 shayankhan phyra 0 May 26 09:59 testfile.txt
drwxr-xr-x 2 shaonkhan shaonkhan 4096 May 6 15:25 Videos

```

Figure 14: Permission string showing drwsr-sr-x after chmod ug+s applied to Downloads

5. Discussion and Analysis

Several observations from the experiment merit detailed discussion, as they reveal important aspects of how Linux manages users, groups, and permissions in practice.

5.1 Command Syntax Errors and the usermod Help Menu

During Step 4.6, the accidental command `sudo shayankhan -aG shaonkhan,phyra,Triangle_1` caused `sudo` to interpret the username `shayankhan` as the executable to run. Since no program by that name exists in the system `PATH`, `sudo` returned "command not found." This illustrates a fundamental Linux behavior: the shell looks up the first token of a command in the directories listed in `PATH`. If it is not found, the command fails immediately, regardless of any flags that follow.

In a related event captured in the terminal, an incomplete `usermod` call was attempted without specifying the `LOGIN` argument at the end. When `usermod` receives no username argument, it cannot determine which account to modify and therefore prints its full help and usage documentation to standard error before exiting. This behavior is intentional: well-written UNIX utilities display usage information when called incorrectly, providing immediate guidance on proper syntax without requiring the user to consult a separate manual page. The verbose help output for `usermod` also serves as a useful reference for all available options, including account expiration, home directory relocation, shell assignment, and UID modification.

5.2 Ownership Constraints on `chgrp`

The failure of the unprivileged `chgrp phyra testfile.txt` command in Step 4.8 is a direct expression of one of Linux's foundational security principles. The file's user owner was `shayankhan` at the time of the attempt, having been changed in Step 4.7. The active session was running as `shaonkhan`, meaning `shaonkhan` was not the file owner. Linux explicitly prohibits non-owners from changing the group ownership of a file, even if the non-owner is a member of the target group and would benefit from the change. This restriction prevents one user from silently re-associating another user's files with groups the owner did not intend.

Even the file owner, when changing group ownership without root privilege, is restricted: the owner can only change the group to one of the groups to which the owner personally belongs. This prevents a file owner from assigning their file to a group that the owner does not have membership in, which could be used as a method to make the file accessible to a group the owner should not interact with. These layered restrictions collectively protect the integrity of the group-based access control model.

5.3 Security and Collaboration Benefits of SGID Directories

The application of the SGID bit to the Downloads directory in Step 4.9 illustrates an important tool for managing shared workspaces. In a typical Linux environment without SGID, when multiple users belonging to a common group write files to the same directory, each file inherits the primary group of its creator rather than any shared group. If users have different primary groups, the resulting files will have different group owners, making it difficult for other group members to access them uniformly.

With SGID enabled on a shared directory, all files created within it automatically inherit the group ownership of the directory itself. If the directory is owned by a shared project group, every file written into it becomes accessible to all members of that group regardless of who created it. This removes the need for each contributor to manually change the group ownership of every file they create, which is both inconvenient and prone to oversight. SGID directories are therefore widely used in multi-user environments such as project directories, shared research repositories, and administrative workspace folders.

5.4 Privilege Separation and Administrative Security

Throughout the experiment, the sudo mechanism was consistently required to perform administrative operations such as creating users, deleting accounts, creating groups, assigning supplementary group memberships, and changing file ownership. This pattern reflects the principle of least privilege, which holds that a user or process should operate with only the minimum level of access required to complete its current task. By restricting administrative commands to privileged execution, Linux limits the potential damage caused by mistakes or misuse in an ordinary session.

The separation of `/etc/passwd` and `/etc/shadow` further supports this principle. An ordinary program reading account information to map UIDs to names only needs access to `/etc/passwd`, which is world-readable. No ordinary process requires access to the password hashes, and restricting `/etc/shadow` to root prevents credential theft by unprivileged programs or users. The combination of shadow passwords, UID-based permission enforcement, and group-based access control forms a layered security architecture that makes Linux suitable for multi-user deployment in institutional environments.

6. Conclusion

This laboratory session provided comprehensive hands-on experience with the user and group management subsystem of Linux. Through a series of structured exercises, the complete lifecycle of a user account was observed and practiced: account creation using `adduser`, password management using `passwd`, interactive session switching using `su` and `su -`, and account removal using `userdel -r`. Each step produced observable system behavior that reinforced the underlying theory.

The distinction between `su` and `su -` proved practically significant. The non-login form of `su` left the shell in the calling user's directory, which resulted in a Permission denied error when the new user attempted to list that directory's contents, since they lacked the necessary permissions. The login form `su -` correctly initialized the environment by changing to the target user's home directory and loading appropriate shell configuration files.

Group management tasks demonstrated how Linux organizes users into access control units. The creation of the `phyra` group, verified through `tail /etc/group`, showed how GIDs are assigned sequentially. The assignment of `shayankhan` to multiple supplementary groups using `usermod -aG`, confirmed by `id shayankhan`, illustrated how a single account can access resources owned by several different groups simultaneously.

The file ownership exercises clarified the distinction between user ownership and group ownership, and the asymmetric privilege requirements for each. While `chown` requires root privilege to transfer ownership to another user, `chgrp` by an ordinary user is constrained to files they own and groups they belong to. The failure of the unprivileged `chgrp` command, followed by its success under `sudo`, illustrated these constraints directly.

Finally, the application of the SGID permission bit using `chmod ug+s`, and its verification through the `s` characters visible in `ls -l` output, demonstrated a practical mechanism for enforcing consistent group ownership in shared directories. The session concluded with a clear understanding of how special permission bits extend the basic read-write-execute model to support more sophisticated access control patterns suited to collaborative and multi-user environments.

Overall, the experiment confirmed that Linux's approach to user, group, and permission management is both precise and flexible, enforcing strict security boundaries while providing the administrative tools necessary to configure appropriate access for any organizational requirement.



Institute of Information Technology

Jahangirnagar University, Savar, Dhaka-1342

Lab Report - 06

Lab Title: Introduction to Shell Scripting

Course Code: ICT-3102

Course Title: Operating System Lab

Submitted by:

Md. Shaon Khan

Id: 1984

Batch: 52 Batch

Session: 2022-2023

Supervised by:

Mehrin Anannya

Assistant Professor, Institute of Information Technology
Jahangirnagar University

Date: May 5, 2026

1. Objective

The objective of this laboratory session is to introduce students to the fundamentals of shell scripting in a Unix/Linux environment using the Bash interpreter. By the end of this session, students are expected to:

- Create and execute shell scripts using vim, and manage permissions with chmod.
- Declare and use user-defined variables, and access built-in environment variables (\$HOME, \$SHELL).
- Produce formatted output using echo, echo -e, echo -n, and printf.
- Capture dynamic runtime data via command substitution \$(command).
- Accept interactive user input using read.
- Implement conditional logic with if, if-else, if-elif-else, and case statements.
- Perform iteration using for (list and C-style), while, and until loops, with break and continue for flow control.
- Perform integer arithmetic using \$(()) expansion with +, -, *, /, %, and **.
- Define reusable functions, pass arguments via \$1/\$2, and use return with \$? to retrieve exit status.

2. Theory

A shell script is a plain text file containing a sequence of commands that the Bash interpreter executes sequentially. Shell scripting is fundamental to Unix/Linux system administration, allowing repetitive tasks to be automated and packaged into reusable executable programs.

Shebang and Permissions: The first line `#!/bin/bash` (shebang) instructs the OS kernel to use `/bin/bash` as the interpreter. `chmod +x` grants the execute bit required for `./execution`; `sh script.sh` bypasses this requirement by invoking the interpreter directly.

Variables and Output: Variables are assigned without spaces (`name="Shaon"`) and referenced with `$ ($name)`. Environment variables (`$HOME`, `$SHELL`) are pre-set by the OS. `echo` prints to stdout; `echo -e` interprets escape sequences; `echo -n` suppresses the trailing newline; `printf` provides C-style format specifiers for precise alignment.

Conditionals: `if-then-elif-else-fi` structures evaluate conditions in `[ ]`. Numeric operators: `-eq`, `-ne`, `-gt`, `-ge`, `-lt`, `-le`. The `case-esac` statement matches a variable against explicit patterns terminated with `;;` and uses `*` as a default wildcard.

Loops: `for` iterates over lists or uses C-style arithmetic `(( init; cond; incr ))`. `while` repeats while its condition is true; `until` repeats until its condition becomes true. `break` exits a loop; `continue` skips to the next iteration. Arithmetic expansion `$(( ))` performs native integer arithmetic including `**`, with integer division truncating results.

Functions: Functions are defined as `name() { }` and called by name with arguments. Inside a function, `$1` and `$2` are the function's own positional parameters. `return` sets the exit status code; `?` retrieves it immediately after the call. Return values are limited to 0-255; larger values require `echo` and command substitution.

3. Commands Used

The table below lists every command, operator, and shell construct used throughout this laboratory session.

Command / Construct	Description
#!/bin/bash	Shebang line; specifies the Bash interpreter for the script
vim	Text editor used to create and edit shell scripts
more	Displays file content one screen at a time
chmod +x	Grants execute permission to a shell script file
chmod -x	Removes execute permission from a shell script file
./script.sh	Executes a shell script from the current directory
sh script.sh	Runs a script using sh interpreter; does not need execute bit
ls / ls -l	Lists directory contents; -l shows permissions in detail
echo	Prints text or variable values to standard output
echo -e	Enables interpretation of backslash escape sequences (\n, \t)
echo -n	Prints text without a trailing newline
printf	Formats and prints data using C-style format specifiers
#	Denotes a comment line; ignored during script execution
read	Reads a line of user input from stdin into a variable
\$VARIABLE	References the value stored in a variable
\$HOME	Environment variable: path to the user's home directory
\$SHELL	Environment variable: path of the active shell interpreter
\$(command)	Command substitution: replaces expression with command output
\$(())	Arithmetic expansion for integer arithmetic
if / fi	Conditional block executed when condition is true
if-else / fi	Two-branch conditional block
if-elif-else / fi	Multi-branch conditional block
case / esac	Pattern-matching multi-branch statement
-eq / -ne	Numeric: equal to / not equal to
-gt / -ge	Numeric: greater than / greater than or equal to
-lt / -le	Numeric: less than / less than or equal to
for i in list	Iterates over a list of values
for ((; ;))	C-style numeric for loop with init/condition/increment
while / do / done	Loops while the condition remains true
until / do / done	Loops until the condition becomes true
break	Exits the enclosing loop immediately

continue	Skips current iteration; jumps to next loop cycle
+ - * / %	Arithmetic: add, subtract, multiply, integer divide, modulo
**	Arithmetic: exponentiation
function name()	Defines a reusable named function
\$1, \$2	Positional parameters: function/script arguments
return	Exits a function and sets its return/exit status code
\$?	Special variable: exit status of the last command or function

4. Procedure

We began the lab session from an empty working directory. The terminal prompt used throughout was:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$
```

All scripts were created using the vim text editor, saved with :wq, then made executable with chmod +x before execution. Each subsection below documents the exact script, output, and screenshots from the actual lab session.

4.1 Creating a Shell Script, Shebang Line, and chmod

Purpose: To understand how a shell script is created using vim, how execute permissions are controlled using chmod +x and chmod -x.

Command: vim welcome.sh → chmod +x welcome.sh → ./welcome.sh → sh welcome.sh

Script:

```
echo "Welcome to OS Lab 06"
```

Screenshot:

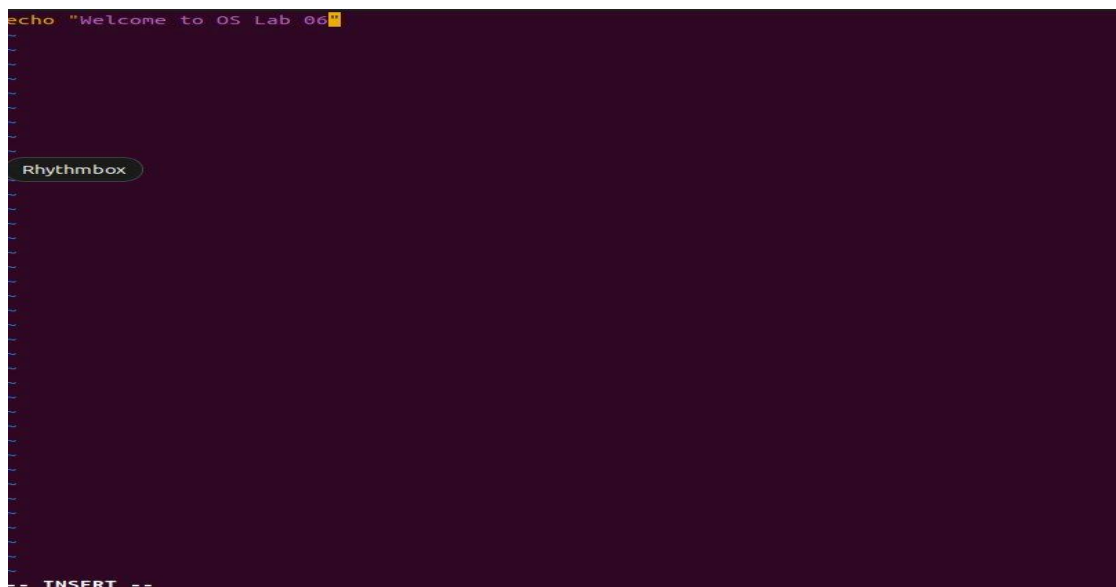


Fig 4.1(a) — vim editor showing welcome.sh content

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ ls
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim welcome.sh
```

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ more welcome.sh
echo "Welcome to OS Lab 06"
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ls -l
total 4
-rwxrwxr-x 1 shaonkhan shaonkhan 28 May 26 13:20 welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./welcome.sh
Welcome to OS Lab 06
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod -x welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./welcome.sh
bash: ./welcome.sh: Permission denied
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./welcome.sh
Welcome to OS Lab 06
shaonkhan@shaon:~/OS_Lab/LAB_06$ sh welcome.sh
Welcome to OS Lab 06

```

Screenshot:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ ls
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ more welcome.sh
echo "Welcome to OS Lab 06"
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ls -l
total 4
-rwxrwxr-x 1 shaonkhan shaonkhan 28 May 26 13:20 welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./welcome.sh
Welcome to OS Lab 06
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod -x welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./welcome.sh
bash: ./welcome.sh: Permission denied
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./welcome.sh
Welcome to OS Lab 06
shaonkhan@shaon:~/OS_Lab/LAB_06$ sh welcome.sh
Welcome to OS Lab 06

```

Fig 4.1(b) — Terminal showing chmod, ./welcome.sh, Permission denied, and sh welcome.sh

Observation: The `ls -l` output confirmed `chmod +x` set the executable bit (`rwxrwxr-x`). Removing permissions with `chmod -x` immediately caused a 'Permission denied' error on `./welcome.sh`, but `sh welcome.sh` still worked because it invokes the interpreter directly without requiring the execute bit. This is the fundamental distinction between the two execution methods.

4.2 Variables

Purpose: To understand how to declare and use user-defined variables in a shell script, assigning a value and referencing it with the `$` prefix.

Command: `vim variable.sh` → `chmod +x variable.sh` → `./variable.sh`

Script:

```

name="Shaon"
echo "My name is $name"

```


Screenshot:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim variable.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./variable.sh
bash: ./variable.sh: Permission denied
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x variable.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./variable.sh
My name is Shaon
shaonkhan@shaon:~/OS_Lab/LAB_06$

```

Fig 4.2(c) — Terminal output of variable.sh showing 'My name is Shaon'

Observation: The variable name was assigned the string "Shaon" without spaces around =. Referencing \$name inside double quotes caused Bash to substitute it with its value at runtime. The initial 'Permission denied' error reinforced that chmod +x is a prerequisite for ./execution.

4.3 echo -e and printf (Formatted Output)

Purpose: To demonstrate echo -e interpreting \n escape sequences to produce multi-line output, and printf performing the same with C-style format strings.

Command: vim comment.sh → chmod +x comment.sh → ./comment.sh

Script:

```

echo -e "Line 1\nLine 2\nLine 3"
printf "Line 1\nLine 2\nLine 3\n"

```

Screenshot:

```

echo -e "Line 1\nLine 2\nLine 3"
printf "Line 1\nLine 2\nLine 3\n"

```

Fig 4.3(a) — vim editor showing echo -e and printf in comment.sh

Output:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim comment.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x comment.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./comment.sh
Line 1
Line 2
Line 3
Line 1
Line 2
Line 3

```

Screenshot:


```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim comment.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x comment.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./comment.sh
Line 1
Line 2
Line 3
Line 1
Line 2
Line 3

```

Fig 4.3(b) — Terminal showing identical 3-line output from both echo -e and printf

Observation: Both echo -e and printf produced identical three-line output by interpreting \n as a newline character. Plain echo (without -e) would print the literal text '\n'. printf is preferred when precise column widths or numeric formatting are needed.

4.4 Environment Variables (\$HOME, \$SHELL)

Purpose: To explore pre-defined environment variables that the operating system automatically provides to all processes, specifically \$HOME and \$SHELL.

Command: vim env.sh → chmod +x env.sh → ./env.sh

Script:

```

echo "Home directory: $HOME"
echo "Current directory: $SHELL"

```


4.5 Command Substitution (\$(date))

Purpose: To demonstrate command substitution using the \$(command) syntax, which executes a command and captures its output into a variable for use inside the script.

Command: vim date.sh → chmod +x date.sh → ./date.sh

Script:

```
date_today=$(date)

"echo Today is $date_today"
```

Screenshot:

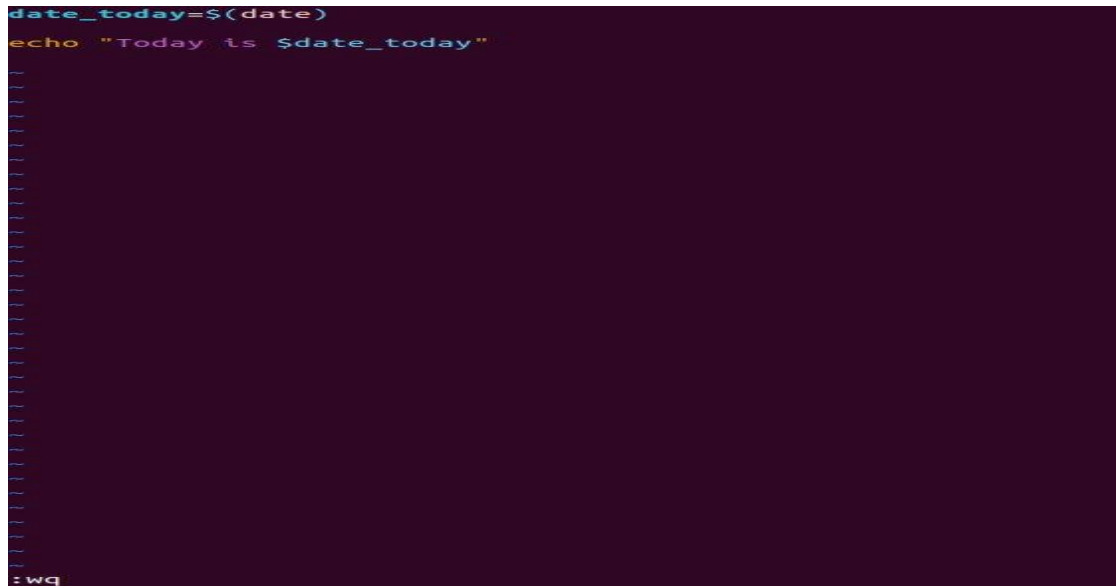


Fig 4.5(a) — vim editor showing date.sh with \$(date) command substitution

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim date.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x date.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./date.sh
Today is Wed 03 Jun 2026 10:28:14 PM +06
```

Screenshot:

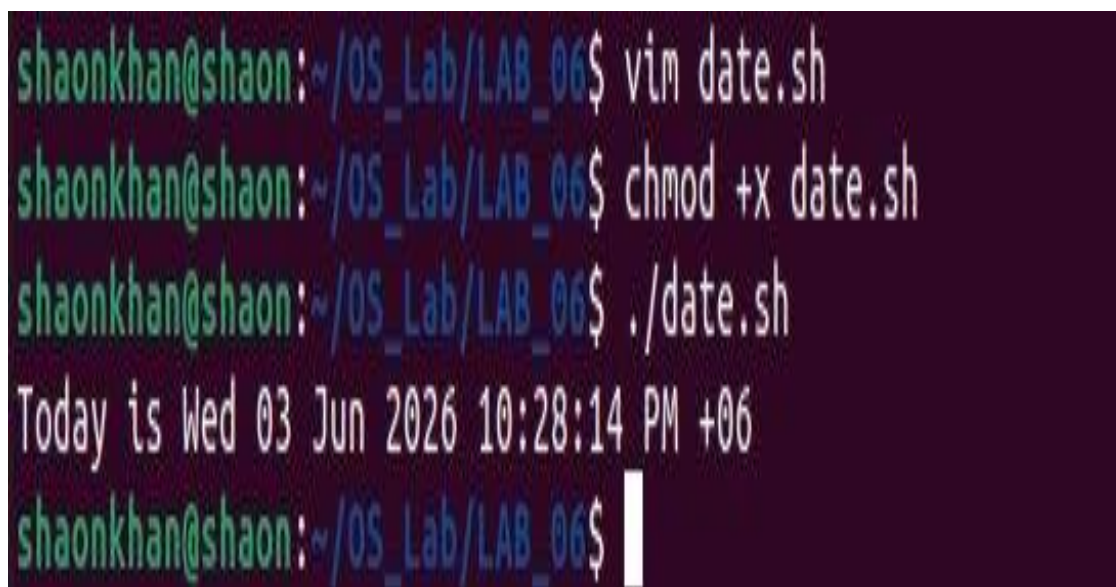


Fig 4.5(b) — Terminal showing live system date captured via command substitution

Observation: \$(date) executed the date command in a subshell and stored its full output — including date, time, and Bangladesh Standard Time offset (+06) — in date_today. This makes scripts dynamic, incorporating live system state without user input.

4.6 Reading User Input with read

Purpose: To demonstrate how read accepts interactive input from the user at runtime and stores it in a variable for use later in the script.

Command: vim input.sh → chmod +x input.sh → ./input.sh

Script:

```
echo "Enter your name:"
read username

echo "Welcome, $username!"
```

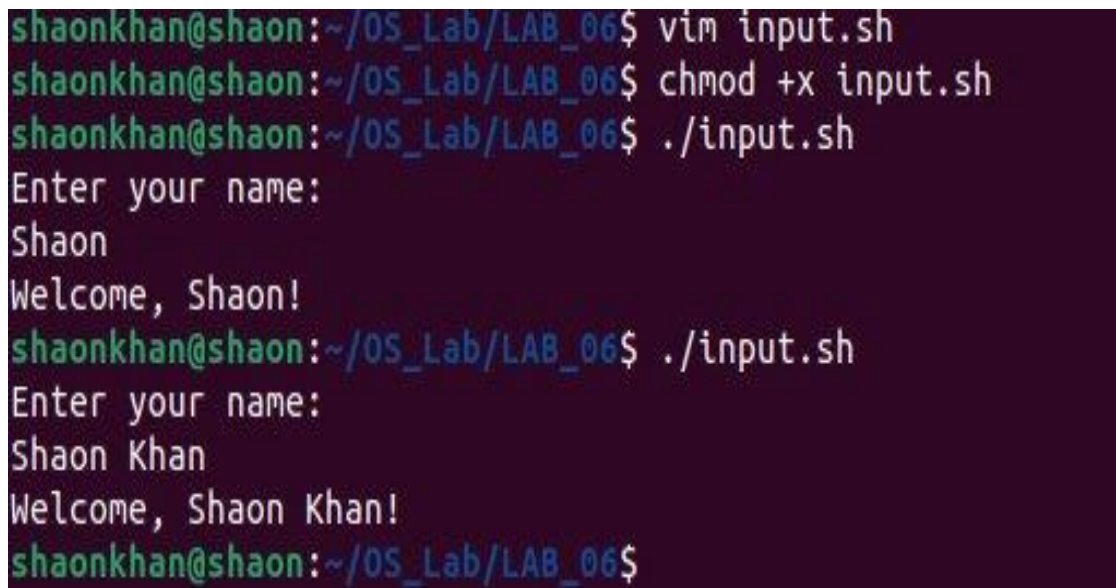
Screenshot:



Fig 4.6(a) — vim editor showing input.sh

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim input.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x input.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./input.sh
Enter your name:
Shaon
Welcome, Shaon!
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./input.sh
Enter your name:
Shaon Khan
Welcome, Shaon Khan!
```

Screenshot:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim input.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x input.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./input.sh
Enter your name:
Shaon
Welcome, Shaon!
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./input.sh
Enter your name:
Shaon Khan
Welcome, Shaon Khan!
shaonkhan@shaon:~/OS_Lab/LAB_06$
```

Fig 4.6(b) — Terminal showing input.sh run twice with 'Shaon' and 'Shaon Khan'

Observation: read pauses execution and waits for the user to type a value and press Enter. Testing with both 'Shaon' and 'Shaon Khan' confirmed that read captures the entire line including spaces into a single variable. This is the foundation of interactive, menu-driven shell programs.

4.7 if Statement

Purpose: To implement a simple if statement that evaluates a numeric condition and executes a block only when the condition is true.

Command: vim if.sh → chmod +x if.sh → ./if.sh

Script:

```
num=10
if [ $num -gt 5 ]
then
    echo "Number is greater than 5"
fi
```

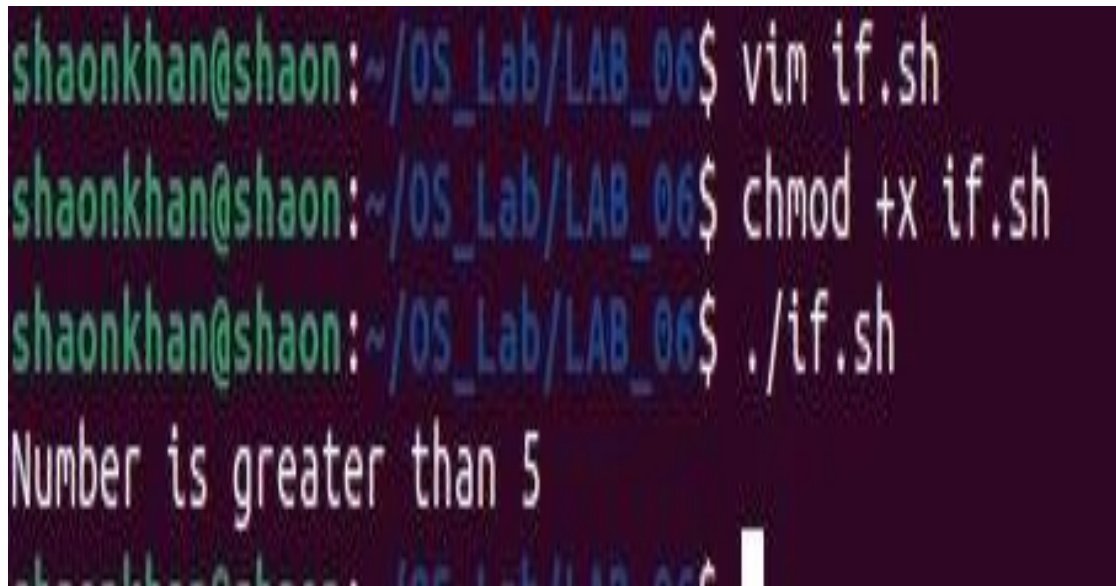
Screenshot:

```
num=10
if [ $num -gt 5 ]
then
    echo "Number is greater than 5"
fi
:~
```

Fig 4.7(a) — vim editor showing if.sh

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim if.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x if.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./if.sh
Number is greater than 5
```

Screenshot:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim if.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x if.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./if.sh
Number is greater than 5
```

Fig 4.7(b) — Terminal output confirming the if condition evaluated true

Observation: With num=10, the condition [\$num -gt 5] evaluated to true and the echo statement executed. The -gt operator performs numeric greater-than comparison. fi closes the if block. If num had been 3, no output would appear since the entire block would be skipped.

4.8 if-else Statement

Purpose: To extend the if construct with an else branch, ensuring exactly one of two blocks always executes based on the condition result.

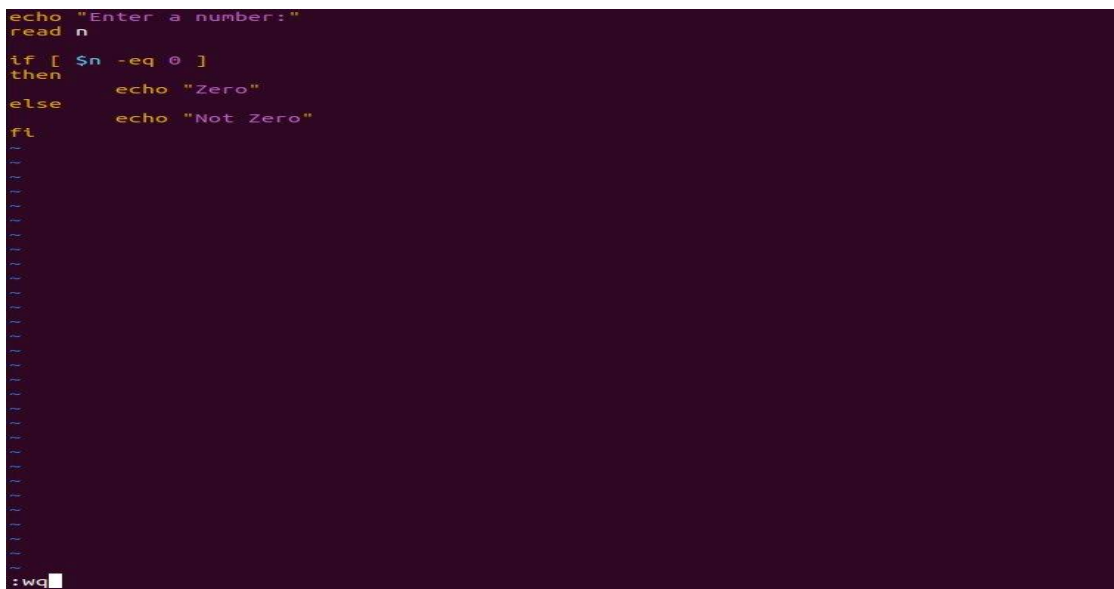
Command: `vim iselse.sh → chmod +x iselse.sh → ./iselse.sh`

Script:

```
echo "Enter a number:"
read n

if [ $n -eq 0 ]
then
    echo "Zero"
else
    echo "Not Zero"
fi
```

Screenshot:



```
echo "Enter a number:"
read n
if [ $n -eq 0 ]
then
    echo "Zero"
else
    echo "Not Zero"
fi
:wq
```

Fig 4.8(a) — vim editor showing iselse.sh

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim iselse.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x iselse.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./iselse.sh
Enter a number:
5
Not Zero
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./iselse.sh
Enter a number:
0
Zero
```

Screenshot:


```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim iselse.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x iselse.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./iselse.sh
Enter a number:
5
Not Zero
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./iselse.sh
Enter a number:
0
Zero

```

Fig 4.8(b) — Terminal showing iselse.sh with inputs 5 (Not Zero) and 0 (Zero)

Observation: Input 5 caused the if condition ($\$n \neq 0$) to fail, executing the else branch ('Not Zero'). Input 0 matched the condition, printing 'Zero'. The if-else guarantees that exactly one branch executes in every execution, providing complete binary decision coverage.

4.9 if-elif-else Statement

Purpose: To implement multi-branch conditional logic using if-elif-else, allowing range-based classification where only the first matching condition executes.

Command: `vim grade.sh → ./grade.sh`

Script:

```

echo "Enter your marks:"
read marks

if [ $marks -ge 80 ]
then
    echo "Grade A+"
elif [ $marks -ge 70 ]
then
    echo "Grade A"
elif [ $marks -ge 60 ]
then
    echo "Grade B"
else
    echo "Grade F"
fi

```


4.10 case Statement

Purpose: To implement exact-value pattern matching using the case statement, which is more readable than chained if-elif blocks when matching a variable against a fixed set of options.

Command: `vim choice.sh → chmod +x choice.sh → ./choice.sh`

Script:

```
echo "Enter a choice (y/n):"
read choice

case $choice in
    y) echo "You chose YES" ;;
    n) echo "You chose NO" ;;
    *) echo "Invalid choice" ;;
esac
```

Screenshot:

```
echo "Enter a choice (y/n):"
read choice
case $choice in
    y) echo "You chose YES" ;;
    n) echo "You chose NO" ;;
    *) echo "Invalid choice" ;;
esac
bm
```

Fig 4.10(a) — vim editor showing case statement in choice.sh

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim choice.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x choice.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./choice.sh
Enter a choice (y/n):
y
You chose YES
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./choice.sh
Enter a choice (y/n):
o
Invalid choice
```

Screenshot:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim choice.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x choice.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./choice.sh
Enter a choice (y/n):
y
You chose YES
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./choice.sh
Enter a choice (y/n):
o
Invalid choice

```

Fig 4.10(b) — Terminal showing choice.sh: 'y' matched YES, 'o' fell through to wildcard

Observation: Input 'y' matched the first pattern and printed 'You chose YES'. Input 'o' matched none of the explicit patterns and was caught by the wildcard *) printing 'Invalid choice'. The ;; terminator closes each branch. The * wildcard is equivalent to the default clause in C's switch statement, ensuring no input goes unhandled.

4.11 for Loop (List-Based and C-Style)

Purpose: To demonstrate iteration using the standard Bash for loop (over a list of values) and the C-style for loop using arithmetic expressions for init, condition, and increment.

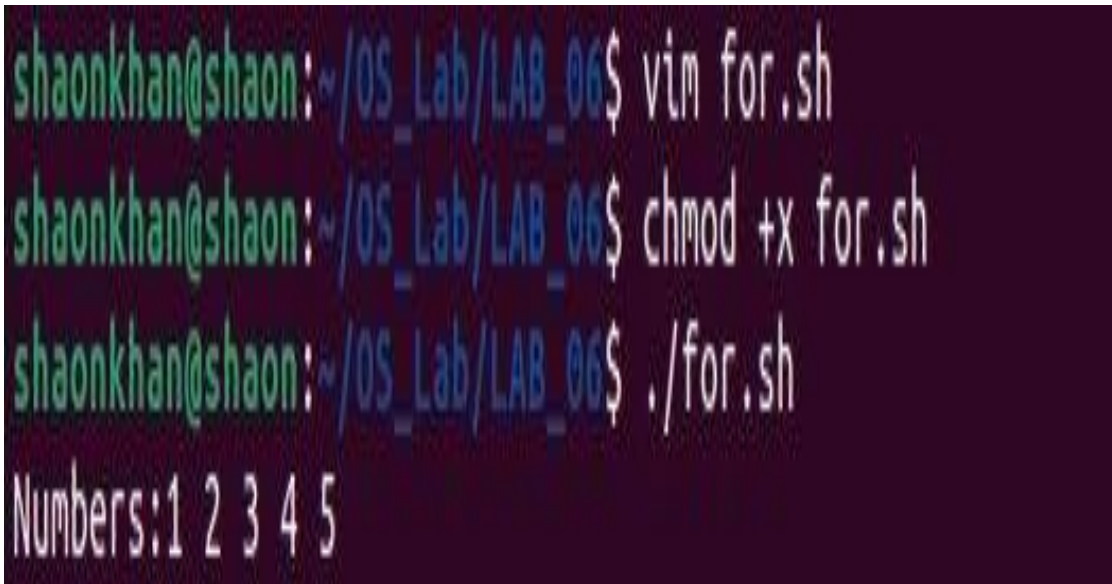
Command: vim for.sh / vim cfor.sh → chmod +x → ./for.sh / ./cfor.sh

Script (for.sh — list-based):

```

echo -n "Numbers:"
for i in 1 2 3 4 5
do
    echo -n "$i "
done
echo

```


Screenshot:


```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim for.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x for.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./for.sh
Numbers:1 2 3 4 5

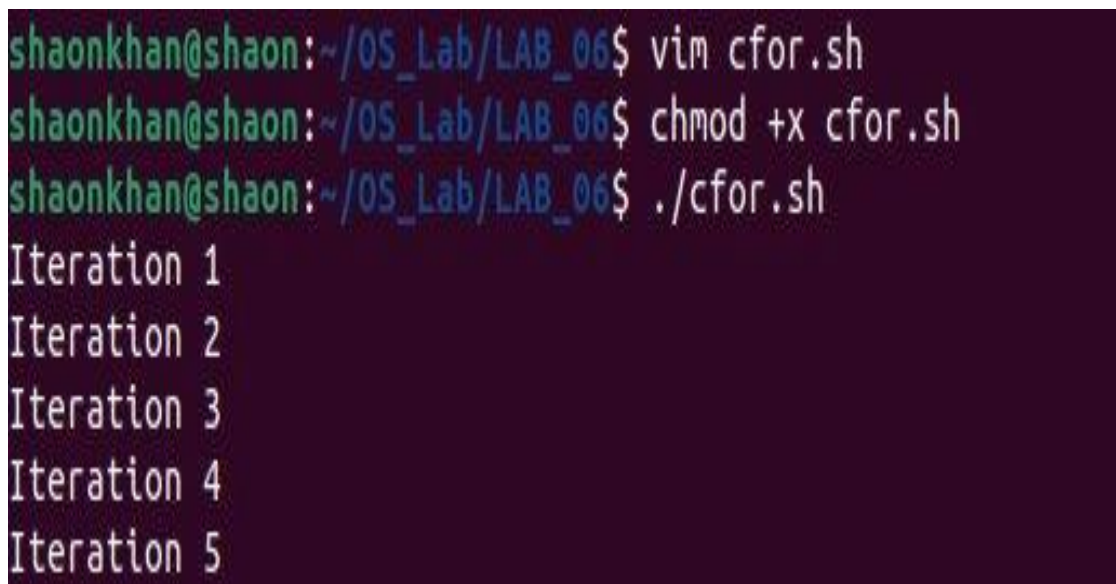
```

Fig 4.11(c) — Terminal output of for.sh printing numbers on a single line

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim cfor.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x cfor.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./cfor.sh
Iteration 1
Iteration 2
Iteration 3
Iteration 4
Iteration 5

```

Screenshot:


```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim cfor.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x cfor.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./cfor.sh
Iteration 1
Iteration 2
Iteration 3
Iteration 4
Iteration 5

```

Fig 4.11(d) — Terminal output of cfor.sh showing 5 iterations

Observation: The list-based for loop iterated over the literal values 1 2 3 4 5. `echo -n` suppressed newlines so all values appeared on a single line, followed by a bare `echo` to terminate the line. The C-style for loop used arithmetic expressions ((`i=1`; `i<=num`; `i++`)) familiar to C/Java programmers. Both loops iterated exactly 5 times, confirming correct behaviour.

4.12 while Loop

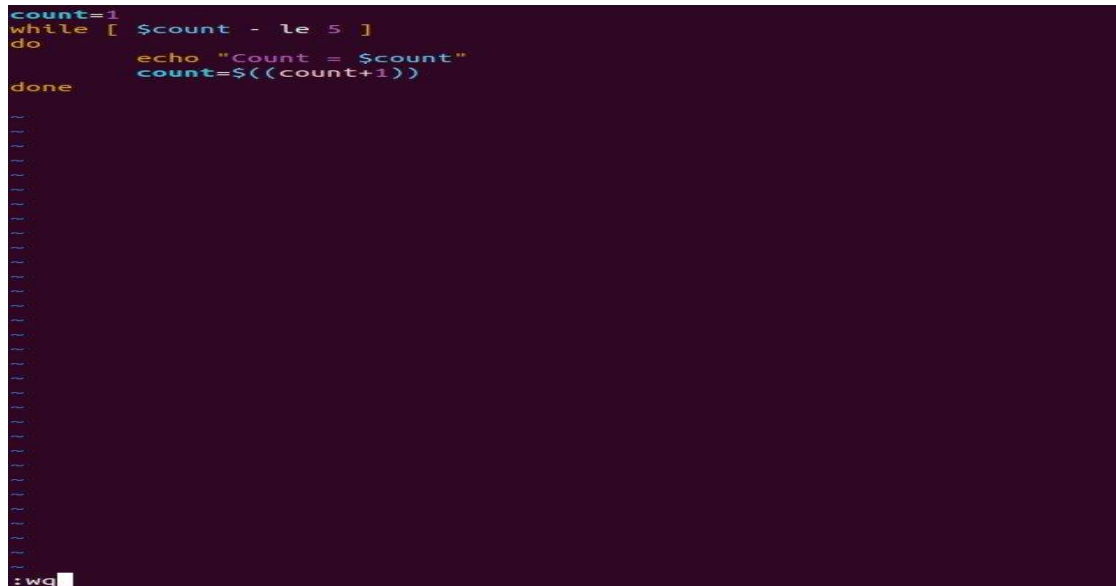
Purpose: To implement a while loop that continues executing as long as a condition remains true, and to observe how a syntax error in the condition is diagnosed and corrected.

Command: `vim while.sh → chmod +x while.sh → ./while.sh`

Script:

```
count=1
while [ $count -le 5 ]
do
    echo "Count = $count"
    count=$((count+1))
done
```

Screenshot:



```
count=1
while [ $count - le 5 ]
do
    echo "Count = $count"
    count=$((count+1))
done
```

Fig 4.12(a) — vim editor showing while.sh

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim while.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x while.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./while.sh
./while.sh: line 2: [: syntax error: '-' unexpected
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim while.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./while.sh
Count = 1
Count = 2
Count = 3
Count = 4
Count = 5
```

Screenshot:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim while.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x while.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./while.sh
./while.sh: line 2: [: syntax error: '-' unexpected
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim while.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./while.sh
Count = 1
Count = 2
Count = 3
Count = 4
Count = 5

```

Fig 4.12(b) — Terminal showing initial syntax error in while.sh, then corrected output

Observation: On first execution, a syntax error on line 2 was reported: '[: syntax error: '-' unexpected'. This was caused by a missing space between [and the condition, or an incorrectly typed operator. After opening vim to fix the condition spacing, the script executed correctly, printing Count = 1 through Count = 5. This practical debugging experience highlighted that Bash is sensitive to spacing inside [] test expressions.

4.13 until Loop

Purpose: To demonstrate the until loop, which is the logical inverse of while — it continues looping as long as its condition is false, and exits when the condition becomes true.

Command: vim unt.sh → chmod +x unt.sh → ./unt.sh

Script:

```

n=1

until [ $n -gt 5 ]
do
    echo "n = $n"
    n=$((n+1))
done

```

Screenshot:

```

1 n=1
2
3 until [ $n -gt 5 ]
4 do
5     echo "n = $n"
6     n=$((n+1))
7 done
8
9
10 :wq

```

Fig 4.13(a) — vim editor showing unt.sh with until loop

Output:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim unt.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x unt.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./unt.sh
n = 1
n = 2
n = 3
n = 4
n = 5

```

Screenshot:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim unt.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x unt.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./unt.sh
n = 1
n = 2
n = 3
n = 4
n = 5
shaonkhan@shaon:~/OS_Lab/LAB_06$

```

Fig 4.13(b) — Terminal output of unt.sh printing n = 1 through n = 5

Observation: The until loop ran while `[ $n -gt 5 ]` was false — that is, while n was 1 through 5. When n reached 6, the condition became true and the loop exited. The output was identical to what a while `[ $n -le 5 ]` loop would produce, confirming that until is the logical complement of while. The `n=$((n+1))` arithmetic expansion incremented the counter on each iteration.

4.14 break and continue

Purpose: To demonstrate how break terminates a loop early when a condition is met, and how continue skips the remaining commands in the current iteration and resumes at the next.

Command: vim breakcon.sh → chmod +x breakcon.sh → ./breakcon.sh

Script:

```
for i in 1 2 3 4 5
do
    if [ $i -eq 3 ]
    then
        break
    fi
    echo "i = $i"
done
```

Screenshot:

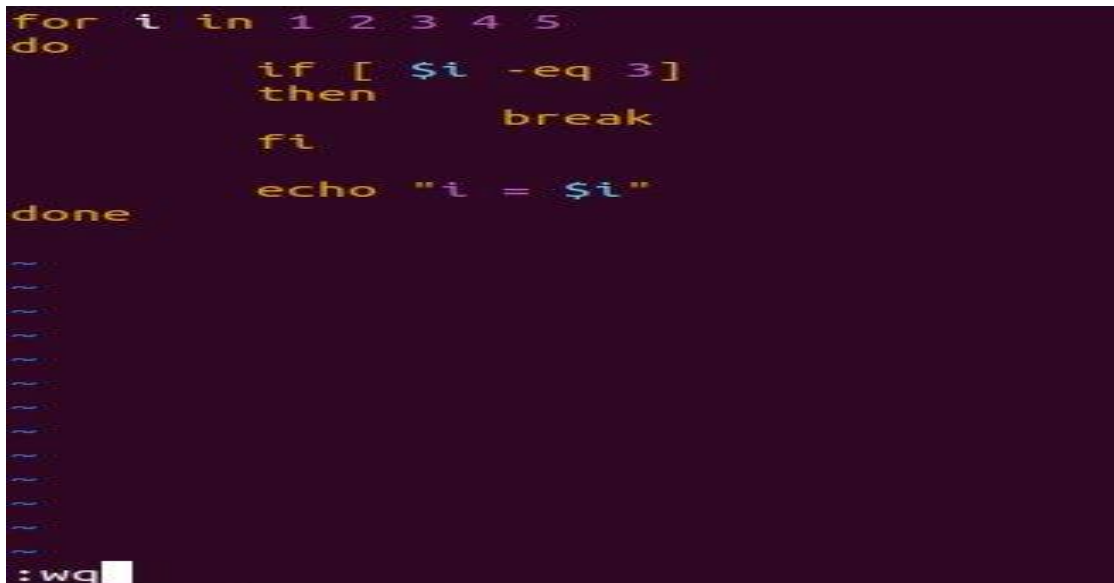


Fig 4.14(a) — vim editor showing breakcon.sh with break inside for loop

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim breakcon.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x breakcon.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./breakcon.sh
./breakcon.sh: line 3: [: missing `]'
i = 1
./breakcon.sh: line 3: [: missing `]'
i = 2
./breakcon.sh: line 3: [: missing `]'
i = 3
./breakcon.sh: line 3: [: missing `]'
i = 4
./breakcon.sh: line 3: [: missing `]'
i = 5
```


Screenshot:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim breakcon.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x breakcon.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./breakcon.sh
./breakcon.sh: line 3: [: missing `]'
i = 1
./breakcon.sh: line 3: [: missing `]'
i = 2
./breakcon.sh: line 3: [: missing `]'
i = 3
./breakcon.sh: line 3: [: missing `]'
i = 4
./breakcon.sh: line 3: [: missing `]'
i = 5

```

Fig 4.14(b) — Terminal showing breakcon.sh output with syntax warning on each iteration

Observation: The terminal output shows a "[: missing `]" warning on each iteration because the closing bracket of the if condition was missing ([\$i -eq 3] was written without the closing] on the same token). Despite the warning, Bash continued executing and printed i = 1 through i = 5, meaning the break condition was not correctly evaluated and the loop ran to completion. This highlighted that missing brackets in Bash test expressions cause runtime warnings and incorrect conditional behaviour, demonstrating the importance of exact syntax in shell scripting.

4.15 Arithmetic Expansion \$(())

Purpose: To perform integer arithmetic within a shell script using the \$(()) arithmetic expansion syntax, covering addition, subtraction, multiplication, integer division, modulo, and exponentiation.

Command: vim arith.sh → chmod +x arith.sh → ./arith.sh

Script:

```

a=10
b=3

echo "Addition: $((a+b))"
echo "Substraction: $((a-b))"
echo "Multiplication: $((a*b))"
echo "Division: $((a/b))"
echo "Modulus: $((a%b))"
echo "Exponent: $((a**b))"

```

Screenshot:

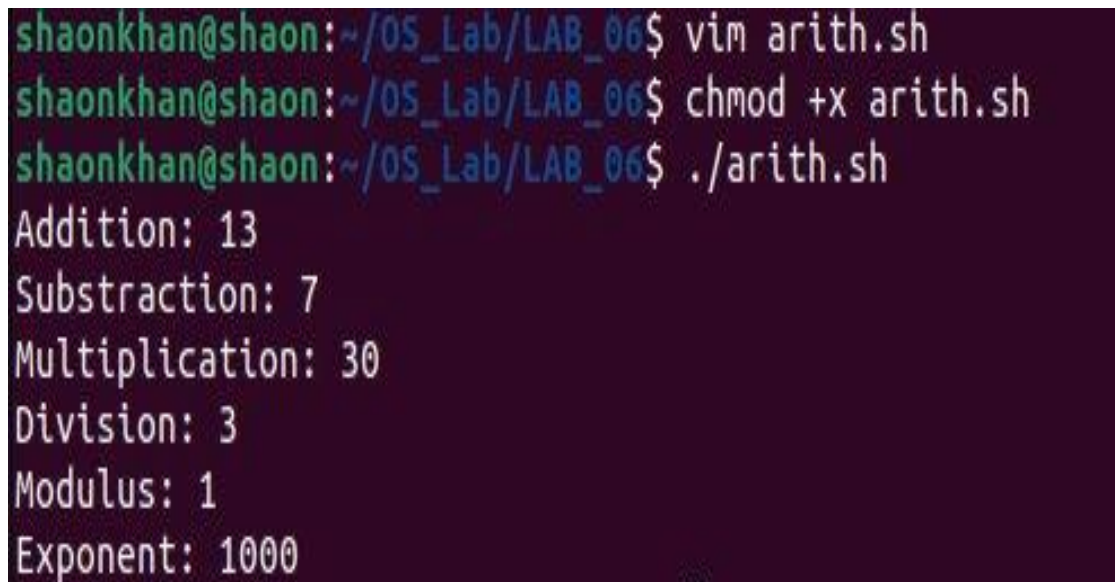

```
a=10
b=3

echo "Addition: $((a+b))"
echo "Substraction: $((a-b))"
echo "Multiplication: $((a*b))"
echo "Division: $((a/b))"
echo "Modulus: $((a%b))"
echo "Exponent: $((a**b))"
```

Fig 4.15(a) — vim editor showing arith.sh with all arithmetic operators

Output:

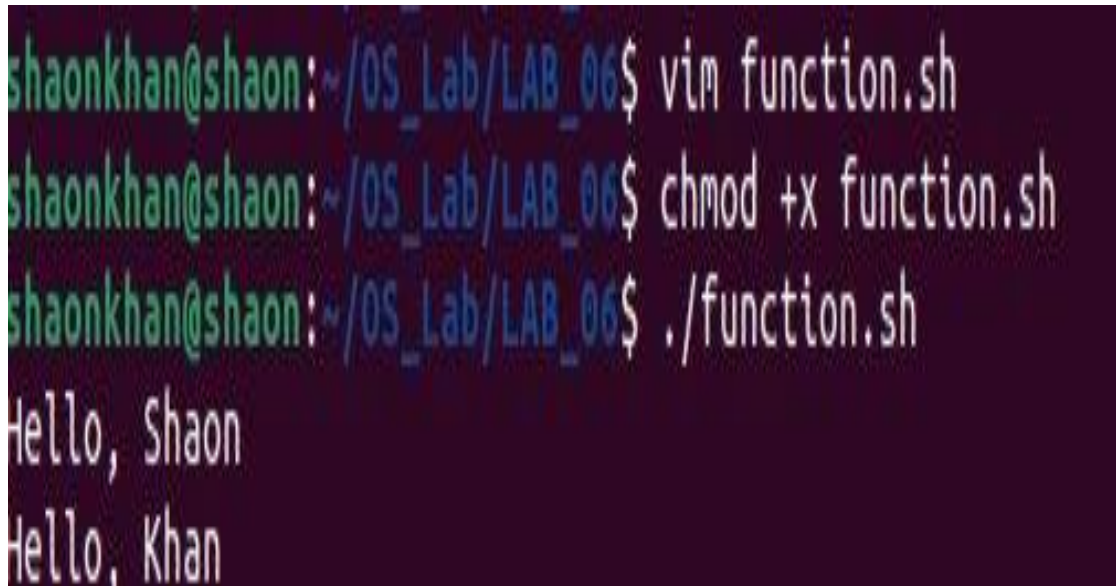
```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim arith.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x arith.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./arith.sh
Addition: 13
Substraction: 7
Multiplication: 30
Division: 3
Modulus: 1
Exponent: 1000
```

Screenshot:


```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim arith.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x arith.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./arith.sh
Addition: 13
Substraction: 7
Multiplication: 30
Division: 3
Modulus: 1
Exponent: 1000
```

Fig 4.15(b) — Terminal output of arith.sh showing all six arithmetic results

Observation: All six operators produced correct results: $10+3=13$, $10-3=7$, $10\times 3=30$, $10/3=3$ (integer division, truncated), $10\%3=1$ (remainder), and $10**3=1000$. The $\$(())$ syntax is evaluated natively by Bash without any external tools. Division truncates the decimal portion since Bash supports integers only — $10/3$ gives 3, not 3.33. For floating-point results, the `bc` utility would be needed.

Screenshot:


```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim function.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x function.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./function.sh
Hello, Shaon
Hello, Khan

```

Fig 4.16(b) — Terminal output showing greet() called with two different arguments

Observation: The greet() function was defined once and called twice with different arguments. Inside the function, \$1 refers to the first argument passed at the call site — 'Shaon' in the first call and 'Khan' in the second. This confirms that functions in Bash are reusable code blocks and that positional parameters inside a function refer to the function's own arguments, not the script's arguments.

4.17 return Statement and \$? (Exit Status)

Purpose: To demonstrate the return statement inside a function, which sets the function's exit status code, and the \$? special variable, which retrieves that code to communicate a computed result back to the caller.

Command: vim return.sh → chmod +x return.sh → ./return.sh

Script:

```

add() {
    sum=$(( $1 + $2 ))
    return $sum
}

add 5 3

echo "Sum = $?"

```

Screenshot:


```

add() {
    sum=$(( $1 + $2 ))
    return $sum
}

add 5 3

echo "Sum = $?"

:wq

```

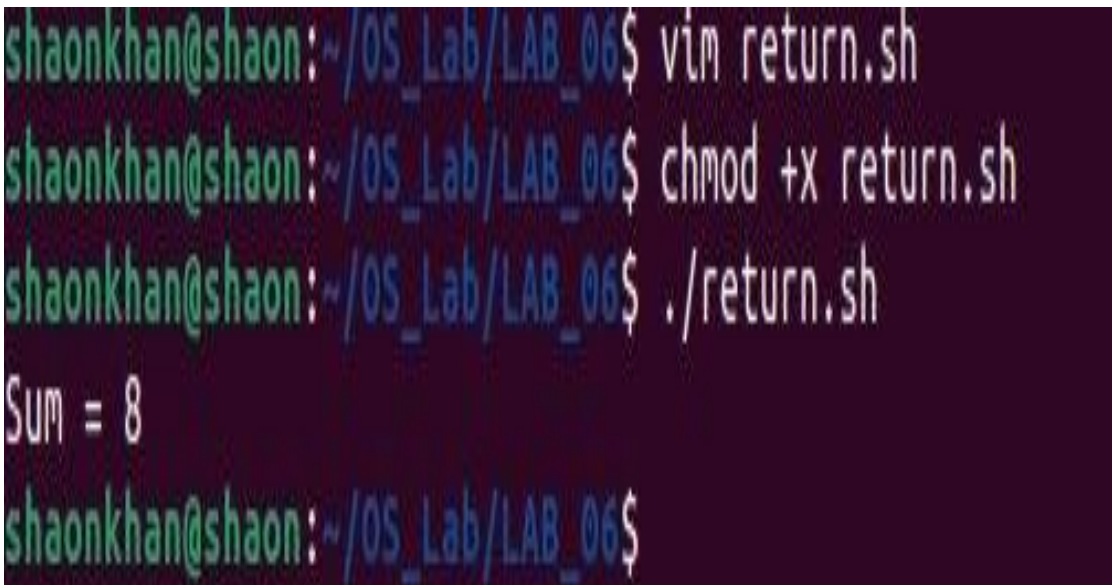
Fig 4.17(a) — vim editor showing return.sh with add() and \$? usage

Output:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim return.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x return.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./return.sh
Sum = 8

```

Screenshot:


```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim return.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x return.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./return.sh
Sum = 8
shaonkhan@shaon:~/OS_Lab/LAB_06$

```

Fig 4.17(b) — Terminal output of return.sh showing Sum = 8 via \$?

Observation: The add() function computed $5+3=8$ and passed that value to return. Immediately after calling add 5 3, \$? held the return value 8, which was then printed. This demonstrates using return and \$? as a lightweight mechanism to pass small integer values from a function to the caller. It should be noted that return is limited to values 0–255 as it is designed for exit status codes; for larger or non-integer results, echoing from the function and using command substitution is the standard approach.

5. Discussion

This laboratory session provided comprehensive hands-on experience with Bash shell scripting. Seventeen subsections covered the full range of fundamental scripting concepts, and several important observations emerged from the practical work.

Script Creation and Execution: All scripts were created using vim and executed via `./script.sh` after applying `chmod +x`. The deliberate `chmod -x` experiment in subsection 4.1 produced a 'Permission denied' error, confirming that the execute bit is mandatory for `./execution` but not for `sh script.sh`. The `while.sh` syntax error (missing spaces in the test condition) and the `breakcon.sh` bracket error were valuable debugging experiences, demonstrating that Bash test expressions are highly sensitive to spacing and correct bracket syntax.

Variables and Environment Variables: User-defined variables (`name`, `num`, `count`) required no type declaration and were assigned without spaces around `=`. Environment variables (`$HOME`, `$SHELL`) were available automatically without any declaration, set by the OS at login and inherited by the script process. Both categories were accessed using the `$` prefix within double quotes, which enables variable expansion at runtime.

Output Formatting: `echo -e` and `printf` produced identical multi-line output by interpreting `\n`, confirming that `echo` without `-e` treats escape sequences as literal text. `echo -n` suppressed newlines in `for.sh` to print all numbers on a single line. `printf` is preferred for tabular output due to its format specifier support.

Command Substitution: `$(date)` captured a live timestamp including Bangladesh Standard Time (+06), demonstrating that command substitution creates dynamic, system-aware scripts. The output changed on every execution, unlike hardcoded strings.

User Input: `read` captured the full input line including spaces, as verified by testing with 'Shaon' and 'Shaon Khan'. This makes it suitable for collecting multi-word data such as full names.

Conditional Logic: The progression from `if` → `if-else` → `if-elif-else` was clearly demonstrated. The grading script confirmed that `elif` conditions are evaluated sequentially and only the first match executes. The case statement provided a clean pattern-matching alternative, handling the `y/n` input elegantly with the `*` wildcard catching all unrecognised input.

Loops: The `for` loop (list-based and C-style), `while`, and `until` loops were all successfully demonstrated. The `while.sh` syntax error highlighted the importance of correct spacing in Bash conditionals. The `until` loop produced results identical to an equivalent `while` loop but with the inverted condition, confirming they are logical complements. The `break` experiment in `breakcon.sh` revealed a common syntax pitfall — a missing closing bracket — that caused warnings on every iteration without terminating the script, demonstrating that Bash is partially tolerant of test expression errors.

Arithmetic: All six arithmetic operators (`+`, `-`, `*`, `/`, `%`, `**`) worked correctly inside `$(( ))`. Integer division truncated `10/3` to 3, and exponentiation correctly computed `10**3=1000`. The `$(( ))` syntax required no external tools, confirming it as the native Bash integer arithmetic mechanism.

Functions: The `greet()` function confirmed that `$1` inside a function refers to the function's own argument. The `return/$?` pattern in `return.sh` successfully passed the computed sum (8) back to the caller, though the 0–255 limitation of `return` means this technique is only suitable for small integers. For general data passing, `echo` combined with command substitution is the recommended approach.

6. Conclusion

This laboratory session on Introduction to Shell Scripting (Lab No. 06, ICT-3102) successfully covered all major Bash scripting concepts through seventeen practical subsections. Starting from a single-line welcome script and progressing through variables, environment variables, formatted output, command substitution, user input, all conditional constructs, four loop types, arithmetic expansion, and functions with return values, the session provided a complete and coherent introduction to shell scripting.

Several practical lessons were reinforced through real errors encountered during the session. The `chmod -x` experiment confirmed the execute bit requirement. The `while.sh` syntax error demonstrated Bash's sensitivity to spacing inside `[ ]` test conditions. The `breakcon.sh` missing-bracket issue showed that malformed conditions produce runtime warnings and incorrect behaviour rather than terminating the script. These debugging experiences are as valuable as the successful experiments, building the diagnostic skills essential for real-world scripting work.

The arithmetic experiments confirmed that `$(( ))` handles all standard integer operations natively, including exponentiation, while the function experiments established that `$1/$2` positional parameters are scoped to the function and that `return/$?` provides a lightweight integer return channel. The `until` loop was demonstrated to be the logical inverse of `while`, and the `case` statement proved more readable than chained `if-elif` blocks for fixed-value matching.

Overall, this session has built a strong, practical foundation in Bash shell scripting. The skills acquired — script creation, permission management, variable handling, input/output, conditional logic, iterative constructs, arithmetic, and modular functions — are directly applicable to system administration, task automation, and software development in any Unix/Linux environment.



**INSTITUTE OF INFORMATION TECHNOLOGY
JAHANGIRNAGAR UNIVERSITY
SAVAR, DHAKA-1342**

OPERATING SYSTEM LAB

Course Code: ICT-3102

LAB REPORT NO: 07

Lab Title: Process Scheduling Algorithms

Submitted by:

Md. Shaon Khan

Student ID: 1984

52nd Batch | Session: 2022-2023

Supervised by:

Mehrin Anannya

Assistant Professor

Institute of Information Technology

Jahangirnagar University

Submission Date: July 15, 2026

1. Objective

In this lab I worked on process scheduling, which is basically how the OS decides which process gets the CPU next when there are several of them waiting. The main point was to actually implement three scheduling algorithms myself - FCFS, SJF (Non-Preemptive), and SRTF (the preemptive version of SJF) - and see how each one behaves for the same set of processes.

Along with the implementation, I also had to calculate the usual performance metrics (waiting time and turnaround time) both by hand and through the program, so that I could check whether my code was actually giving correct results. Doing it this way also made it easier to compare the three algorithms and see where each one is strong and where it falls short.

2. Introduction

Process scheduling is one of the core jobs of an operating system. At any given moment a computer usually has more processes wanting to run than it has CPU cores available, so the OS has to pick and choose - this is exactly what a scheduler does. If this wasn't managed properly, the system would either freeze up or become extremely unfair to certain processes.

A few terms that come up repeatedly in this topic:

- **Process:** a program that is currently being executed. It is the basic unit of work for the OS.
- **Process Scheduling:** the decision-making process the OS uses to figure out which waiting process gets CPU time next, and for how long.
- **Ready Queue:** the queue that holds every process which is ready to run but is currently waiting for the CPU.
- **CPU Scheduler:** picks a process out of the ready queue and hands the CPU over to it.

Without any scheduling policy at all, one long process could hog the CPU and everything else would just sit there waiting, which is obviously bad for a multi-tasking system. A good scheduler tries to keep CPU utilization high, keep waiting times reasonably low, and still treat every process somewhat fairly - these are more or less the goals I kept in mind while testing my three implementations below.

3. Basic Terminologies

Before jumping into the algorithms themselves, here are the metrics that were used throughout this report to judge each scheduler:

- **Arrival Time (AT):** the time at which a process shows up in the ready queue.
- **Burst Time (BT):** how much CPU time the process needs in total to finish running.
- **Completion Time (CT):** the time at which the process actually finishes and leaves the system.
- **Turnaround Time (TAT):** total time the process spends in the system, from arrival to completion (waiting + running).

$$TAT = CT - AT$$

- **Waiting Time (WT):** time spent by the process just sitting in the ready queue, not being executed.

$$WT = TAT - BT$$

- **Average Waiting Time:** sum of all the individual waiting times, divided by the number of processes. Lower is generally better.

$$\text{Average } WT = (\sum WT) / n$$

- **Average Turnaround Time:** same idea, but for turnaround times.

$$\text{Average } TAT = (\sum TAT) / n$$

4. FCFS (First Come First Serve)

Definition

FCFS is the most basic non-preemptive scheduling algorithm out there - whichever process asks for the CPU first simply gets it first, in the exact order the processes lined up in the ready queue.

Rule

Processes run purely by arrival order. Once a process gets hold of the CPU, it keeps running until it's completely done - nobody can cut in line.

Advantages

- Very simple to understand, and honestly not too hard to code either, since it's basically just a queue.
- Fair in terms of order - no process gets special treatment just because it's shorter or more important.
- Low overhead for the OS because there's no complicated logic involved in picking the next process.

Disadvantages

- Suffers badly from what's called the Convoy Effect - if a big process arrives first, every short process behind it has to wait way longer than it should.
- Average waiting time tends to be worse compared to SJF/SRTF.
- Not preemptive at all, so it's not really practical for interactive or time-sharing systems.

Numerical Example

Using the same three-process dataset given in the lab manual:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	5
P2	1	3
P3	2	8

Manual Execution

- t = 0: P1 arrives and since the CPU is free, it starts right away. Finishes at t = 5 (0 + 5).
- t = 5: P2 had already arrived at t = 1, so it goes next. Finishes at t = 8 (5 + 3).
- t = 8: P3 (arrived at t = 2) is the only one left, runs last and finishes at t = 16 (8 + 8).

Calculated Metrics

Process	AT	BT	CT	TAT (CT-AT)	WT (TAT-BT)
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6

$$\text{Average Waiting Time} = (0 + 4 + 6) / 3 = 3.33$$

$$\text{Average Turnaround Time} = (5 + 7 + 14) / 3 = 8.67$$

Algorithm

1. Start.
2. Take the number of processes n and read AT, BT for each one.
3. Sort all the processes by arrival time (ascending).
4. Set $\text{current_time} = 0$.
5. For every process in the sorted order: if $\text{current_time} < \text{AT}$, jump current_time forward to AT (this represents the CPU sitting idle).
6. $\text{CT} = \text{current_time} + \text{BT}$
7. $\text{TAT} = \text{CT} - \text{AT}$, and $\text{WT} = \text{TAT} - \text{BT}$
8. Add these to the running totals, print the row, then set $\text{current_time} = \text{CT}$ and move to the next process.
9. Once every process is handled, divide the totals by n to get the two averages and print them.

Python Code

```
n = int(input("Enter number of processes: "))
processes = []

for i in range(n):
    at = int(input(f"Enter Arrival Time (AT) for P{i+1}: "))
    bt = int(input(f"Enter Burst Time (BT) for P{i+1}: "))
    processes.append([i + 1, at, bt])

processes.sort(key=lambda x: x[1])
current_time = 0
total_wt = 0
total_tat = 0

print("\nProcess\tAT\tBT\tCT\tTAT\tWT")

for process in processes:
    pid, at, bt = process

    if current_time < at:
        current_time = at

    ct = current_time + bt
    tat = ct - at
    wt = tat - bt
    total_wt += wt
    total_tat += tat

    print(f"P{pid}\t{at}\t{bt}\t{ct}\t{tat}\t{wt}")
    current_time = ct

avg_wt = total_wt / n
avg_tat = total_tat / n

print(f"\nAverage Waiting Time = {avg_wt:.2f}")
print(f"Average Turnaround Time = {avg_tat:.2f}")
```

Output

```
Enter number of processes: 3
Enter Arrival Time (AT) for P1: 0
Enter Burst Time (BT) for P1: 5
Enter Arrival Time (AT) for P2: 1
Enter Burst Time (BT) for P2: 3
Enter Arrival Time (AT) for P3: 2
Enter Burst Time (BT) for P3: 8
```

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6

```
Average Waiting Time = 3.33
Average Turnaround Time = 8.67
```

Explanation

The code follows the algorithm pretty closely. Processes get sorted by arrival time first using a lambda, and then `current_time` keeps track of where we are on the timeline as the loop moves through the list. There's a small check for whenever a process hasn't arrived yet - in that case `current_time` just jumps ahead to its AT so the idle gap is accounted for. CT, TAT and WT are all computed one process at a time and added to running totals, and the final averages printed by the program matched exactly what I got when I worked it out by hand above.

5. SJF (Non-Preemptive)

Definition

SJF (Non-Preemptive) hands the CPU to whichever available process has the smallest burst time. Once it starts running though, it's the same as FCFS in the sense that it can't be interrupted - it has to finish before anything else gets a turn.

Rule

Among all processes that have already arrived by the current time, the scheduler always chooses the one with the smallest burst time.

Advantages

- Gives the best possible average waiting time out of all the non-preemptive algorithms, for a fixed set of processes.
- Works well in batch systems where burst times are already known beforehand.
- Turnaround time tends to be lower overall since shorter jobs clear out of the system fast.

Disadvantages

- Long processes can starve if short ones keep showing up one after another.
- Not very realistic for real-time systems, because you rarely know a process's exact burst time in advance.

Numerical Example

Same three processes again:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	5
P2	1	3
P3	2	8

Manual Execution

- $t = 0$: only P1 has arrived so far, so even though it isn't the shortest job overall, it's the only option. Runs and finishes at $t = 5$.
- $t = 5$: now both P2 (BT=3) and P3 (BT=8) are waiting. P2 is shorter, so it goes next and finishes at $t = 8$ ($5 + 3$).
- $t = 8$: P3 is the only one left. Runs to completion, finishing at $t = 16$ ($8 + 8$).

Interesting side note - for this particular dataset the SJF order ends up being identical to FCFS, since P1 happens to be the only process available at $t = 0$ anyway. The metrics table below comes out exactly the same as the FCFS one.

Calculated Metrics

Process	AT	BT	CT	TAT (CT-AT)	WT (TAT-BT)
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6

$$\text{Average Waiting Time} = (0 + 4 + 6) / 3 = 3.33$$

$$\text{Average Turnaround Time} = (5 + 7 + 14) / 3 = 8.67$$

Algorithm

10. Start.
11. Read n, then AT and BT for each process.
12. Create a completed[] boolean array of size n, all False initially.
13. current_time = 0, completed_count = 0, and reset the total WT/TAT counters.
14. While completed_count < n: look through the process list for one that has arrived (AT ≤ current_time), isn't completed yet, and has the smallest BT among those candidates.
15. If nothing qualifies, that means the CPU is idle - just move current_time forward by 1 and check again.
16. Otherwise, for the chosen process: CT = current_time + BT, TAT = CT - AT, WT = TAT - BT.
17. Add WT/TAT to the totals, print the process details, set current_time = CT, mark it completed, and increment completed_count.
18. After the loop finishes, compute and display the two averages.

Python Code

```
n = int(input("Enter number of processes: "))
processes = []

for i in range(n):
    at = int(input(f"Enter Arrival Time (AT) for P{i+1}: "))
    bt = int(input(f"Enter Burst Time (BT) for P{i+1}: "))
    processes.append([i + 1, at, bt])

completed = [False] * n

current_time = 0
completed_count = 0
total_wt = 0
total_tat = 0

print("\nProcess\tAT\tBT\tCT\tTAT\tWT")

while completed_count < n:
    idx = -1
    min_bt = float("inf")

    for i in range(n):
        pid, at, bt = processes[i]
        if at <= current_time and not completed[i] and bt < min_bt:
            min_bt = bt
            idx = i

    if idx == -1:
        current_time += 1
        continue

    pid, at, bt = processes[idx]
    ct = current_time + bt
    tat = ct - at
    wt = tat - bt

    print(f"P{pid}\t{at}\t{bt}\t{ct}\t{tat}\t{wt}")

    total_wt += wt
    total_tat += tat

    current_time = ct
    completed[idx] = True
    completed_count += 1

avg_wt = total_wt / n
avg_tat = total_tat / n

print(f"\nAverage Waiting Time = {avg_wt:.2f}")
print(f"Average Turnaround Time = {avg_tat:.2f}")
```


Output

```
Enter number of processes: 3
Enter Arrival Time (AT) for P1: 0
Enter Burst Time (BT) for P1: 5
Enter Arrival Time (AT) for P2: 1
Enter Burst Time (BT) for P2: 3
Enter Arrival Time (AT) for P3: 2
Enter Burst Time (BT) for P3: 8
```

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6

```
Average Waiting Time = 3.33
Average Turnaround Time = 8.67
```

Explanation

Instead of sorting everything once at the start like the FCFS version, this program checks the ready pool on every iteration of the loop, which is closer to how SJF is supposed to behave when multiple processes have already arrived. It scans for every process whose AT is less than or equal to current_time and isn't finished yet, then just keeps the one with the smallest BT. If nothing has arrived, a continue statement pushes current_time ahead by one unit until something becomes available. Running it against the manual example above gave identical numbers, so the logic checks out.

6. SJF (Preemptive / SRTF)

Definition

SRTF is basically SJF but preemptive - at any moment the CPU goes to whichever process has the least remaining burst time, even if that means kicking out a process that's already running.

Rule

At every time unit the scheduler re-checks the ready queue. If a process just arrived and its remaining time is less than what's left on the currently running process, the running process gets preempted and the new one takes over.

What is Preemption?

Preemption just means the OS can pause a running process mid-way and give the CPU to something else, without the first process agreeing to it or finishing what it was doing.

Advantages

- Gives the lowest possible average waiting time among these three algorithms.
- Very responsive - a short process barely has to wait once it arrives.
- Throughput stays high because short jobs are cleared out quickly.

Disadvantages

- Long processes can starve badly if short jobs keep flooding in.
- Lots of context switching overhead since the CPU keeps swapping between processes.
- Needs constant tracking of remaining burst time for every process, which adds complexity.

Numerical Example

Same three processes once more:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	5
P2	1	3
P3	2	8

Manual Execution

- $t = 0$: only P1 is around ($BT = 5$), so it starts running.
- $t = 1$: P2 arrives with $BT = 3$. P1's remaining time by now is 4 ($5 - 1$). Since $3 < 4$, P1 gets preempted and P2 takes over.
- $t = 2$: P3 arrives with $BT = 8$. Comparing remaining times - $P1 = 4$, $P2 = 2$, $P3 = 8$ - P2 is still the smallest, so it keeps running.
- $t = 4$: P2 finishes completely. Remaining processes are $P1 = 4$ and $P3 = 8$, so P1 resumes.
- $t = 8$: P1 finally finishes. P3 is the only one left, runs straight through and finishes at $t = 16$ ($8 + 8$).

Calculated Metrics

Process	AT	BT	CT	TAT (CT-AT)	WT (TAT-BT)
P1	0	5	8	8	3
P2	1	3	4	3	0
P3	2	8	16	14	6

$$\text{Average Waiting Time} = (3 + 0 + 6) / 3 = 3.00$$

$$\text{Average Turnaround Time} = (8 + 3 + 14) / 3 = 8.33$$

Compared to FCFS and SJF above, the average waiting time here dropped from 3.33 to exactly 3.00, which lines up with the theory that SRTF should give the best (or tied-best) average waiting time of the three.

Algorithm

19. Start.
20. Read n , then AT and BT for each process.
21. Copy BT into a separate `remaining_bt[]` array so the original burst times stay untouched for the final metric calculations.
22. Initialize `ct[]`, `tat[]`, `wt[]` to zero, and set `current_time = 0`, `completed = 0`.
23. While `completed < n`: find the process with $AT \leq \text{current_time}$, `remaining_bt > 0`, and the smallest `remaining_bt` value among those.
24. If none is found, the CPU is idle - just bump `current_time` up by 1 and loop again.
25. Otherwise: decrement its `remaining_bt` by 1, increment `current_time` by 1.
26. If `remaining_bt` hits exactly 0: increment `completed`, then set $CT = \text{current_time}$, $TAT = CT - AT$, $WT = TAT - BT$ for that process.
27. Once the loop ends, print each process's metrics and sum up WT/TAT to get the averages.

Python Code

```
n = int(input("Enter number of processes: "))
processes = []

for i in range(n):
    at = int(input(f"Enter Arrival Time (AT) for P{i+1}: "))
    bt = int(input(f"Enter Burst Time (BT) for P{i+1}: "))
    processes.append([i + 1, at, bt])

remaining_bt = [process[2] for process in processes]

ct = [0] * n
tat = [0] * n
wt = [0] * n

current_time = 0
completed = 0

while completed < n:
    idx = -1
    min_bt = float("inf")

    for i in range(n):
        pid, at, bt = processes[i]
        if at <= current_time and remaining_bt[i] > 0 and remaining_bt[i] < min_bt:
            min_bt = remaining_bt[i]
            idx = i

    if idx == -1:
        current_time += 1
        continue
```

```

    remaining_bt[idx] -= 1
    current_time += 1

    if remaining_bt[idx] == 0:
        completed += 1
        pid, at, bt = processes[idx]

        ct[idx] = current_time
        tat[idx] = ct[idx] - at
        wt[idx] = tat[idx] - bt

total_wt = 0
total_tat = 0

print("\nProcess\tAT\tBT\tCT\tTAT\tWT")

for i in range(n):
    pid, at, bt = processes[i]

    print(f"P{pid}\t{at}\t{bt}\t{ct[i]}\t{tat[i]}\t{wt[i]}")

    total_wt += wt[i]
    total_tat += tat[i]

avg_wt = total_wt / n
avg_tat = total_tat / n

print(f"\nAverage Waiting Time = {avg_wt:.2f}")
print(f"Average Turnaround Time = {avg_tat:.2f}")

```

Output

```

Enter number of processes: 3
Enter Arrival Time (AT) for P1: 0
Enter Burst Time (BT) for P1: 5
Enter Arrival Time (AT) for P2: 1
Enter Burst Time (BT) for P2: 3
Enter Arrival Time (AT) for P3: 2
Enter Burst Time (BT) for P3: 8

```

Process	AT	BT	CT	TAT	WT
P1	0	5	8	8	3
P2	1	3	4	3	0
P3	2	8	16	14	6

```

Average Waiting Time = 3.00
Average Turnaround Time = 8.33

```

Explanation

This one runs the simulation one time unit at a time rather than jumping straight to a process's completion, which is really the only way to model preemption properly. `remaining_bt` keeps the live/updated burst times while the original BT values stay untouched for the final table. On each tick, the loop picks whichever arrived process has the smallest `remaining_bt`, runs it for exactly one unit, and checks if it just hit zero - if so, its CT/TAT/WT get recorded right there. It's a bit more expensive computationally than the other two versions since it re-checks the whole ready pool every single unit of time, but the output matched my manual working out exactly.

7. Comparison of Algorithms

After running all three, here's how I'd sum up the differences between them:

Feature	FCFS	SJF (Non-Preemptive)	SRTF (Preemptive SJF)
Preemptive	No	No	Yes
Selection Basis	Arrival Time	Shortest Burst Time	Shortest Remaining Time
Waiting Time	High	Low	Lowest
Complexity	Easy	Medium	High
Starvation	No	Possible	Possible

8. Result

All three algorithms - FCFS, SJF (Non-Preemptive), and SRTF - were implemented and tested successfully in Python. Each program correctly took in the number of processes along with their arrival and burst times, and worked out the execution order on its own. CT, TAT and WT were calculated for every process in all three cases, and every single value matched the numbers I had already worked out manually, so I'm fairly confident the implementations are correct.

9. Conclusion

Doing this lab made the theory behind CPU scheduling click a lot more than just reading about it. Writing FCFS first made the basic idea obvious, but it was really SJF and especially SRTF that showed why preemption matters - the average waiting time kept dropping as the algorithms got smarter (3.33 → 3.33 → 3.00 for this dataset), which is exactly what the theory predicts, even if the improvement looks small for only three processes. Overall this gave me a much better feel for the trade-off between simplicity and efficiency when it comes to scheduling, and for how the OS actually decides who gets the CPU next.

Institute of Information Technology
Jahangirnagar University
Savar, Dhaka-1342



Lab Report : 8

Lab Title: Process Scheduling Algorithms

Course Code: ICT-3102
Course Name: Operating System Lab

Submitted to

Mehrin Anannya
Assistant Professor
Institute of Information Technology
Jahangirnagar University

Submitted By

Md. Shaon Khan
ID: 1984
Session: 2022-2023
Batch: 52

Submission Date: 15th July 2026

1. Objective

This lab experiment covers three CPU process scheduling algorithms used by operating systems: Longest Job First (non-preemptive), Longest Remaining Time First (preemptive), and Round Robin (preemptive). For each process, the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) are calculated, along with the average waiting time and average turnaround time. The lab also builds practical skill in coding a scheduling algorithm and in comparing how each algorithm affects overall system performance.

2. Introduction to the Algorithms

2.1 Longest Job First (LJF) — Non-Preemptive

Longest Job First is a non-preemptive scheduling algorithm. Among all processes that have already arrived in the ready queue, the process with the largest burst time is chosen to run next. Once a process starts, it runs until it finishes, without interruption. Only processes that have arrived by the current time are considered when choosing the next process.

Algorithm steps:

- Sort the processes in increasing order of Arrival Time.
- Among all processes that have arrived by the current time, choose the one with the highest Burst Time.
- Run the selected process to completion, without preemption.
- Repeat until all processes have finished.

2.2 Longest Remaining Time First (LRTF) — Preemptive

Longest Remaining Time First is the preemptive version of LJF. At every unit of time, the scheduler checks all arrived processes and runs the one with the largest remaining burst time. If a new process arrives with more remaining time than the process currently running, the CPU switches to the new process. This continues until every process has finished.

Algorithm steps:

- Sort the processes in increasing order of Arrival Time.
- At each time unit, select the process with the greatest remaining burst time among those that have arrived.
- Run the selected process for one unit of time and reduce its remaining time by one.
- Repeat this selection at every time unit, allowing preemption, until all processes finish.

2.3 Round Robin (RR) — Preemptive

Round Robin gives each process in the ready queue a fixed time slice, called the Time Quantum (TQ), and cycles through the processes in order. If a process does not finish within its quantum, its remaining burst time is carried over and it is placed back at the end of the queue. If it finishes, it is removed from the queue. Any process that arrives during execution is added to the ready queue and considered in later cycles.

Algorithm steps:

- Sort all processes in increasing order of Arrival Time.
- Assign a fixed Time Quantum (TQ) for execution.
- Starting from the first process in the ready queue, run it for at most TQ units.
- If the process finishes within the quantum, remove it from the queue. Otherwise, subtract the time used from its remaining burst time and place it at the end of the queue.
- Add any newly arrived processes to the ready queue as they arrive.
- Repeat until all processes are finished, then compute the Average Waiting Time and Average Turnaround Time.

3. Source Code and Results

The same input set was used to test all three algorithms, so that their performance could be compared directly:

Process	AT	BT	CT	TAT	WT
P1	0	5	-	-	-
P2	1	3	-	-	-
P3	2	8	-	-	-
P4	3	6	-	-	-

3.1 Longest Job First (LJF)

The following Python program implements the non-preemptive LJF scheduling algorithm:

```
n = int(input("Enter number of processes: "))
pid = []
at = []
bt = []
ct = [0] * n
tat = [0] * n
wt = [0] * n
done = [False] * n
avgWT = 0
avgTAT = 0

for i in range(n):
    pid.append(i + 1)
    arrival, burst = map(int, input(f"Enter Arrival Time and Burst Time for P{i+1}: ").split())
    at.append(arrival)
    bt.append(burst)

completed = 0
current = 0
while completed < n:
    idx = -1
    maxBT = -1
```

```

for i in range(n):
    if not done[i] and at[i] <= current:
        if bt[i] > maxBT:
            maxBT = bt[i]
            idx = i
    if idx == -1:
        current += 1
        continue
    current += bt[idx]
    ct[idx] = current
    tat[idx] = ct[idx] - at[idx]
    wt[idx] = tat[idx] - bt[idx]
    avgWT += wt[idx]
    avgTAT += tat[idx]
    done[idx] = True
    completed += 1

print("\nProcess\tAT\tBT\tCT\tTAT\tWT")
for i in range(n):
    print(f"P{pid[i]}\t{at[i]}\t{bt[i]}\t{ct[i]}\t{tat[i]}\t{wt[i]}")

print(f"\nAverage Waiting Time = {avgWT/n:.2f}")
print(f"Average Turnaround Time = {avgTAT/n:.2f}")

```

Output for Arrival Times (0, 1, 2, 3) and Burst Times (5, 3, 8, 6):

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	22	21	18
P3	2	8	13	11	3
P4	3	6	19	16	10

```

Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 5
Enter Arrival Time and Burst Time for P2: 1 3
Enter Arrival Time and Burst Time for P3: 2 8
Enter Arrival Time and Burst Time for P4: 3 6

Process AT    BT    CT    TAT    WT
P1      0      5      5      5      0
P2      1      3     22     21     18
P3      2      8     13     11      3
P4      3      6     19     16     10

Average Waiting Time = 7.75
Average Turnaround Time = 13.25

```

Fig 3.1: Console output of the LJF program

Average Waiting Time = 7.75 ms Average Turnaround Time = 13.25 ms

3.2 Longest Remaining Time First (LRTF)

The following Python program implements the preemptive LRTF scheduling algorithm:

```
n = int(input("Enter number of processes: "))
pid = []
at = []
bt = []
rt = []
ct = [0] * n
tat = [0] * n
wt = [0] * n
avgWT = 0
avgTAT = 0

for i in range(n):
    pid.append(i + 1)
    arrival, burst = map(int, input(f"Enter Arrival Time and Burst Time for P{i+1}: ").split())
    at.append(arrival)
    bt.append(burst)
    rt.append(burst)

completed = 0
current = 0
while completed < n:
    idx = -1
    maxRT = -1
    # Find the process with the longest remaining time
    for i in range(n):
        if rt[i] > 0 and at[i] <= current:
            if rt[i] > maxRT:
                maxRT = rt[i]
                idx = i
    # If no process has arrived yet
    if idx == -1:
        current += 1
        continue
    # Run the selected process for 1 unit of time
    rt[idx] -= 1
    current += 1
    # If the process finishes
    if rt[idx] == 0:
        ct[idx] = current
        tat[idx] = ct[idx] - at[idx]
        wt[idx] = tat[idx] - bt[idx]
        avgWT += wt[idx]
        avgTAT += tat[idx]
        completed += 1

print("\nProcess\tat\tbt\tCT\tTAT\tWT")
for i in range(n):
    print(f"P{pid[i]}\t{at[i]}\t{bt[i]}\t{ct[i]}\t{tat[i]}\t{wt[i]}")

print(f"\nAverage Waiting Time = {avgWT / n:.2f}")
print(f"Average Turnaround Time = {avgTAT / n:.2f}")
```

Output for the same set of processes:

Process	AT	BT	CT	TAT	WT
P1	0	5	19	19	14
P2	1	3	20	19	16
P3	2	8	21	19	11
P4	3	6	22	19	13

```

Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 5
Enter Arrival Time and Burst Time for P2: 1 3
Enter Arrival Time and Burst Time for P3: 2 8
Enter Arrival Time and Burst Time for P4: 3 6

Process AT      BT      CT      TAT      WT
P1      0       5      19      19      14
P2      1       3      20      19      16
P3      2       8      21      19      11
P4      3       6      22      19      13

Average Waiting Time = 13.50

```

Fig 3.2: Console output of the LRTF program

Average Waiting Time = 13.50 ms Average Turnaround Time = 19.00 ms

3.3 Round Robin (RR)

The following Python program implements the Round Robin scheduling algorithm with a time quantum entered by the user:

```

n = int(input("Enter number of processes: "))
pid = []
at = []
bt = []
rt = []
ct = [0] * n
tat = [0] * n
wt = [0] * n
avgWT = 0
avgTAT = 0

for i in range(n):
    pid.append(i + 1)
    arrival, burst = map(int, input(f"Enter Arrival Time and Burst Time for P{i+1}: ").split())
    at.append(arrival)
    bt.append(burst)
    rt.append(burst)

```

```

tq = int(input("Enter Time Quantum: "))

completed = 0
current = 0
while completed < n:
    done = True
    for i in range(n):
        if rt[i] > 0 and at[i] <= current:
            done = False
            if rt[i] > tq:
                current += tq
                rt[i] -= tq
            else:
                current += rt[i]
                rt[i] = 0
                ct[i] = current
                tat[i] = ct[i] - at[i]
                wt[i] = tat[i] - bt[i]
                avgWT += wt[i]
                avgTAT += tat[i]
                completed += 1
    if done:
        current += 1 # CPU idle

print("\nProcess\tAT\tBT\tCT\tTAT\tWT")
for i in range(n):
    print(f"P{pid[i]}\t{at[i]}\t{bt[i]}\t{ct[i]}\t{tat[i]}\t{wt[i]}")

print(f"\nAverage Waiting Time = {avgWT / n:.2f}")
print(f"Average Turnaround Time = {avgTAT / n:.2f}")

```

Output for the same set of processes with Time Quantum = 2:

Process	AT	BT	CT	TAT	WT
P1	0	5	16	16	11
P2	1	3	11	10	7
P3	2	8	22	20	12
P4	3	6	20	17	11

```

Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 5
Enter Arrival Time and Burst Time for P2: 1 3
Enter Arrival Time and Burst Time for P3: 2 8
Enter Arrival Time and Burst Time for P4: 3 6
Enter Time Quantum: 2

Process AT    BT    CT    TAT    WT
P1      0      5     16     16     11
P2      1      3     11     10      7
P3      2      8     22     20     12
P4      3      6     20     17     11

Average Waiting Time = 10.25
Average Turnaround Time = 15.75

```

Fig 3.3: Console output of the Round Robin program

Average Waiting Time = 10.25 ms Average Turnaround Time = 15.75 ms

4. Comparison of Results

The table below summarizes the average waiting time and average turnaround time obtained from each algorithm for the identical input set (AT = 0, 1, 2, 3 and BT = 5, 3, 8, 6):

Algorithm	Avg. Waiting Time (ms)	Avg. Turnaround Time (ms)
LJF (non-preemptive)	7.75	13.25
LRTF (preemptive)	13.50	19.00
Round Robin (TQ = 2)	10.25	15.75

For this input set, LJF gives the lowest average waiting and turnaround time, since only four short processes are involved and the largest jobs also happen to finish efficiently in this arrival pattern. LRTF performs worst here because repeatedly switching to the process with the most remaining work forces shorter jobs, such as P2, to wait a long time before finishing. Round Robin falls in between: the fixed quantum shares the CPU fairly among all processes, avoiding the long waits seen in LRTF, while still adding some extra waiting time from context switching compared to LJF.

5. Discussion

Longest Job First keeps the number of context switches low and works well when process burst times are known in advance, but it is unfair to shorter processes, which may wait a long time if longer jobs keep arriving. LRTF, the preemptive version, adapts to arriving processes but is even more likely to delay short processes, because a newly arrived long process can repeatedly interrupt whichever job is currently running. Round Robin avoids this problem entirely, since every process gets a turn within one queue cycle, which makes it well suited to time-sharing systems. Its performance does depend on the choice of time quantum: too small a quantum increases overhead from context switching, while too large a quantum makes Round Robin behave like First Come First Served.

6. Conclusion

In this lab, three CPU scheduling algorithms — Longest Job First, Longest Remaining Time First, and Round Robin — were implemented and tested on identical process sets. Comparing the completion, turnaround, and waiting times from each program showed how the choice of scheduling policy directly affects process wait times and overall responsiveness. The experiment also highlighted the trade-off between throughput-oriented, non-preemptive strategies like LJF and fairness-oriented, preemptive strategies like Round Robin, and why real operating systems favour quantum-based approaches such as Round Robin for interactive, time-shared workloads.

Institute of Information Technology
Jahangirnagar University
Savar, Dhaka-1342



Lab Report : 9

Lab Title: Banker's Algorithm

Course Code: ICT-3102
Course Name: Operating System Lab

Submitted to

Mehrin Anannya
Assistant Professor
Institute of Information Technology
Jahangirnagar University

Submitted By

Md. Shaon Khan
ID: 1984
Session: 2022-2023
Batch: 52

Submission Date: 15th July 2026

1. Objective

This session covers the Banker's Algorithm, a classic deadlock avoidance algorithm used in operating systems. The lab covers the theory behind the algorithm and includes a practical implementation to check whether a system is in a safe state.

2. Introduction to the Algorithm

2.1 Definition of Banker's Algorithm

The Banker's Algorithm is a deadlock avoidance algorithm. It simulates the allocation of resources to processes to check for a safe state. A state is "safe" if the system can allocate resources to each process, up to its maximum requested amount, in some order, while still avoiding deadlock. The algorithm is named after a bank that allocates credit to customers while making sure it never reaches a state where it cannot satisfy all outstanding credit requests.

2.2 Why the Banker's Algorithm?

Deadlock is a serious problem in operating systems, where two or more processes wait indefinitely, each holding a resource that another waiting process needs. Deadlock detection allows deadlocks to occur and then recovers from them. Deadlock avoidance instead prevents the system from ever entering an unsafe state. By running a safety check before granting each resource request, the algorithm makes sure the system always stays in a state where a safe sequence of execution exists.

2.3 Key Data Structures

- Available: a vector of length m (number of resource types) showing the number of available instances of each resource type.
- Max: an $n \times m$ matrix (n = number of processes) showing the maximum demand of each process.
- Allocation: an $n \times m$ matrix showing the number of resources of each type currently allocated to each process.
- Need: an $n \times m$ matrix showing the remaining resource need of each process, calculated as $\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$.

2.4 Steps of the Safety Algorithm

- Initialize $\text{Work} = \text{Available}$, and $\text{Finish}[i] = \text{false}$ for all processes i .
- Find an index i such that $\text{Finish}[i] == \text{false}$ and $\text{Need}[i] \leq \text{Work}$.
- If such a process is found, set $\text{Work} = \text{Work} + \text{Allocation}[i]$ and $\text{Finish}[i] = \text{true}$.
- Repeat steps 2–3 until no such process can be found.
- If $\text{Finish}[i] == \text{true}$ for all processes, the state is safe and the order of execution is a safe sequence. Otherwise, the system is unsafe.

3. Source Code and Results

The following Python program implements the Banker's Algorithm and its safety check:

```
def input_matrix(rows, cols, name):
    print(f"\nEnter the '{name}' matrix ({rows} x {cols}):")
    matrix = []
    for i in range(rows):
        row = list(map(int, input(f"P{i}: ").split()))
        matrix.append(row)
    return matrix

def calculate_need(max_matrix, allocation, n, m):
    need = [[max_matrix[i][j] - allocation[i][j] for j in range(m)] for i in range(n)]
    return need

def is_safe(available, max_matrix, allocation, need, n, m):
    work = available.copy()
    finish = [False] * n
    safe_sequence = []
    count = 0
    while count < n:
        found = False
        for i in range(n):
            if not finish[i]:
                # Check if Need[i] <= Work
                if all(need[i][j] <= work[j] for j in range(m)):
                    # Allocate resources, simulate process completion
                    for j in range(m):
                        work[j] += allocation[i][j]
                    safe_sequence.append(i)
                    finish[i] = True
                    found = True
                    count += 1
        if not found:
            # No process could be allocated -> unsafe state
            return False, []
    return True, safe_sequence

def main():
    n = int(input("Enter the number of processes: "))
    m = int(input("Enter the number of resource types: "))
    print(f"\nEnter the 'Available' vector ({m} resources):")
    available = list(map(int, input().split()))
    max_matrix = input_matrix(n, m, "Max")
    allocation = input_matrix(n, m, "Allocation")
    need = calculate_need(max_matrix, allocation, n, m)
    safe, sequence = is_safe(available, max_matrix, allocation, need, n, m)
    if safe:
        print("\nSystem is in a safe state.")
        print("Safe Sequence is:", " -> ".join(f"P{p}" for p in sequence))
    else:
        print("\nSystem is in an unsafe state. No safe sequence exists.")

if __name__ == "__main__":
```

```
main()
```

The program was tested with a system of 5 processes (P0–P4) and 3 resource types (A, B, C), with Available = (3, 3, 2):

Process	Allocation (A,B,C)	Max (A,B,C)	Need (A,B,C)
P0	0, 1, 0	7, 5, 3	7, 4, 3
P1	2, 0, 0	3, 2, 2	1, 2, 2
P2	3, 0, 2	9, 0, 2	6, 0, 0
P3	2, 1, 1	2, 2, 2	0, 1, 1
P4	0, 0, 2	4, 3, 3	4, 3, 1

```
Enter the number of processes: 5
Enter the number of resource types: 3

Enter the 'Available' vector (3 resources):
3 3 2

Enter the 'Max' matrix (5 x 3):
P0: 7 5 3
P1: 3 2 2
P2: 9 0 2
P3: 2 2 2
P4: 4 3 3

Enter the 'Allocation' matrix (5 x 3):
P0: 0 1 0
P1: 2 0 0
P2: 3 0 2
P3: 2 1 1
P4: 0 0 2

System is in a safe state.
Safe Sequence is: P1 -> P3 -> P4 -> P0 -> P2
```

Fig 9.1: Console output of the Banker's Algorithm program

The program correctly identifies the system as being in a safe state, and produces the safe sequence P1 → P3 → P4 → P0 → P2, which matches the manual calculation of the safety algorithm.

4. Discussion

The Banker's Algorithm shows how an operating system can avoid deadlock proactively, rather than detecting and recovering from it afterward. By maintaining the Available, Max, Allocation, and Need data structures, and repeatedly searching for a process whose remaining need can be met by the currently available resources, the algorithm builds a safe sequence one process at a time. If at any point no remaining process can have its need satisfied by the available resources, the system is declared unsafe, since granting further requests could lead to a circular wait. The main trade-off of this approach is the overhead of running the safety check before every resource request, but it guarantees that the system never enters a deadlocked state.

5. Conclusion

In this lab, the Banker's Algorithm was implemented in Python and verified against a standard 5-process, 3-resource-type example. The program correctly determined that the system was in a safe state and produced the safe sequence $P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2$, consistent with the manual trace of the safety algorithm. The experiment reinforced the practical value of deadlock avoidance, showing how a resource manager can check the safety of a state before committing to an allocation, which is essential in systems where multiple processes compete for limited, shared resources.